Assignment 3
FUMINI HOTEL MANAGEMENT SYSTEM (A3)

1. Thông tin project

1.1. Tên dự án

FUMINI HOTEL MANAGEMENT SYSTEM (A3)

Link GitHub: https://github.com/Michaelht271/SBA301-Spring-Boot-Application-With-ReactJS/tree/main/assigment/assignment3

1.2. Thông tin tác giả

FullName: Nguyễn Văn An

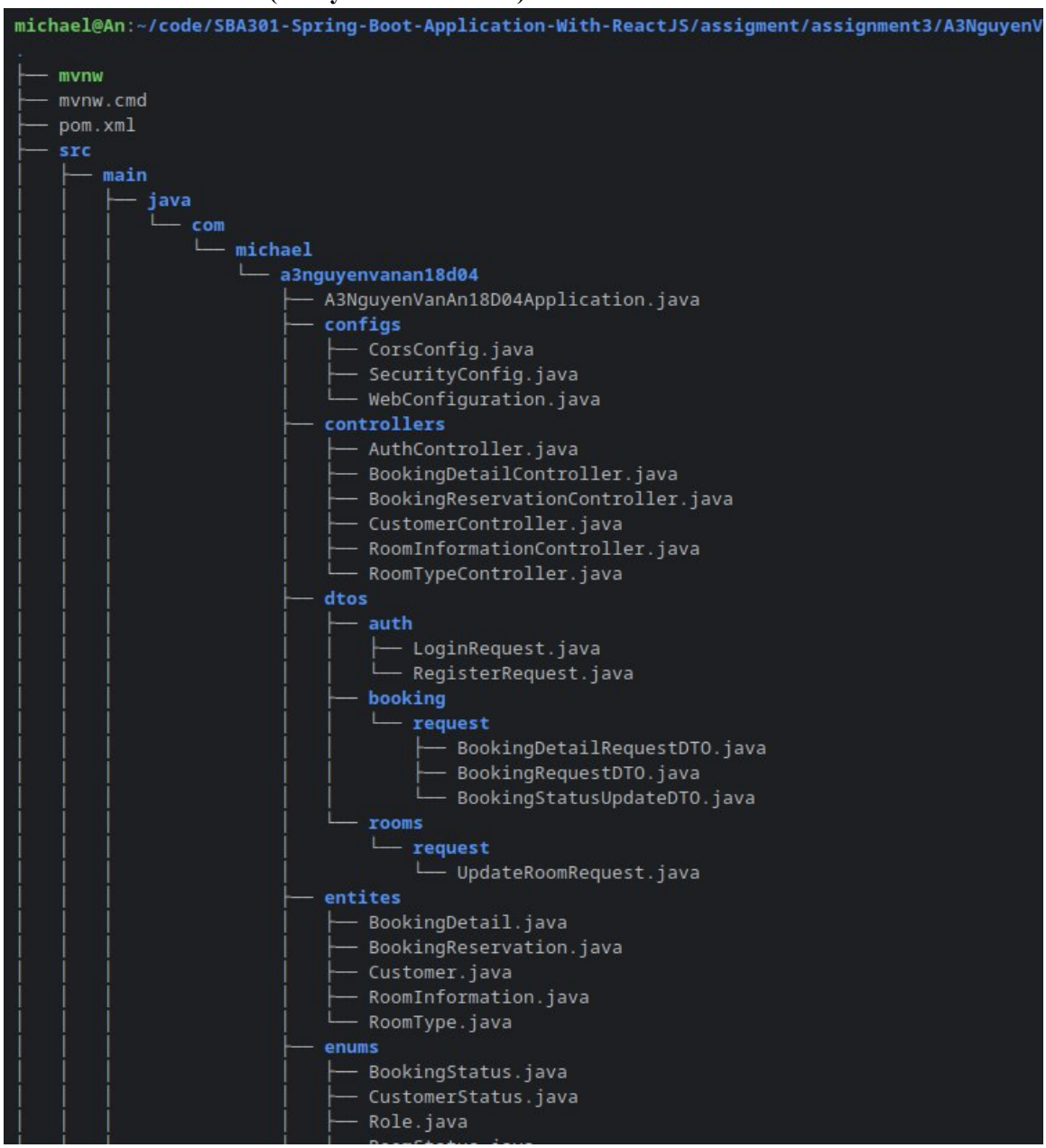
Class: SE18D04

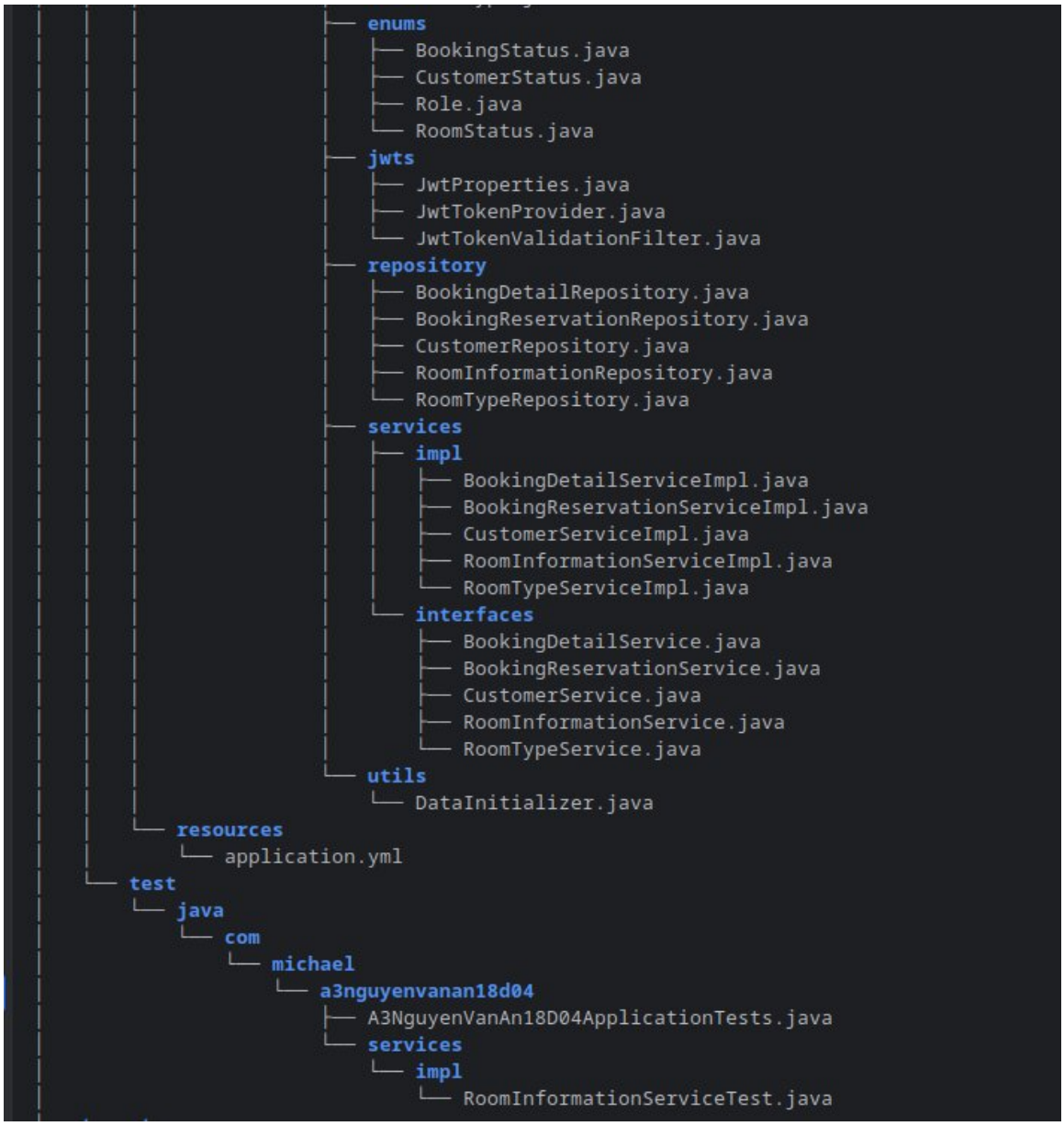
Subject: SBA301

Công nghệ: Spring Boot 3 + ReactJS + MS SQL Server

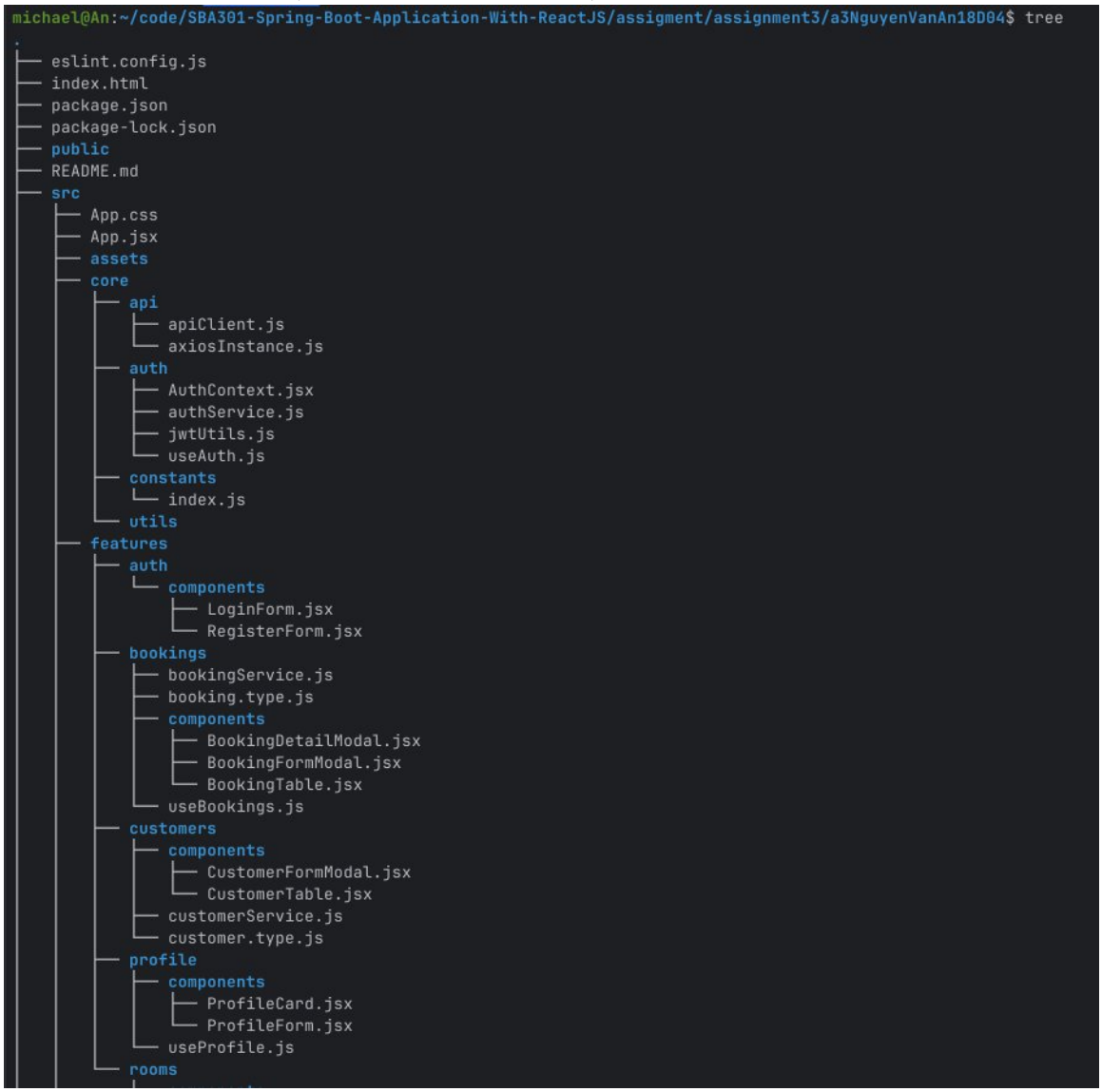
2. CẤU TRÚC PROJECT

2.1.1. 2.1 Backend (3-Layer Architecture)




enums
BookingStatus.java
CustomerStatus.java
Role.java
RoomStatus.java
jwts
JwtProperties.java
JwtTokenProvider.java
JwtTokenValidationFilter.java
repository
BookingDetailRepository.java
BookingReservationRepository.java
CustomerRepository.java
RoomInformationRepository.java
RoomTypeRepository.java
services
impl
BookingDetailServiceImpl.java
BookingReservationServiceImpl.java
CustomerServiceImpl.java
RoomInformationServiceImpl.java
RoomTypeServiceImpl.java
interfaces
BookingDetailService.java
BookingReservationService.java
CustomerService.java
RoomInformationService.java
RoomTypeService.java
utils
DataInitializer.java
resources
application.yml
test
java
com
michael
a3nguyenvanan18d04
A3NguyenVanAn18D04ApplicationTests.java
services
impl
RoomInformationServiceTest.java

2.1.2. 2.2 Frontend (Feature-based Structure)

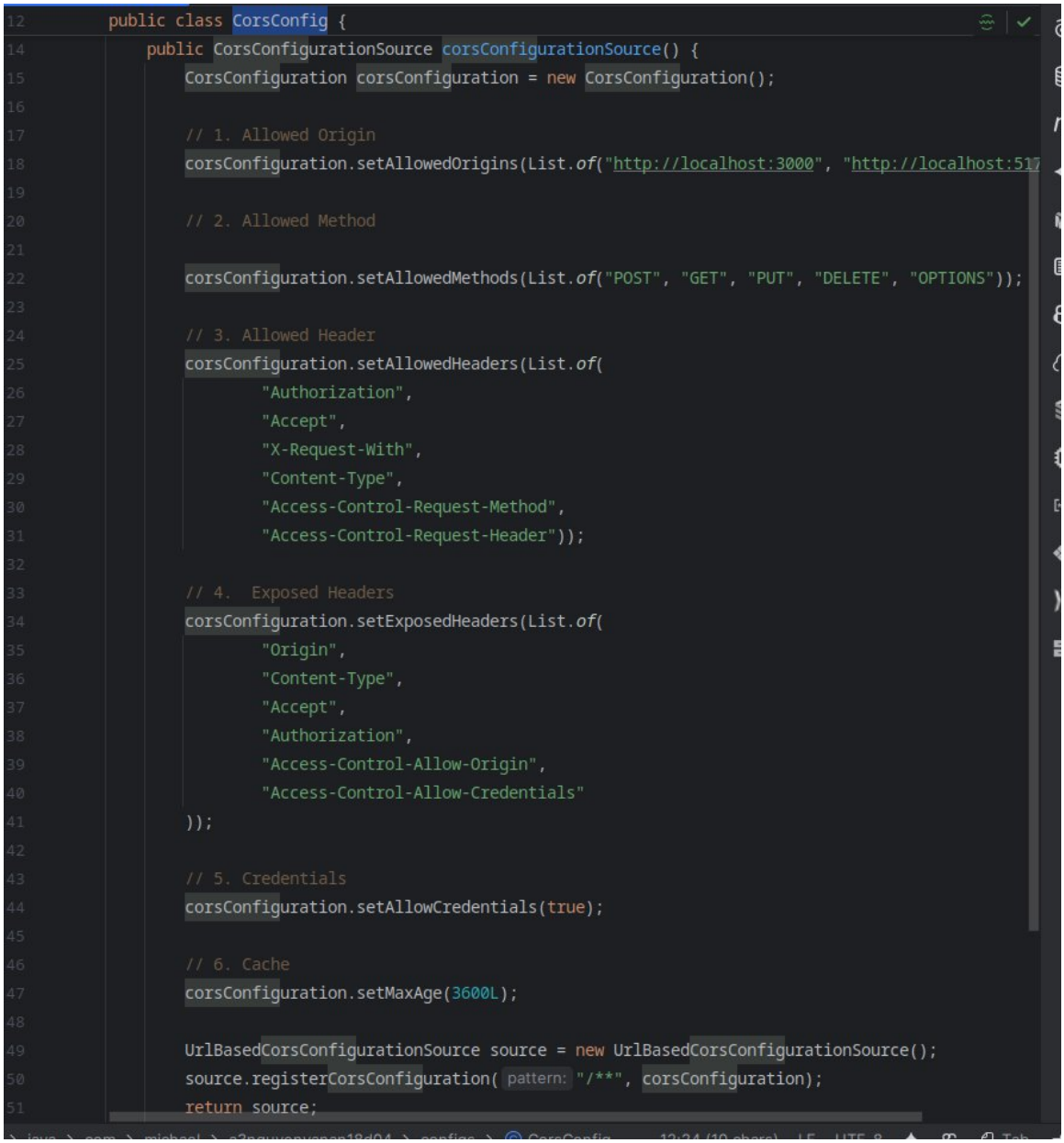



└── useProfile.js
└── rooms
├── components
├── RoomCard.jsx
├── RoomFilter.jsx
├── RoomFormModal.jsx
└── RoomTable.jsx
├── roomService.js
├── room.type.js
├── roomType.type.js
└── useRooms.js
├── hooks
├── index.css
├── main.jsx
├── pages
├── auth
├── LoginPage.jsx
└── RegisterPage.jsx
├── customer
├── BookingHistoryPage.jsx
├── BookingPage.jsx
└── ProfilePage.jsx
├── public
└── HomePage.jsx
└── staff
├── BookingsPage.jsx
├── CustomersPage.jsx
├── DashboardPage.jsx
└── RoomsPage.jsx
├── routes
└── AppRouter.jsx
└── shared
├── components
├── common
├── Footer.jsx
├── Header.jsx
└── PrivateRoute.jsx
└── ui
└── layouts
├── CustomerLayout.jsx
├── PublicLayout.jsx
└── StaffLayout.jsx
├── vite.config.js
└── yarn.lock
32 directories, 58 files
michael@An:~/code/SBA301-Spring-Boot-Application-With-ReactJS/assigment/assignment3/a3NguyenVanAn18D04$
a3NguyenVanAn18D04

3. CHI TIẾT BACKEND API (CONTROLLERS)

3.1. Authentication

3.1.1. CorsConfig



```
public class CorsConfig {
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration corsConfiguration = new CorsConfiguration();

        // 1. Allowed Origin
        corsConfiguration.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:517

        // 2. Allowed Method

        corsConfiguration.setAllowedMethods(List.of("POST", "GET", "PUT", "DELETE", "OPTIONS"));

        // 3. Allowed Header
        corsConfiguration.setAllowedHeaders(List.of(
                "Authorization",
                "Accept",
                "X-Request-With",
                "Content-Type",
                "Access-Control-Request-Method",
                "Access-Control-Request-Header"));

        // 4.  Exposed Headers
        corsConfiguration.setExposedHeaders(List.of(
                "Origin",
                "Content-Type",
                "Accept",
                "Authorization",
                "Access-Control-Allow-Origin",
                "Access-Control-Allow-Credentials"
        ));

        // 5. Credentials
        corsConfiguration.setAllowCredentials(true);

        // 6. Cache
        corsConfiguration.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration( pattern: "/**", corsConfiguration);
        return source;
```

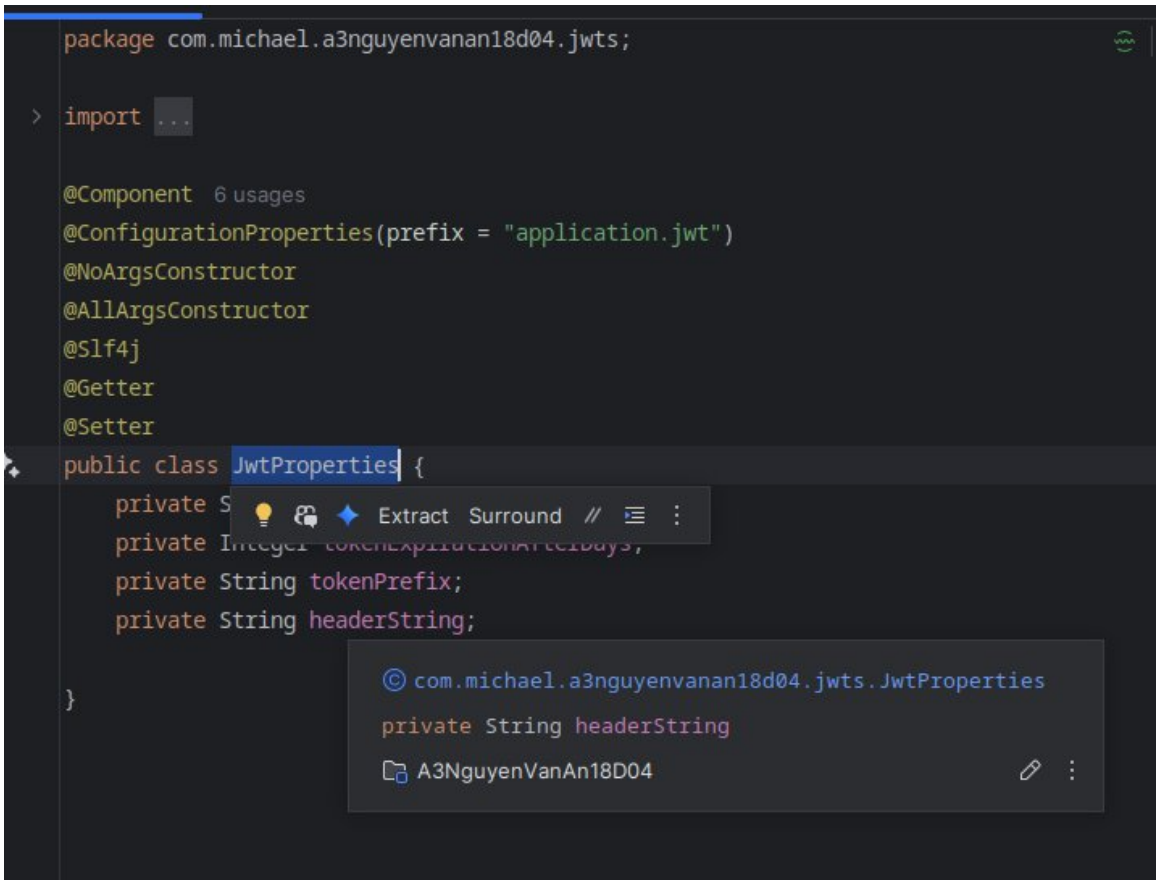
3.1.2. SecurityConfig

```java
@Configuration
@EnableWebSecurity
@Slf4j
public class SecurityConfig {
    private final JwtProperties jwtProperties;  3 usages

    public SecurityConfig(JwtProperties jwtProperties) { this.jwtProperties = jwtProperties; }

    @Bean
    public SecretKey secretKey() { return Keys.hmacShaKeyFor(jwtProperties.getSecretKey().getBytes()); }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http){
        JwtTokenValidationFilter jwtTokenValidationFilter = new JwtTokenValidationFilter(secretKey(), jwtProperties);
        return http
                .csrf(AbstractHttpConfigurer::disable)
                .cors(Customizer.withDefaults())
                .sessionManagement( session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests( auth ->
                        auth.requestMatchers("/api/v1/auth/**").permitAll()
                                .anyRequest().authenticated()

                )
                .addFilterBefore(jwtTokenValidationFilter, UsernamePasswordAuthenticationFilter.class)
                .build();
    }

}
```

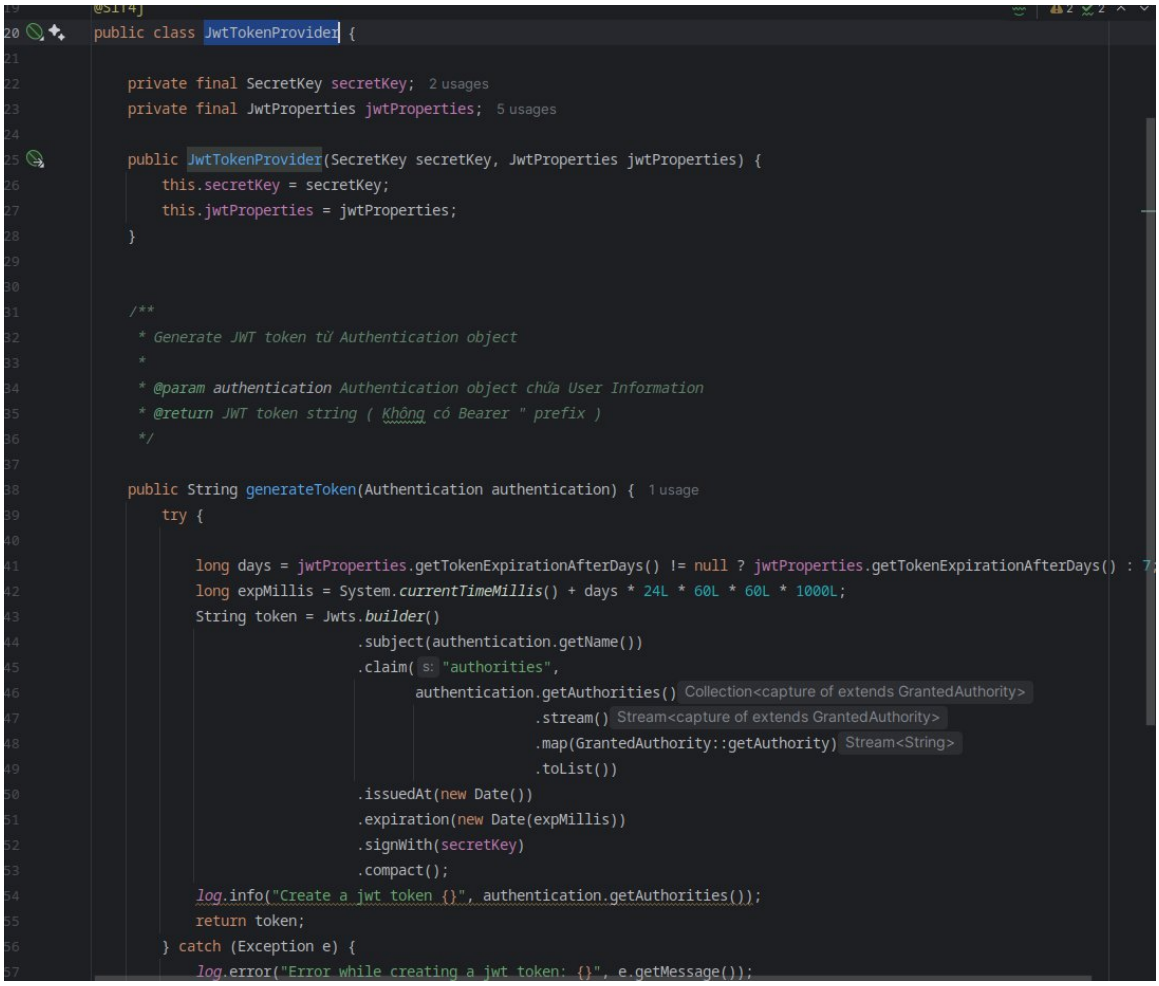
3.1.3. WebConfiguration

```java
@Configuration
public class WebConfiguration {
    @Bean
    public PasswordEncoder passwordEncoder () { return new BCryptPasswordEncoder( strength: 12); }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration authenticationConfiguration) {
        return authenticationConfiguration.getAuthenticationManager();
    }
}
```

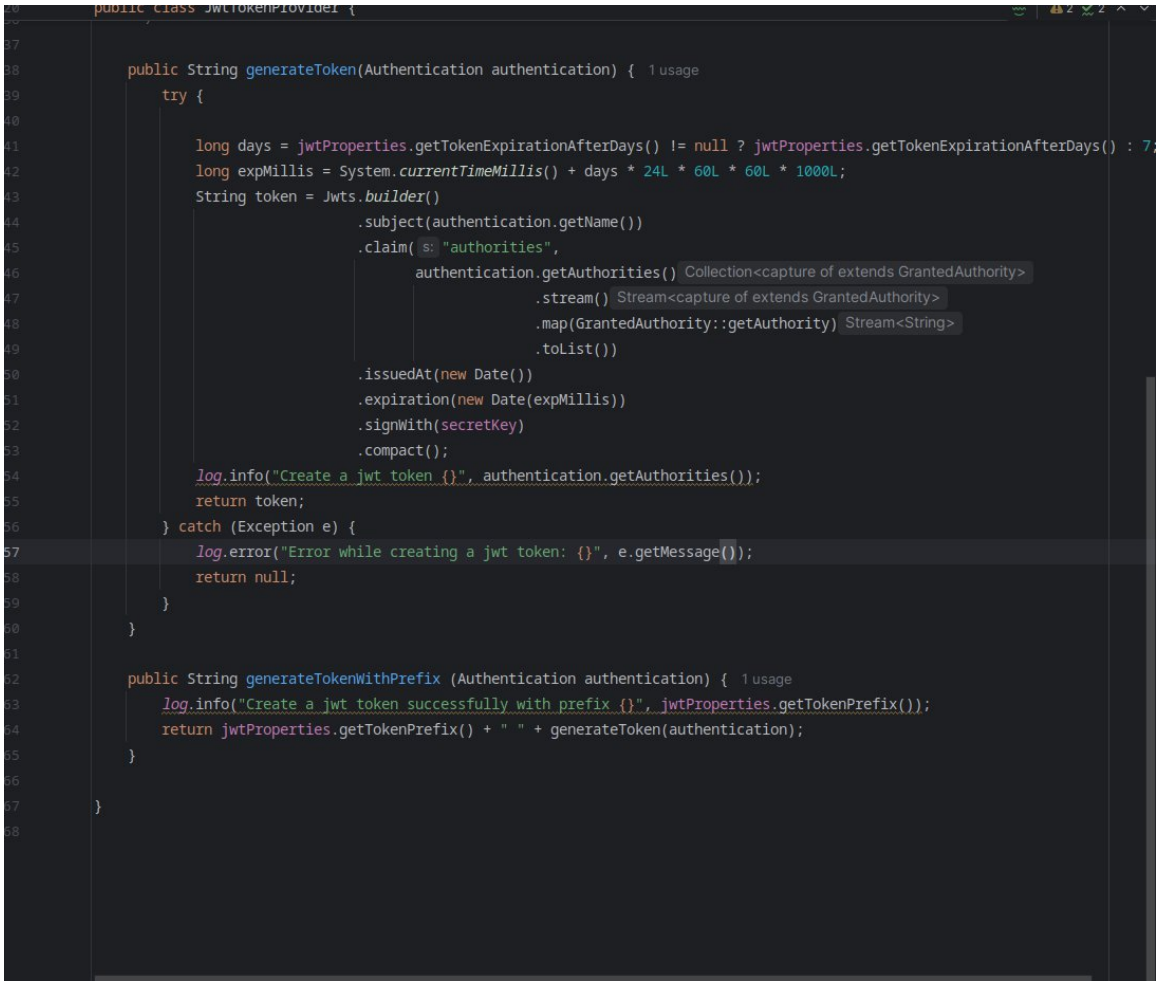
3.1.4. JwtProperties



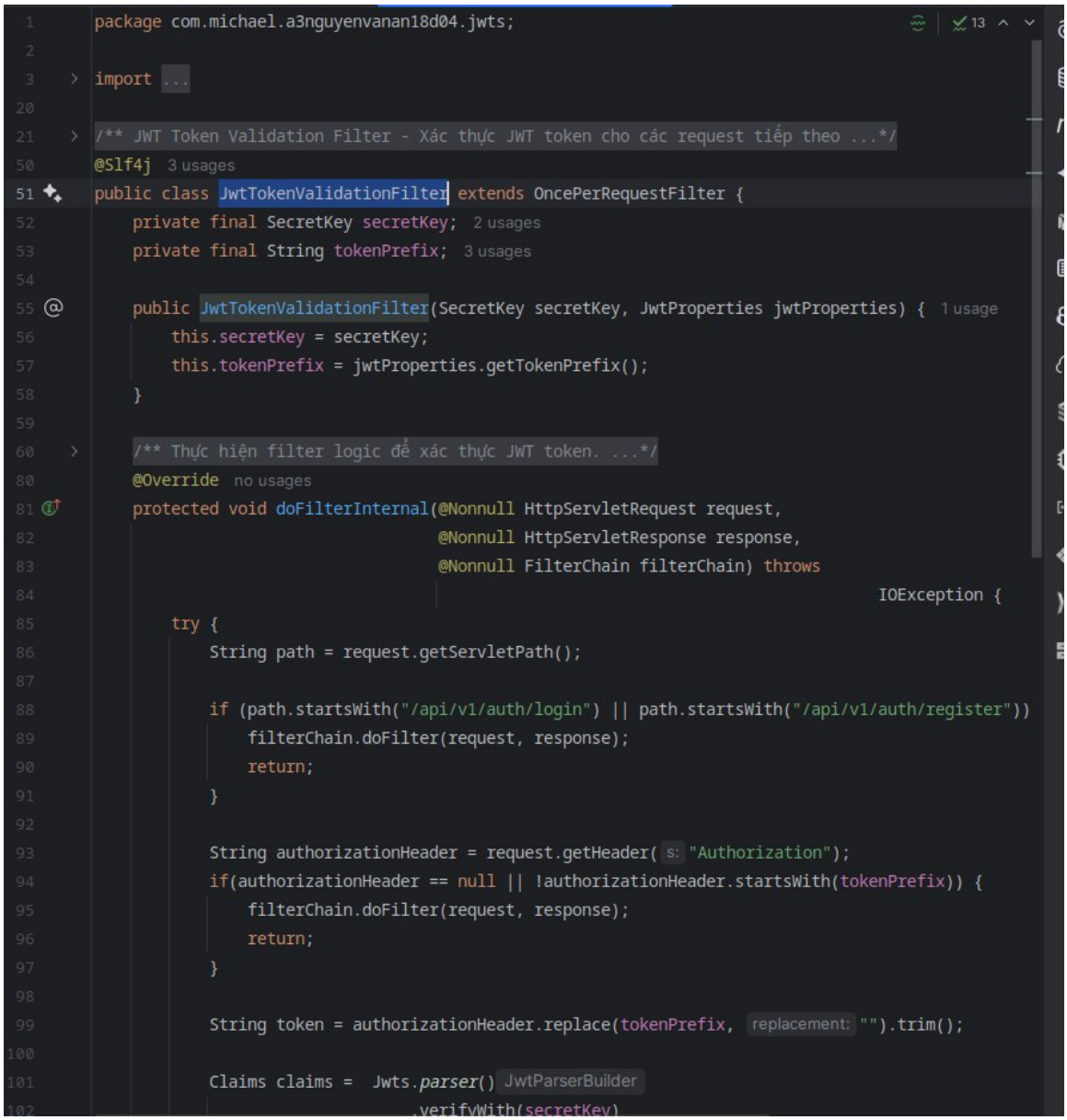
3.1.5. JwtTokenProvider



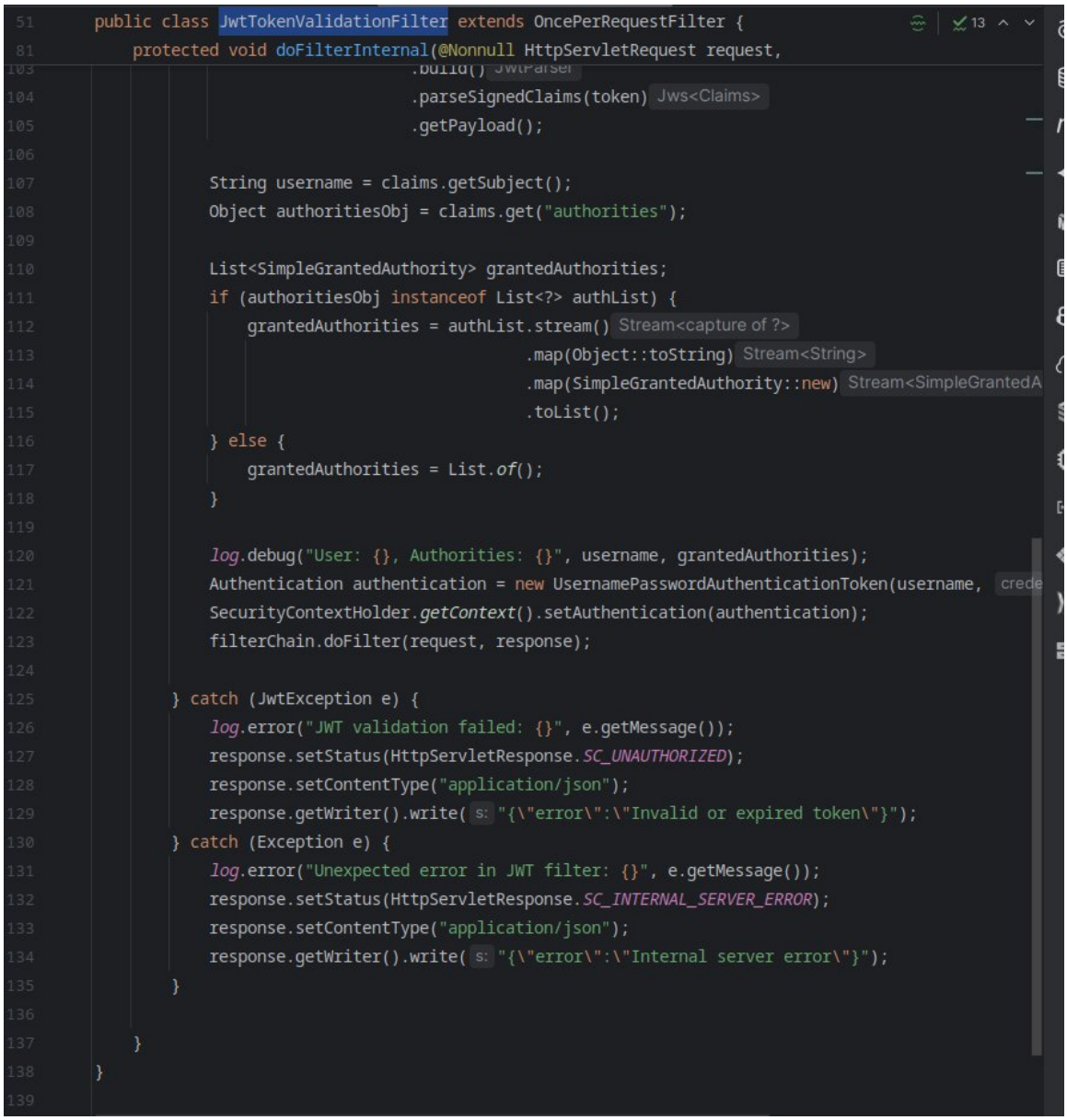
```java
@Slf4j
public class JwtTokenProvider {

    private final SecretKey secretKey;  2 usages
    private final JwtProperties jwtProperties;  5 usages

    public JwtTokenProvider(SecretKey secretKey, JwtProperties jwtProperties) {
        this.secretKey = secretKey;
        this.jwtProperties = jwtProperties;
    }


    /**
     * Generate JWT token từ Authentication object
     *
     * @param authentication Authentication object chứa User Information
     * @return JWT token string ( Không có Bearer " prefix )
     */

    public String generateToken(Authentication authentication) {  1 usage
        try {

            long days = jwtProperties.getTokenExpirationAfterDays() != null ? jwtProperties.getTokenExpirationAfterDays() : 7;
            long expMillis = System.currentTimeMillis() + days * 24L * 60L * 60L * 1000L;
            String token = Jwts.builder()
                    .subject(authentication.getName())
                    .claim( s: "authorities",
                            authentication.getAuthorities()  Collection<capture of extends GrantedAuthority>
                                    .stream()  Stream<capture of extends GrantedAuthority>
                                    .map(GrantedAuthority::getAuthority)  Stream<String>
                                    .toList())
                    .issuedAt(new Date())
                    .expiration(new Date(expMillis))
                    .signWith(secretKey)
                    .compact();
            log.info("Create a jwt token {}", authentication.getAuthorities());
            return token;
        } catch (Exception e) {
            log.error("Error while creating a jwt token: {}", e.getMessage());
```


public class JwtTokenProvider {
public String generateToken(Authentication authentication) { 1 usage
try {
long days = jwtProperties.getTokenExpirationAfterDays() != null ? jwtProperties.getTokenExpirationAfterDays() : 7;
long expMillis = System.currentTimeMillis() + days * 24L * 60L * 60L * 1000L;
String token = Jwts.builder()
.subject(authentication.getName())
.claim(s: "authorities",
authentication.getAuthorities() Collection<capture of extends GrantedAuthority>
.stream() Stream<capture of extends GrantedAuthority>
.map(GrantedAuthority::getAuthority) Stream<String>
.toList())
.issuedAt(new Date())
.expiration(new Date(expMillis))
.signWith(secretKey)
.compact();
log.info("Create a jwt token {}", authentication.getAuthorities());
return token;
} catch (Exception e) {
log.error("Error while creating a jwt token: {}", e.getMessage());
return null;
}
}
public String generateTokenWithPrefix (Authentication authentication) { 1 usage
log.info("Create a jwt token successfully with prefix {}", jwtProperties.getTokenPrefix());
return jwtProperties.getTokenPrefix() + " " + generateToken(authentication);
}
}

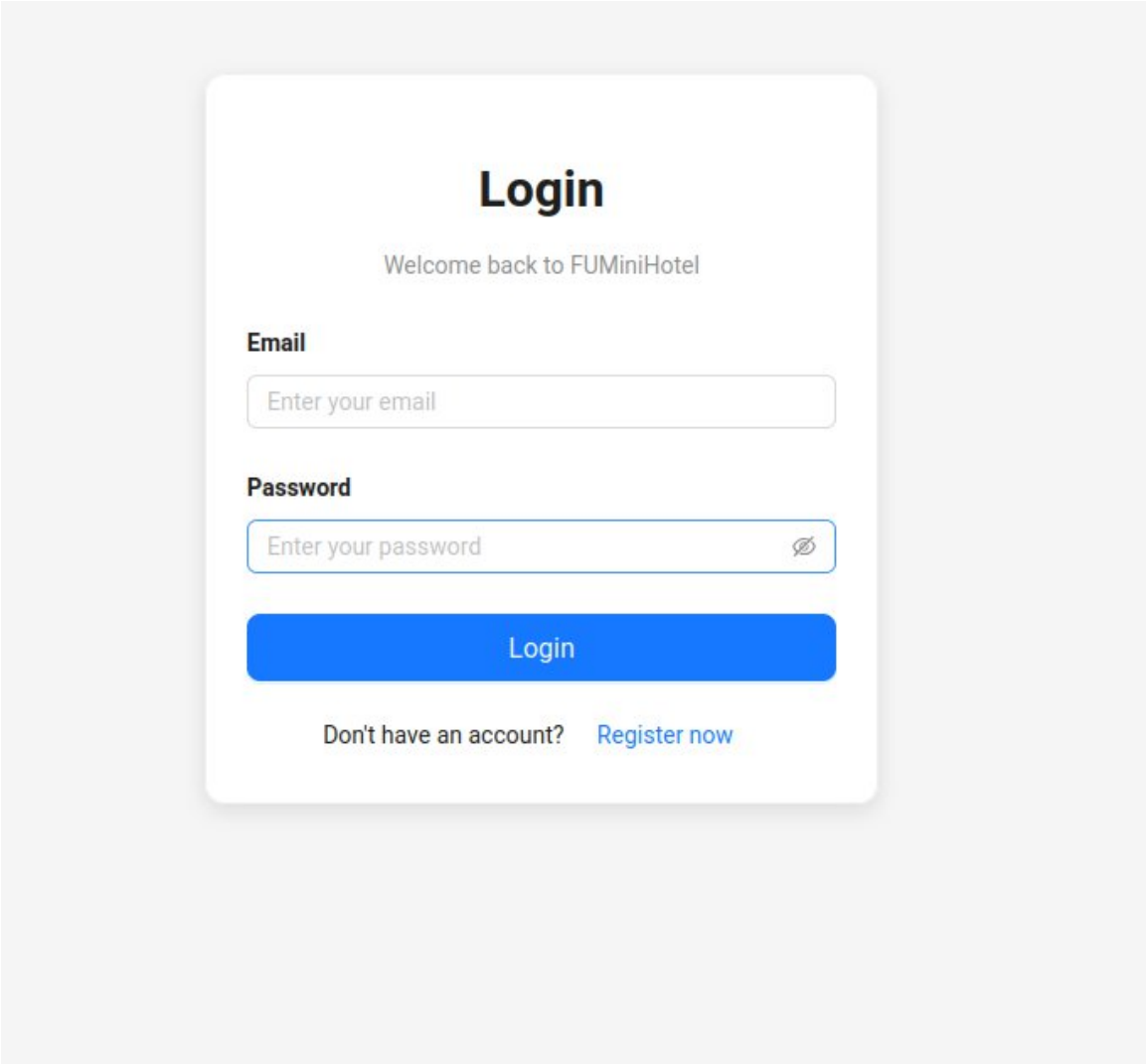
3.1.6. JwtTokenValidationFilter

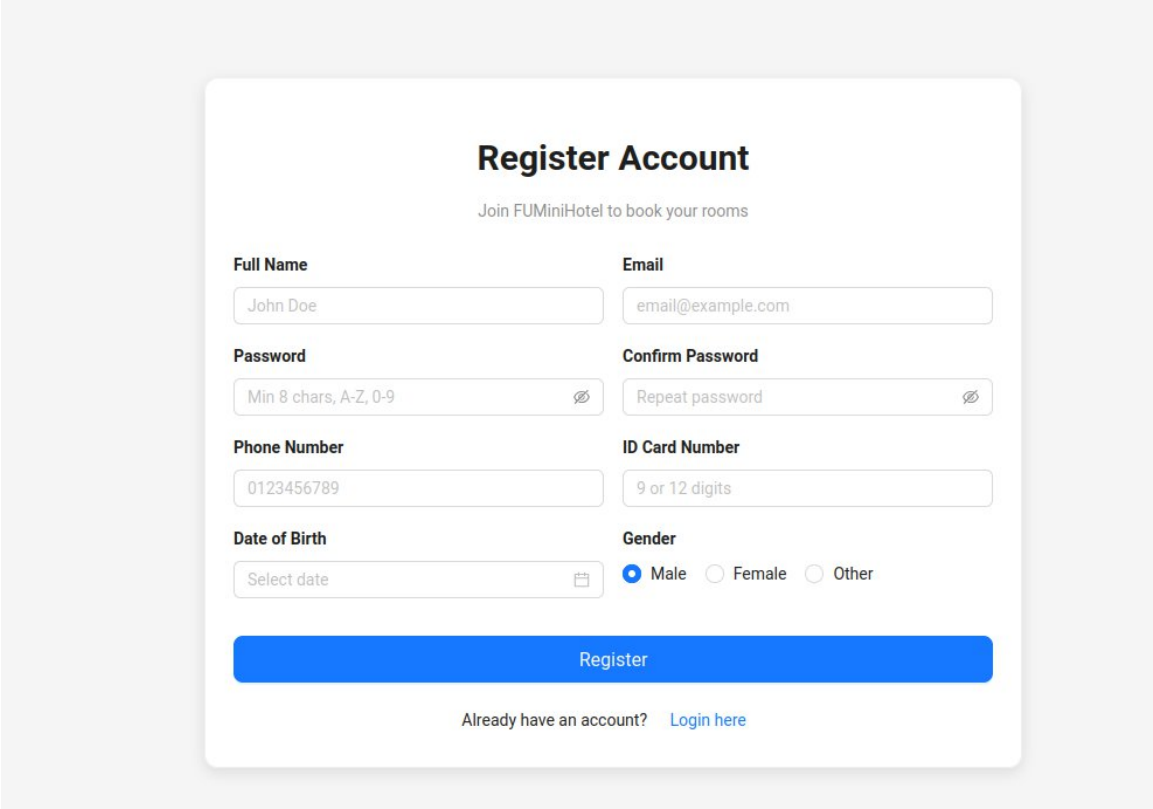


```java
package com.michael.a3nguyenvanan18d04.jwts;

import ...

/** JWT Token Validation Filter - Xác thực JWT token cho các request tiếp theo ...*/
@Slf4j  3 usages
public class JwtTokenValidationFilter extends OncePerRequestFilter {
    private final SecretKey secretKey;  2 usages
    private final String tokenPrefix;  3 usages

    public JwtTokenValidationFilter(SecretKey secretKey, JwtProperties jwtProperties) {  1 usage
        this.secretKey = secretKey;
        this.tokenPrefix = jwtProperties.getTokenPrefix();
    }

    /** Thực hiện filter logic để xác thực JWT token. ...*/
    @Override  no usages
    protected void doFilterInternal(@Nonnull HttpServletRequest request,
                                    @Nonnull HttpServletResponse response,
                                    @Nonnull FilterChain filterChain) throws
                                                                        IOException {
        try {
            String path = request.getServletPath();

            if (path.startsWith("/api/v1/auth/login") || path.startsWith("/api/v1/auth/register"))
                filterChain.doFilter(request, response);
                return;
            }

            String authorizationHeader = request.getHeader( s: "Authorization");
            if(authorizationHeader == null || !authorizationHeader.startsWith(tokenPrefix)) {
                filterChain.doFilter(request, response);
                return;
            }

            String token = authorizationHeader.replace(tokenPrefix,  replacement: "").trim();

            Claims claims =  Jwts.parser() JwtParserBuilder
                                .verifyWith(secretKey)
```


public class JwtTokenValidationFilter extends OncePerRequestFilter {
protected void doFilterInternal(@Nonnull HttpServletRequest request,
.build() JwtParser
.parseSignedClaims(token) Jws<Claims>
.getPayload();
String username = claims.getSubject();
Object authoritiesObj = claims.get("authorities");
List<SimpleGrantedAuthority> grantedAuthorities;
if (authoritiesObj instanceof List<?> authList) {
grantedAuthorities = authList.stream() Stream<capture of ?>
.map(Object::toString) Stream<String>
.map(SimpleGrantedAuthority::new) Stream<SimpleGrantedA
.toList();
} else {
grantedAuthorities = List.of();
}
log.debug("User: {}, Authorities: {}", username, grantedAuthorities);
Authentication authentication = new UsernamePasswordAuthenticationToken(username, crede
SecurityContextHolder.getContext().setAuthentication(authentication);
filterChain.doFilter(request, response);
} catch (JwtException e) {
log.error("JWT validation failed: {}", e.getMessage());
response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
response.setContentType("application/json");
response.getWriter().write(s: "{\"error\":\"Invalid or expired token\"}");
} catch (Exception e) {
log.error("Unexpected error in JWT filter: {}", e.getMessage());
response.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
response.setContentType("application/json");
response.getWriter().write(s: "{\"error\":\"Internal server error\"}");
}
}
}

3.1.7. Result


Login
Welcome back to FUMiniHotel
Email
Enter your email
Password
Enter your password
Login
Don't have an account?
Register now



3.2. Customer Management

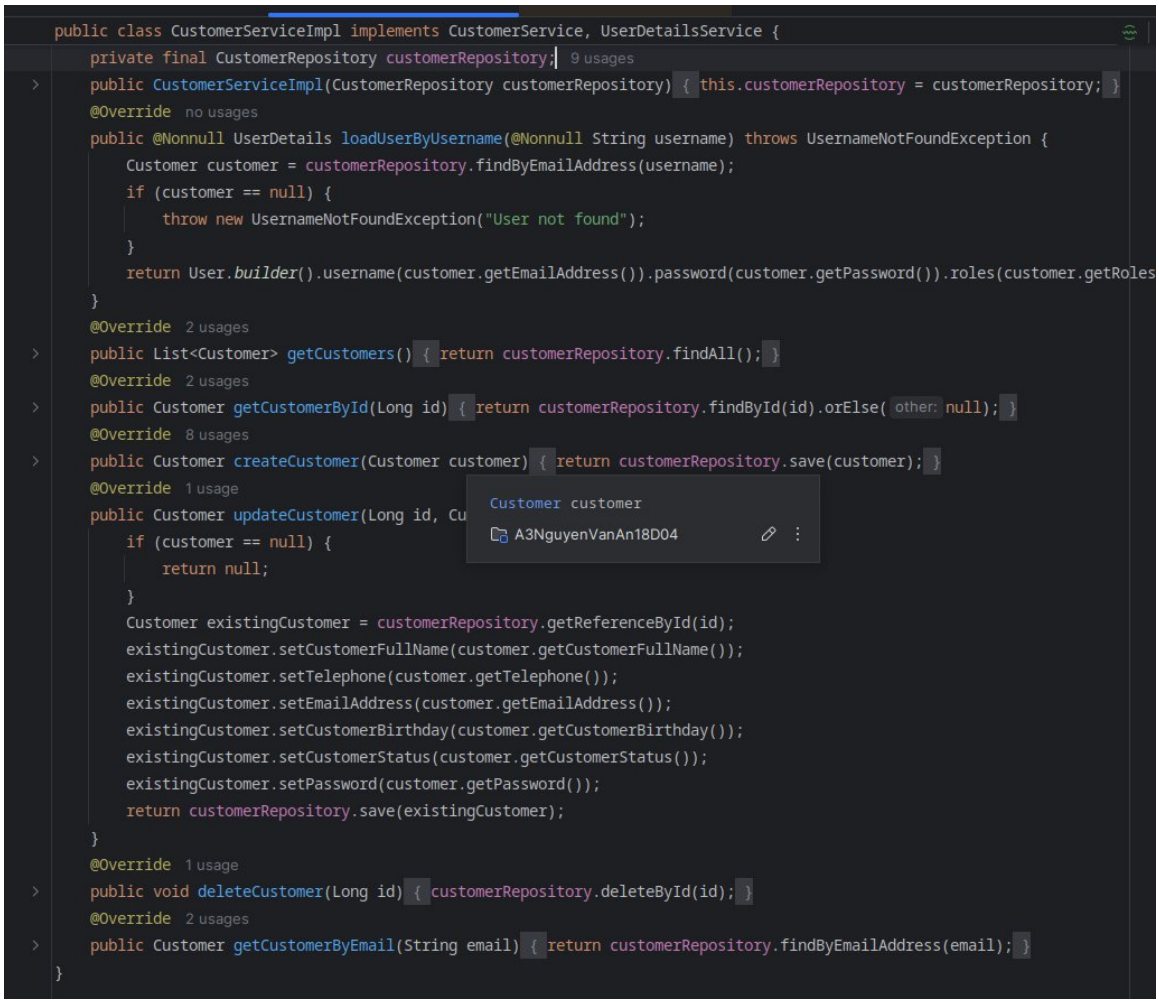
3.2.1. Customer

```java
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long customerID;

    private String customerFullName;

    private String telephone;

    private String emailAddress;

    private LocalDate customerBirthday;

    private CustomerStatus customerStatus;

    private String password;

    private Role roles;

    @OneToMany(mappedBy = "customer", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    @JsonIgnoreProperties("customer")
    private List<BookingReservation> bookingReservation =  new ArrayList<>();


    public void addBookingReservation(BookingReservation bookingReservation) {  1 usage
        this.bookingReservation.add(bookingReservation);
        bookingReservation.setCustomer(this);
    }

    public void removeBookingReservation(BookingReservation bookingReservation) {  no usages
        this.bookingReservation.remove(bookingReservation);
        bookingReservation.setCustomer(this);
    }
}
```

3.2.2. CustomerRepository

```java
package com.michael.a3nguyenvanan18d04.repository;

import ...

@Repository  3 usages
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    Customer findByEmailAddress(String emailAddress);  2 usages
    Customer findByTelephone(String telephone);  no usages
}
```

3.2.3. CustomerServiceImpl



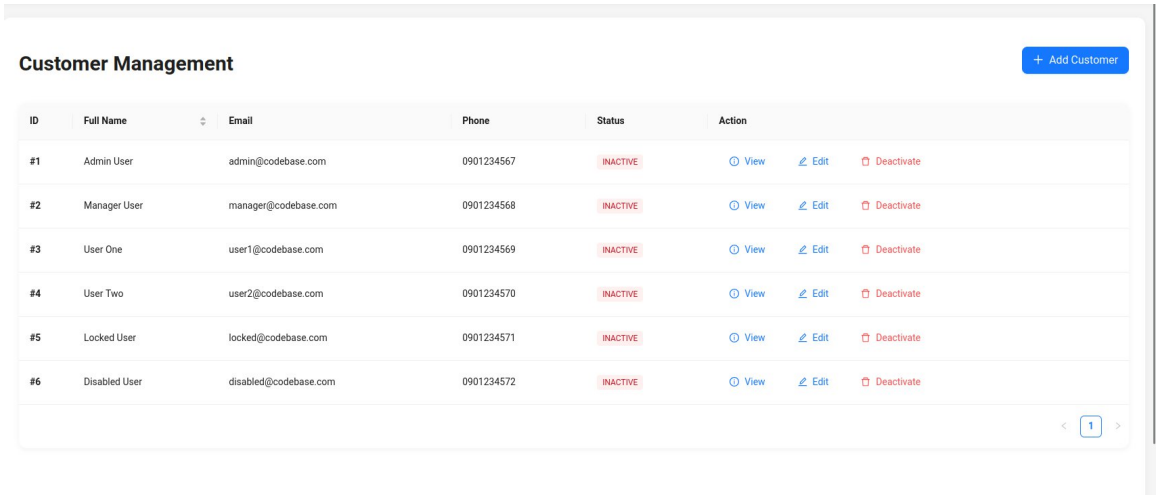
```
public class CustomerServiceImpl implements CustomerService, UserDetailsService {
    private final CustomerRepository customerRepository; 9 usages
    public CustomerServiceImpl(CustomerRepository customerRepository) { this.customerRepository = customerRepository; }
    @Override no usages
    public @Nonnull UserDetails loadUserByUsername(@Nonnull String username) throws UsernameNotFoundException {
        Customer customer = customerRepository.findByEmailAddress(username);
        if (customer == null) {
            throw new UsernameNotFoundException("User not found");
        }
        return User.builder().username(customer.getEmailAddress()).password(customer.getPassword()).roles(customer.getRoles
    }
    @Override 2 usages
    public List<Customer> getCustomers() { return customerRepository.findAll(); }
    @Override 2 usages
    public Customer getCustomerById(Long id) { return customerRepository.findById(id).orElse( other: null); }
    @Override 8 usages
    public Customer createCustomer(Customer customer) { return customerRepository.save(customer); }
    @Override 1 usage
    public Customer updateCustomer(Long id, Cu
        if (customer == null) {
            return null;
        }
        Customer existingCustomer = customerRepository.getReferenceById(id);
        existingCustomer.setCustomerFullName(customer.getCustomerFullName());
        existingCustomer.setTelephone(customer.getTelephone());
        existingCustomer.setEmailAddress(customer.getEmailAddress());
        existingCustomer.setCustomerBirthday(customer.getCustomerBirthday());
        existingCustomer.setCustomerStatus(customer.getCustomerStatus());
        existingCustomer.setPassword(customer.getPassword());
        return customerRepository.save(existingCustomer);
    }
    @Override 1 usage
    public void deleteCustomer(Long id) { customerRepository.deleteById(id); }
    @Override 2 usages
    public Customer getCustomerByEmail(String email) { return customerRepository.findByEmailAddress(email); }
}
```

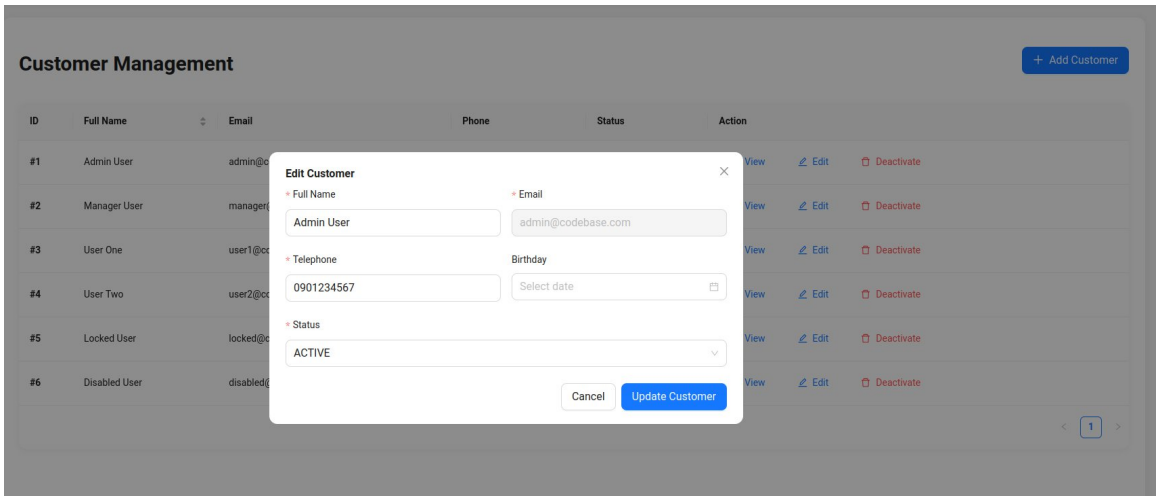
3.2.4. CustomerController

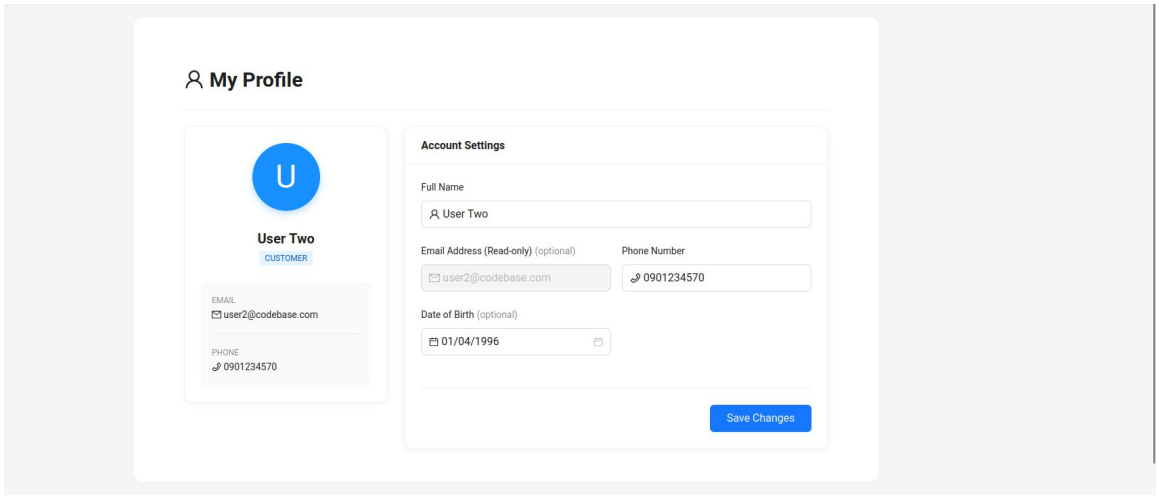
```java
package com.michael.a3nguyenvanan18d04.controllers;

import ...

@RestController
@RequestMapping("/api/v1/customers")

public class CustomerController {
    private final CustomerService customerService;  7 usages
    public CustomerController(CustomerService customerService) { this.customerService = customerService; }

    @GetMapping("")
    public ResponseEntity<Object> getAllCustomers() { return ResponseEntity.ok(customerService.getCustomers()); }

    @PostMapping("")
    public ResponseEntity<Object> addCustomer(@RequestBody Customer cus
        return ResponseEntity.ok(customerService.createCustomer(custome
    }

    @GetMapping("/{id}")
    public ResponseEntity<Object> getCustomerById(@PathVariable Long id) {
        return ResponseEntity.ok(customerService.getCustomerById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Object> updateCustomer(@PathVariable Long id,@RequestBody Customer customer) {
        if(customerService.updateCustomer(id, customer) != null) {
            return ResponseEntity.ok(customerService.getCustomers());
        }
        return ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteCustomer(@PathVariable Long id) {
        customerService.deleteCustomer(id);
        return ResponseEntity.ok( body: "Customer deleted");
```

```
org.springframework.http
public class ResponseEntity<T>
extends org.springframework.http.HttpEntity<T>
Maven: org.springframework:spring-w...-7.0.5.jar)
```

3.2.5. Result




Customer Management
+ Add Customer
ID
Full Name
Email
Phone
Status
Action
#1
Admin User
#2
Manager User
#3
User One
#4
User Two
#5
Locked User
#6
Disabled User
View
Edit
Deactivate
Edit Customer
* Full Name
Admin User
* Email
admin@codebase.com
* Telephone
0901234567
Birthday
Select date
* Status
ACTIVE
Cancel
Update Customer
1


My Profile
U
User Two
CUSTOMER
EMAIL
user2@codebase.com
PHONE
0901234570
Account Settings
Full Name
User Two
Email Address (Read-only) (optional)
user2@codebase.com
Phone Number
0901234570
Date of Birth (optional)
01/04/1996
Save Changes

3.3. Quản lý phòng (RoomInformationController)

3.3.1. RoomInformation

```java
package com.michael.a3nguyenvanan18d04.entites;
import ...
@Entity
@Table(name = "RoomInformation")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class RoomInformation {
    @Id
    @GeneratedValue(strategy = jakarta.persistence.GenerationType.IDENTITY)
    private Long roomID;
    private String roomNumber;
    private String roomDetailDescription;

    private int roomMaxCapacity;

    private RoomStatus roomStatus;

    private double roomPricePerDay;

    @ManyToOne
    @JoinColumn(name = "roomTypeID")
    @JsonIgnoreProperties("roomInformation")
    private RoomType roomType;

    @OneToMany(mappedBy = "roomInformation", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    @JsonIgnoreProperties("roomInformation")
    private List<BookingDetail> bookingDetails = new ArrayList<>();

    public void addBookingDetail(BookingDetail bookingDetail) {  1 usage
        this.bookingDetails.add(bookingDetail);
        bookingDetail.setRoomInformation(this);
    }
    public void removeBookingDetail(BookingDetail bookingDetail) {  no usages
        this.bookingDetails.remove(bookingDetail);
        bookingDetail.setRoomInformation(null);
    }
```

3.3.2. RoomType

```java
package com.michael.a3nguyenvanan18d04.entites;

import ...
@Entity
@Table
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class RoomType {
    @Id
    @GeneratedValue(strategy= GenerationType.IDENTITY)
    private Long roomTypeID;
    private String roomTypeName;

    private String roomTypeDescription;

    private String roomTypeNote;

    @OneToMany(mappedBy = "roomType", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)
    @JsonIgnoreProperties("roomType")
    private List<RoomInformation> roomInformation = new ArrayList<>();

    public void addRoomInformation(RoomInformation roomInformation) { 6 usages
        if (roomInformation == null) return;
        if (this.roomInformation == null) this.roomInformation = new ArrayList<>();
        if (!this.roomInformation.contains(roomInformation)) {
            this.roomInformation.add(roomInformation);
            roomInformation.setRoomType(this);
        }
    }
    public void removeRoomInformation(RoomInformation roomInformation) { no usages
        if (roomInformation == null || this.roomInformation == null) return;
        if (this.roomInformation.remove(roomInformation)) {
            roomInformation.setRoomType(null);
        }
    }
}
```

3.3.3. RoomInformationRepository

```java
package com.michael.a3nguyenvanan18d04.repository;

import ...

@Repository 7 usages
public interface RoomInformationRepository extends JpaRepository<RoomInformation, Long> {
}
```

3.3.4. RoomInformationService

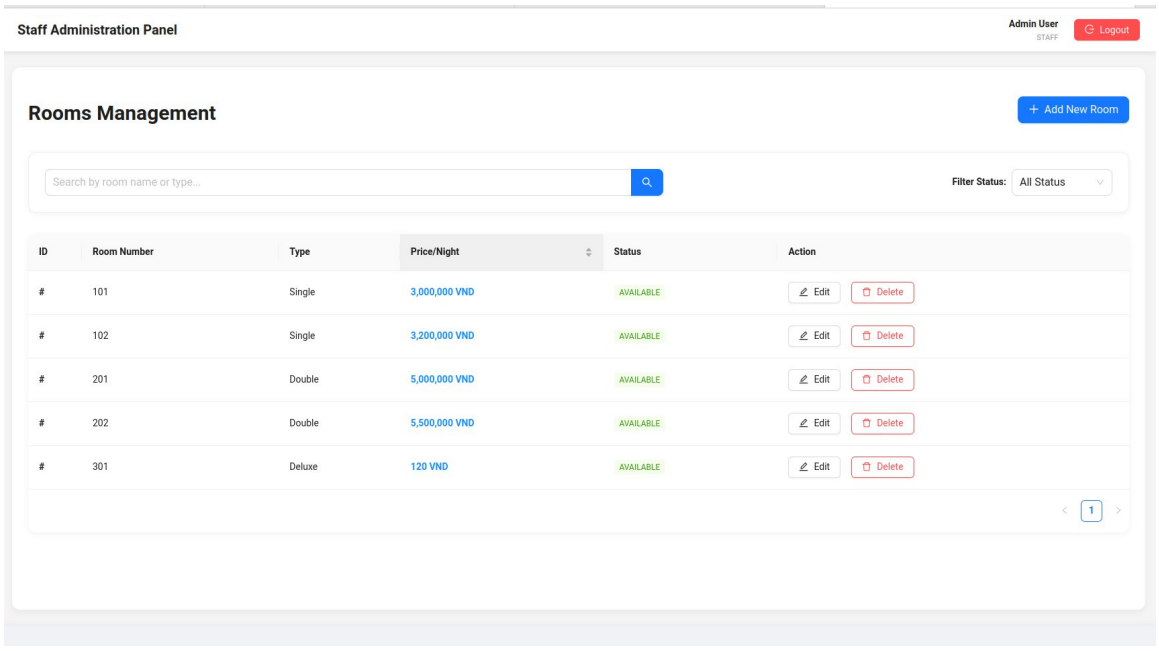
```java
package com.michael.a3nguyenvanan18d04.services.impl;

import ...

@Service
public class BookingReservationServiceImpl implements BookingReservationService {
    private final BookingReservationRepository bookingReservationRepository;  6 usages
    private final CustomerService customerService;  3 usages
    private final BookingDetailService bookingDetailService;  2 usages

    public BookingReservationServiceImpl(BookingReservationRepository bookingReservationRepository, CustomerServiceImpl customerService, BookingDetailService bookingD
        this.bookingReservationRepository = bookingReservationRepository;
        this.customerService = customerService;
        this.bookingDetailService = bookingDetailService;
    }

    @Override  1 usage
    public List<BookingReservation> getBookingReservations(Authentication authentication) {
        boolean isStaff = authentication.getAuthorities().stream()  Stream<capture of extends GrantedAuthority>
                                        .map(a -> a.getAuthority())  Stream<String>
                                        .anyMatch(role -> role.equals("ROLE_STAFF") || role.equals("ROLE_ADMIN") || role.equals("ROLE_MANAGER"));
        if (isStaff) {
            return bookingReservationRepository.findAll();
        }
        String principalName = authentication.getName();
        Customer customer = customerService.getCustomerByEmail(principalName);
        return customer.getBookingReservation();
    }
    @Override  1 usage
    public BookingReservation getBookingReservationById(Long id) {
        return bookingReservationRepository.getReferenceById(id);
    }
```

```java
    @Override  1 usage
    public BookingReservation getBookingReservationById(Long id) {
        return bookingReservationRepository.getReferenceById(id);
    }
    @Override  1 usage
    public BookingReservation createBookingReservation(BookingRequestDTO bookingRequest) {
        BookingReservation bookings= BookingReservation.builder().bookingDate(bookingRequest.getBookingDate()).bookingStatus(bookingRequest.getBookingStatus()).totalPr
        List<BookingDetail> bookingDetails =
                bookingRequest.getBookingDetails()  List<BookingDetailRequestDTO>
                        .stream()  Stream<BookingDetailRequestDTO>
                        .map(bookingDetailService::createBookingDetail).toList();
        for (BookingDetail detail : bookingDetails) {
            bookings.addBookingDetails(detail);
        }
        Customer customer = customerService.getCustomerById(bookingRequest.getCustomerID());
        customer.addBookingReservation(bookings);
        return bookingReservationRepository.save(bookings);
    }
    @Override  1 usage
    public BookingReservation updateBookingReservation(Long id, BookingReservation bookingReservation) {
        return bookingReservationRepository.save(bookingReservation);
    }
    @Override  no usages
    public void deleteBookingReservation(Long id) {
        bookingReservationRepository.deleteById(id);

    }
}
```

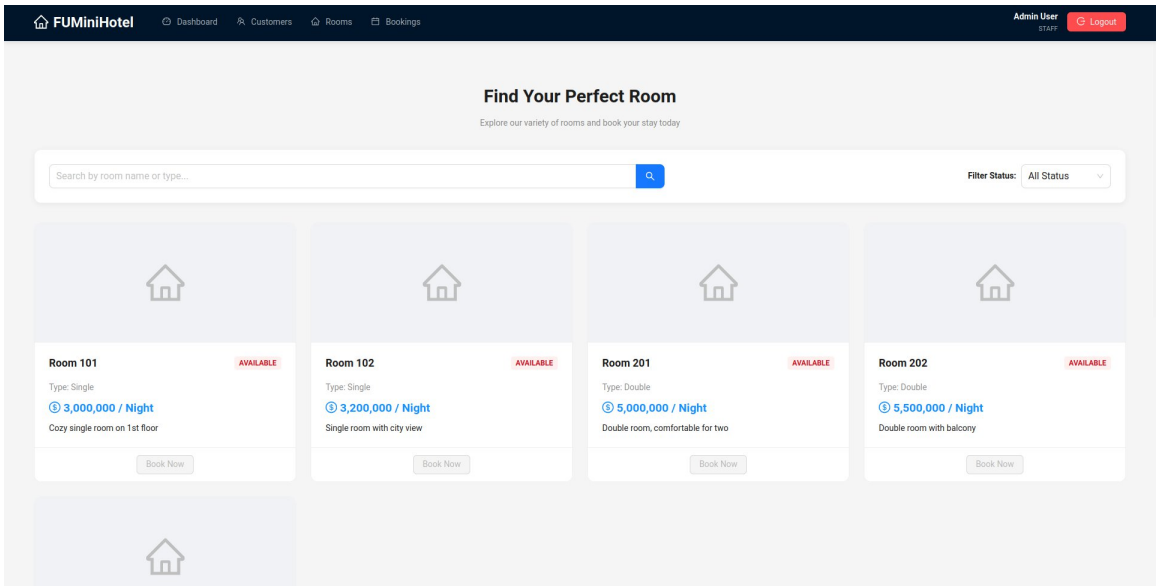
Invalid VCS root mapping
The directory
~/code/SPA301-Spring-Boot-Application-With-ReactJS/...

3.3.5. RoomInformationController

```java
package com.michael.a3nguyenvanan18d04.controllers;

import ...

@RestController
@RequestMapping("/api/v1/rooms")
@Slf4j
public class RoomInformationController {
    private final RoomInformationService roomInformationService;  5 usages

    public RoomInformationController(RoomInformationService roomInformationService) {
        this.roomInformationService = roomInformationService;
    }

    @GetMapping
    public Object getRoomInformation() { return roomInformationService.getRoomInformationServices(); }

    @PutMapping("/{id}")
    public Object updateRoomInformation(@PathVariable Long id, @RequestBody UpdateRoomRequest updateRoomRequest) {
        return roomInformationService.updateRoomInformation(id, updateRoomRequest);
    }

    @PostMapping()
    public Object createRoomInformation( @RequestBody UpdateRoomRequest updateRoomRequest) {
        return roomInformationService.createRoomInformation(updateRoomRequest);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteRoomInformation(@PathVariable Long id) {
         roomInformationService.deleteRoomInformation(id);
         return ResponseEntity.ok().build();
    }


}
```



3.3.6. Result

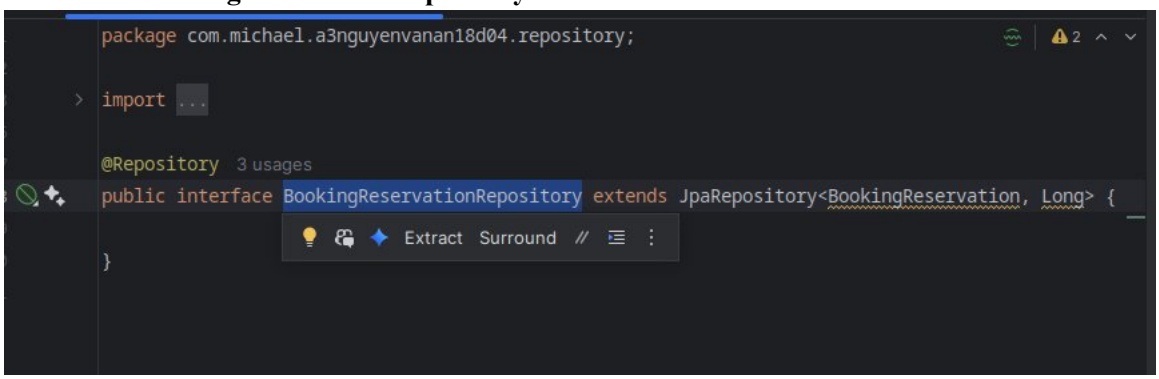

FUMiniHotel
Dashboard
Customers
Rooms
Bookings
Admin User
STAFF
Logout
Find Your Perfect Room
Explore our variety of rooms and book your stay today
Search by room name or type...
Filter Status:
All Status
Room 101
AVAILABLE
Type: Single
3,000,000 / Night
Cozy single room on 1st floor
Book Now
Room 102
AVAILABLE
Type: Single
3,200,000 / Night
Single room with city view
Book Now
Room 201
AVAILABLE
Type: Double
5,000,000 / Night
Double room, comfortable for two
Book Now
Room 202
AVAILABLE
Type: Double
5,500,000 / Night
Double room with balcony
Book Now

3.4. Quản lý Đặt phòng (BookingReservationController)

3.4.1. BookingReservation

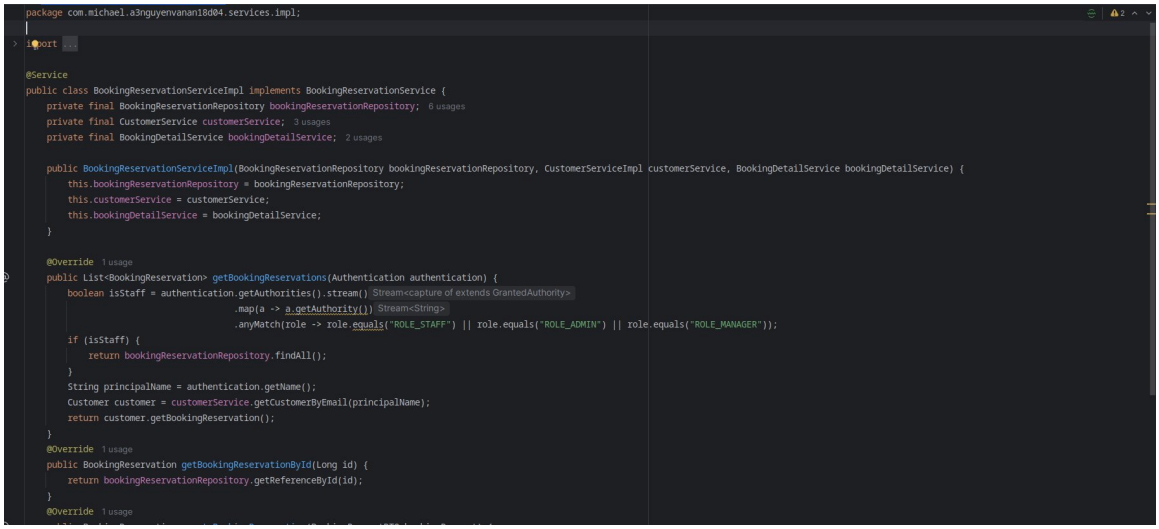
```java
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class BookingReservation {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long bookingReservationID;

    private LocalDateTime bookingDate;

    private double totalPrice;

    private BookingStatus bookingStatus;

    @ManyToOne
    @JoinColumn(name = "customerID")
    @JsonIgnoreProperties("bookingReservation")
    private Customer customer;

    @OneToMany(mappedBy = "bookingReservation", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    @JsonIgnoreProperties("bookingReservation")
    private List<BookingDetail> bookingDetails = new ArrayList<>();

    public void addBookingDetails(BookingDetail detail) {  1 usage
        if (bookingDetails == null) {
            bookingDetails = new ArrayList<>();
        }
        bookingDetails.add(detail);
        detail.setBookingReservation(this); // set back-reference
    }
    public void removeBookingDetails(BookingDetail detail) {  no usages
        if (bookingDetails != null) {
            bookingDetails.remove(detail);
            detail.setBookingReservation(null);
        }
    }
}
```

3.4.2. BookingReservationRepository

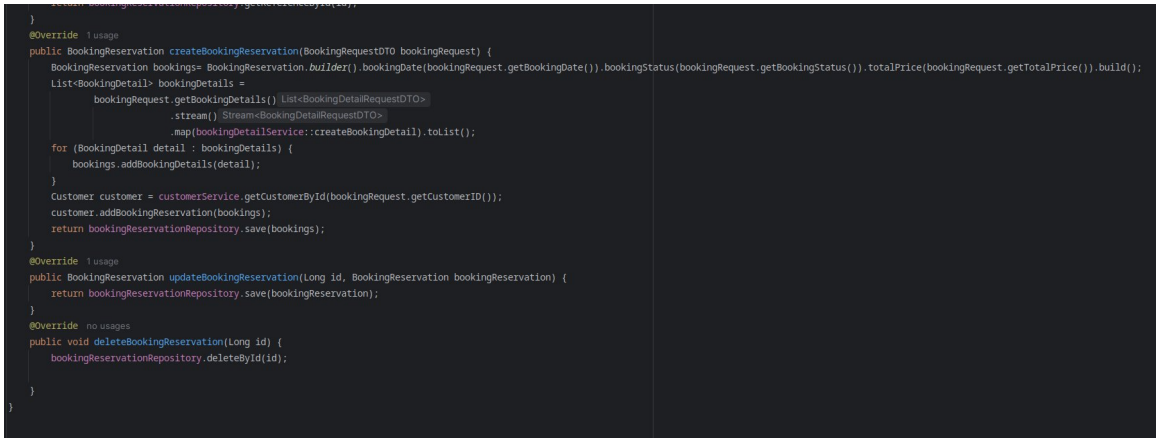


```
package com.michael.a3nguyenvanan18d04.repository;

import ...

@Repository  3 usages
public interface BookingReservationRepository extends JpaRepository<BookingReservation, Long> {

}
```

3.4.3. BookingReservationServiceImpl



```
package com.michael.a3nguyenvanan18d04.services.impl;

import ...

@Service
public class BookingReservationServiceImpl implements BookingReservationService {
    private final BookingReservationRepository bookingReservationRepository;  6 usages
    private final CustomerService customerService;  3 usages
    private final BookingDetailService bookingDetailService;  2 usages

    public BookingReservationServiceImpl(BookingReservationRepository bookingReservationRepository, CustomerServiceImpl customerService, BookingDetailService bookingDetailService) {
        this.bookingReservationRepository = bookingReservationRepository;
        this.customerService = customerService;
        this.bookingDetailService = bookingDetailService;
    }

    @Override  1 usage
    public List<BookingReservation> getBookingReservations(Authentication authentication) {
        boolean isStaff = authentication.getAuthorities().stream()  Stream<capture of extends GrantedAuthority>
                                        .map(a -> a.getAuthority())  Stream<String>
                                        .anyMatch(role -> role.equals("ROLE_STAFF") || role.equals("ROLE_ADMIN") || role.equals("ROLE_MANAGER"));
        if (isStaff) {
            return bookingReservationRepository.findAll();
        }
        String principalName = authentication.getName();
        Customer customer = customerService.getCustomerByEmail(principalName);
        return customer.getBookingReservation();
    }
    @Override  1 usage
    public BookingReservation getBookingReservationById(Long id) {
        return bookingReservationRepository.getReferenceById(id);
    }
    @Override  1 usage
```



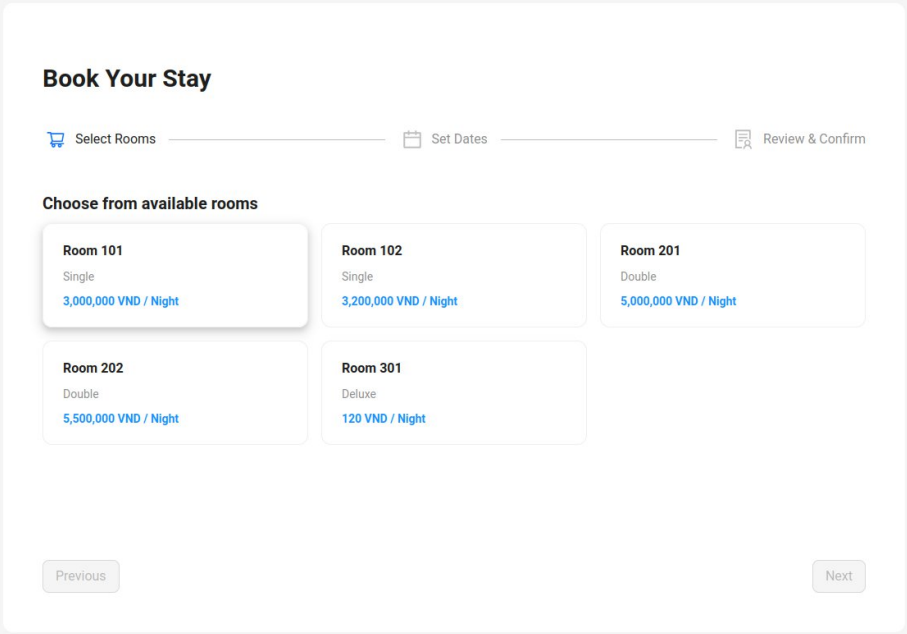
```
    }
    @Override  1 usage
    public BookingReservation createBookingReservation(BookingRequestDTO bookingRequest) {
        BookingReservation bookings= BookingReservation.builder().bookingDate(bookingRequest.getBookingDate()).bookingStatus(bookingRequest.getBookingStatus()).totalPrice(bookingRequest.getTotalPrice()).build();
        List<BookingDetail> bookingDetails =
                bookingRequest.getBookingDetails()  List<BookingDetailRequestDTO>
                        .stream()  Stream<BookingDetailRequestDTO>
                        .map(bookingDetailService::createBookingDetail).toList();
        for (BookingDetail detail : bookingDetails) {
            bookings.addBookingDetails(detail);
        }
        Customer customer = customerService.getCustomerById(bookingRequest.getCustomerID());
        customer.addBookingReservation(bookings);
        return bookingReservationRepository.save(bookings);
    }
    @Override  1 usage
    public BookingReservation updateBookingReservation(Long id, BookingReservation bookingReservation) {
        return bookingReservationRepository.save(bookingReservation);
    }
    @Override  no usages
    public void deleteBookingReservation(Long id) {
        bookingReservationRepository.deleteById(id);

    }
}
```

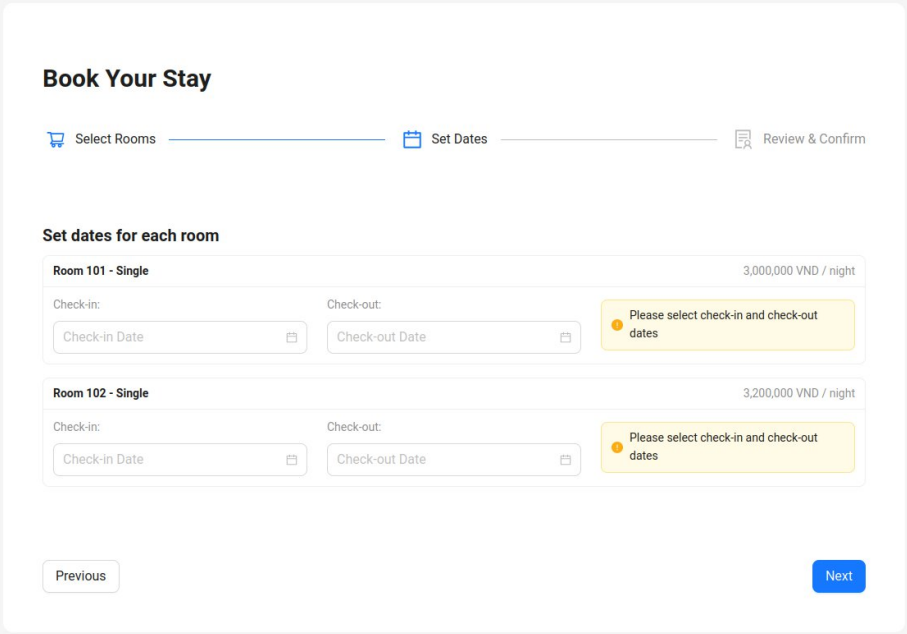
3.4.4. BookingReservationController

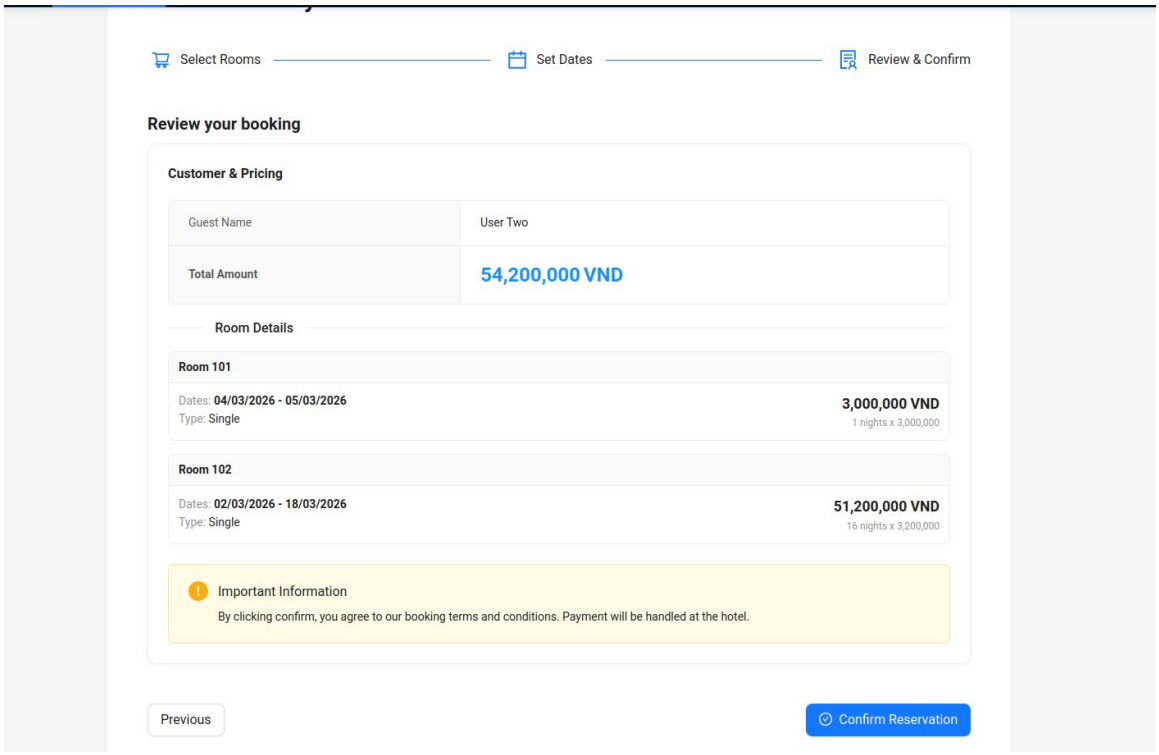
```java
package com.michael.a3nguyenvanan18d04.controllers;

import com.michael.a3nguyenvanan18d04.dtos.booking.request.BookingRequestDTO;
import com.michael.a3nguyenvanan18d04.dtos.booking.request.BookingStatusUpdateDTO;
import com.michael.a3nguyenvanan18d04.entites.BookingReservation;
import com.michael.a3nguyenvanan18d04.services.interfaces.BookingReservationService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/bookings")
public class BookingReservationController {
    private final BookingReservationService bookingReservationService;  5 usages

    public BookingReservationController (BookingReservationService bookingReservationService){
        this.bookingReservationService = bookingReservationService;
    }

    @GetMapping({"", "/customer/{customerID}"})
    @PreAuthorize("hasAnyRole('STAFF','CUSTOMER')")

    public ResponseEntity<Object> getBookings(@PathVariable(required = false) Long customerID, Authentication authentication) {
        return ResponseEntity.ok(
                bookingReservationService.getBookingReservations(authentication)
        );
    }
    @PostMapping
    public ResponseEntity<Object> createBooking(@RequestBody BookingRequestDTO bookingRequestDTO ) {
        return ResponseEntity.ok(
                bookingReservationService.createBookingReservation(bookingRequestDTO)
        );
    }
```

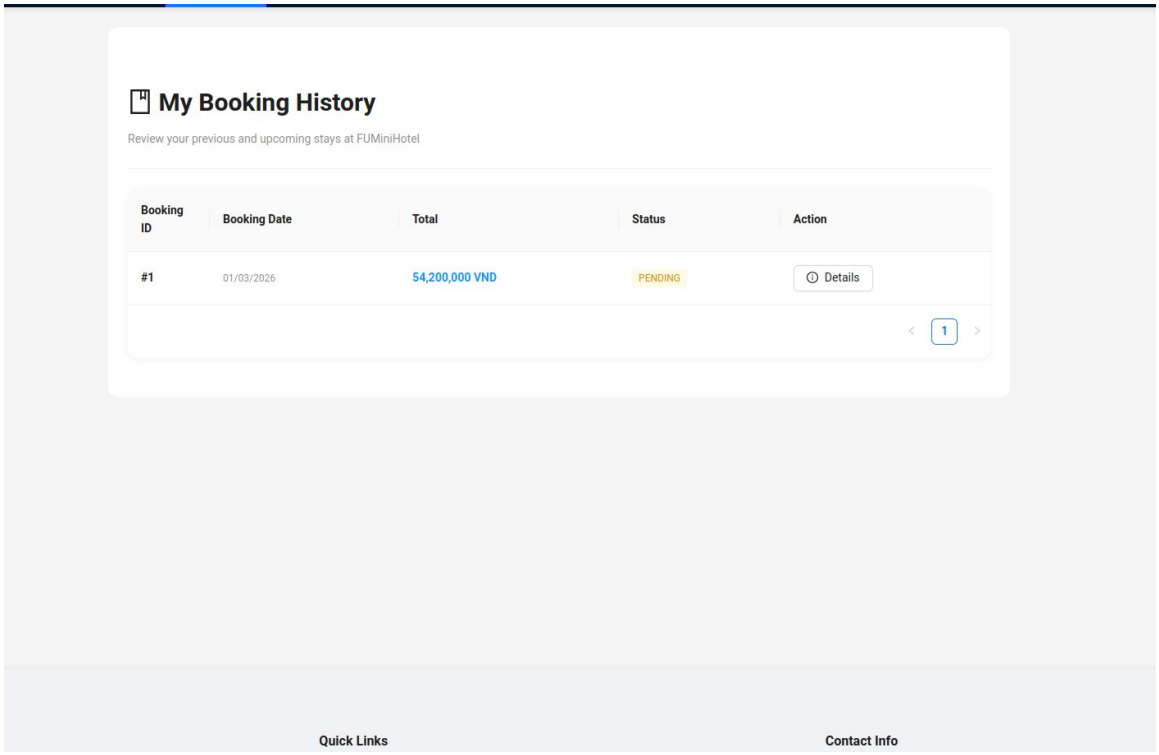
```java
    @PutMapping("/{id}/status")
    public ResponseEntity<Object> updateBookingStatus(@PathVariable Long id, @RequestBody BookingStatusUpdateDTO bookingStatusUpdateDTO) {
        try {
            BookingReservation bookings = bookingReservationService.getBookingReservationById(id);
            bookings.setBookingStatus(bookingStatusUpdateDTO.getStatus());
            bookingReservationService.updateBookingReservation(id, bookings);
        }
        catch (Exception e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
        return ResponseEntity.ok( body: "Booking status updated");
    }

}
```

3.4.5. Result


Book Your Stay
Select Rooms
Set Dates
Review & Confirm
Choose from available rooms
Room 101
Single
3,000,000 VND / Night
Room 102
Single
3,200,000 VND / Night
Room 201
Double
5,000,000 VND / Night
Room 202
Double
5,500,000 VND / Night
Room 301
Deluxe
120 VND / Night
Previous
Next

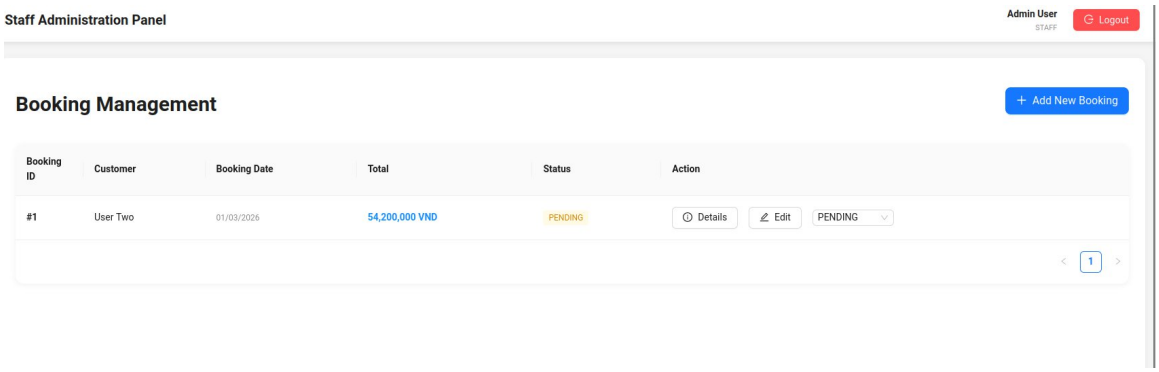

Book Your Stay
Select Rooms
Set Dates
Review & Confirm
Set dates for each room
Room 101 - Single
3,000,000 VND / night
Check-in:
Check-in Date
Check-out:
Check-out Date
Please select check-in and check-out dates
Room 102 - Single
3,200,000 VND / night
Check-in:
Check-in Date
Check-out:
Check-out Date
Please select check-in and check-out dates
Previous
Next

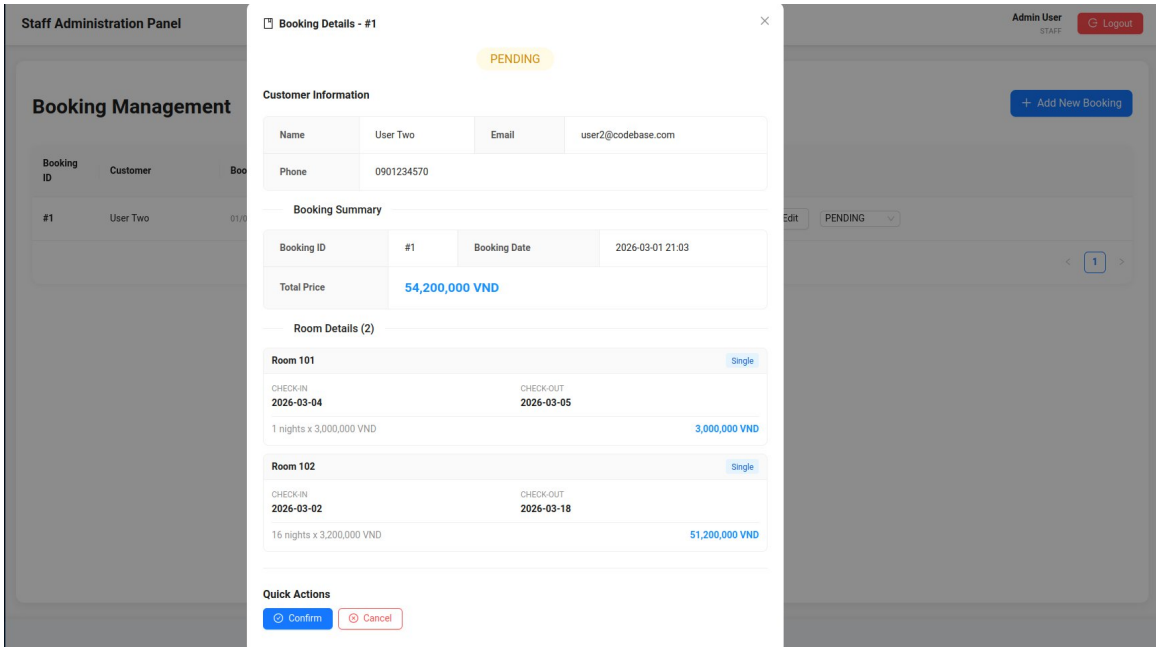




Quick Links

Contact Info





4. FRONTEND: DỊCH VỤ & HIỂN THỊ DỮ LIỆU

4.1. Cấu hình Core & Bảo mật (Core Configuration)

4.1.1. axiosInstance.js (Kết nối API)

- Chức năng: Khởi tạo thực thể Axios để giao tiếp với Backend.

- Chi tiết: Cấu hình baseURL trỏ tới Server Spring Boot và thiết lập Content-Type: application/json.

```
import ...
const axiosInstance : AxiosInstance = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});
  Michaelht271, Today • finish as3
// Request Interceptor
axiosInstance.interceptors.request.use(  Michaelht271
  onFulfilled: (config) => {
    // Don't add token for login/register requests
    const isPublicAuth = config.url.includes(API_ENDPOINTS.AUTH.LOGIN) || config.url.includes(API_ENDPOINTS.AUTH.REGISTER);

    if (!isPublicAuth) {
        const token : string = localStorage.getItem( key: 'token');
        if (token) {
          config.headers.Authorization = token.startsWith('Bearer ') ? token : `Bearer ${token}`;
        }
    }
    return config;
  },
  onRejected: (error) => {
    return Promise.reject(error);
  }
);
```

```
// Response Interceptor
axiosInstance.interceptors.response.use(  Michaelht271
  onFulfilled: (response) => response,
  onRejected: (error) => {
    // Log the error in console for debugging (helpful for 403)
    console.error(`API Error [${error.response?.status}]:`, error.response?.data || error.message);

    if (error.response?.status === 401) {
      localStorage.removeItem( key: 'token');
      localStorage.removeItem( key: 'user');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default axiosInstance;  Show usages  Michaelht271
```

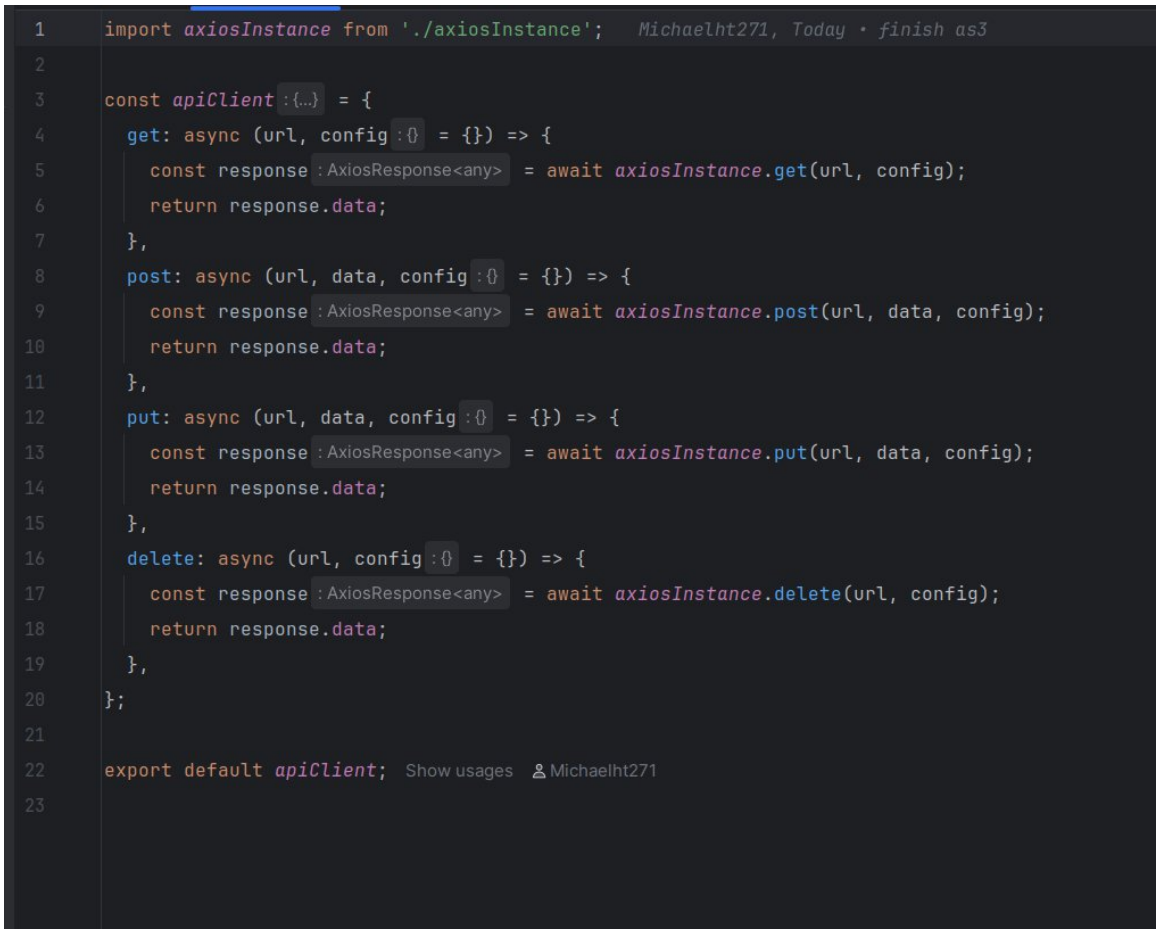
4.1.2. apiClient.js (Interceptors & JWT)

- Chức năng: Tự động đính kèm Token vào mọi yêu cầu gửi đi.

- Logic:

  - Request Interceptor: Kiểm tra localStorage, nếu có Token sẽ thêm vào header Authorization: Bearer <Token>.

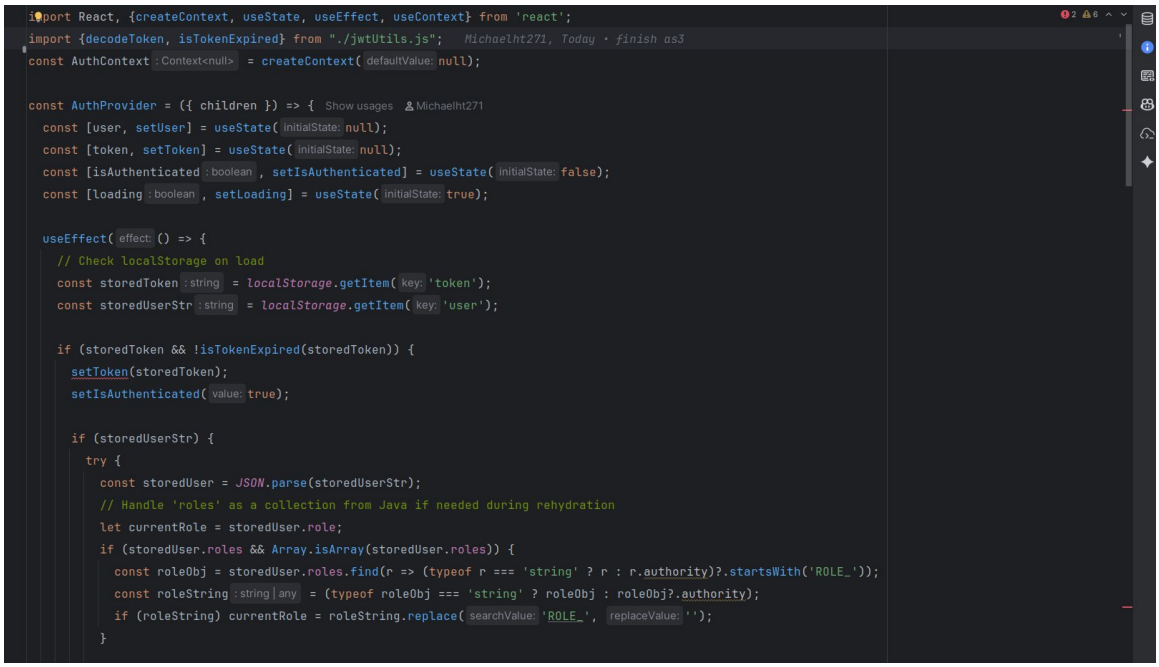
  - Response Interceptor: Bắt lỗi 401 (Unauthorized) để tự động đăng xuất người dùng nếu Token hết hạn.



```
import axiosInstance from './axiosInstance';   Michaelht271, Today • finish as3

const apiClient :{…} = {
  get: async (url, config :{} = {}) => {
    const response :AxiosResponse<any> = await axiosInstance.get(url, config);
    return response.data;
  },
  post: async (url, data, config :{} = {}) => {
    const response :AxiosResponse<any> = await axiosInstance.post(url, data, config);
    return response.data;
  },
  put: async (url, data, config :{} = {}) => {
    const response :AxiosResponse<any> = await axiosInstance.put(url, data, config);
    return response.data;
  },
  delete: async (url, config :{} = {}) => {
    const response :AxiosResponse<any> = await axiosInstance.delete(url, config);
    return response.data;
  },
};

export default apiClient;  Show usages  Michaelht271
```

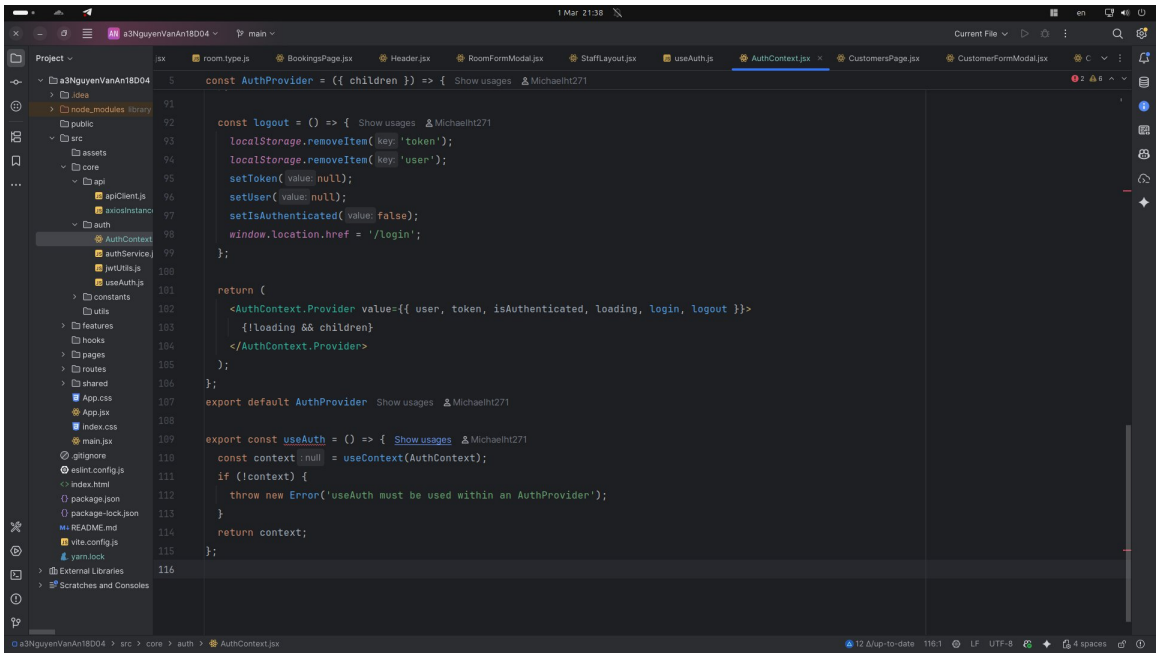
4.1.3. AuthContext.jsx (Quản lý trạng thái toàn cục)

- Chức năng: Lưu trữ thông tin người dùng và trạng thái đăng nhập xuyên suốt ứng dụng.
- Logic: Sử dụng useContext và useState để cung cấp các hàm login, logout cho các component con.



```
import React, {createContext, useState, useEffect, useContext} from 'react';
import {decodeToken, isTokenExpired} from "./jwtUtils.js";   Michaelht271, Today • finish as3
const AuthContext : Context<null> = createContext( defaultValue: null);

const AuthProvider = ({ children }) => {  Show usages   Michaelht271
  const [user, setUser] = useState( initialState: null);
  const [token, setToken] = useState( initialState: null);
  const [isAuthenticated : boolean , setIsAuthenticated] = useState( initialState: false);
  const [loading : boolean , setLoading] = useState( initialState: true);

  useEffect( effect: () => {
    // Check localStorage on load
    const storedToken : string = localStorage.getItem( key: 'token');
    const storedUserStr : string = localStorage.getItem( key: 'user');

    if (storedToken && !isTokenExpired(storedToken)) {
      setToken(storedToken);
      setIsAuthenticated( value: true);

      if (storedUserStr) {
        try {
          const storedUser = JSON.parse(storedUserStr);
          // Handle 'roles' as a collection from Java if needed during rehydration
          let currentRole = storedUser.role;
          if (storedUser.roles && Array.isArray(storedUser.roles)) {
            const roleObj = storedUser.roles.find(r => (typeof r === 'string' ? r : r.authority)?.startsWith('ROLE_'));
            const roleString : string | any = (typeof roleObj === 'string' ? roleObj : roleObj?.authority);
            if (roleString) currentRole = roleString.replace( searchValue: 'ROLE_', replaceValue: '');
          }
```

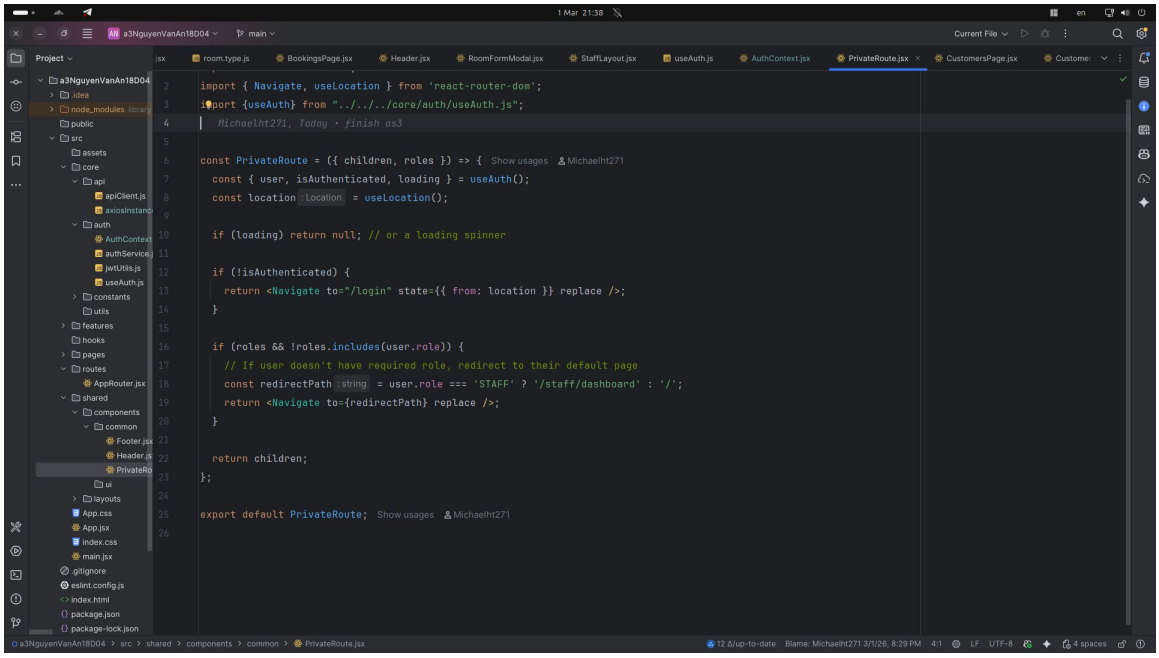
```
          const id = storedUser.customerID || storedUser.customerId || storedUser.id;
          const normalizedUser = {
            ...storedUser,
            role: currentRole || 'CUSTOMER',
            customerID: id,
            customerId: id,
            id: id,
            fullName: storedUser.fullName || storedUser.customerFullName,
            customerFullName: storedUser.fullName || storedUser.customerFullName,
          };
          setUser(normalizedUser);
        } catch (e) {
          console.error("Failed to parse stored user", e);
        }
      } else {

        // Fallback: decode token
        const decoded : JwtPayload = decodeToken(storedToken);
        if (decoded) {
            const authorities = decoded.authorities || [];
            const rawRole : string = authorities.find(a : T => a.startsWith('ROLE_')) || 'CUSTOMER';
            const normalizedRole : string = rawRole.replace( searchValue: 'ROLE_', replaceValue: '');

            const id = decoded.customerID || decoded.customerId || decoded.id;
            setUser( value: {
                email: decoded.sub,
                emailAddress: decoded.sub,
```



```
const AuthProvider = ({ children }) => {

  const logout = () => {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    setToken(null);
    setUser(null);
    setIsAuthenticated(false);
    window.location.href = '/login';
  };

  return (
    <AuthContext.Provider value={{ user, token, isAuthenticated, loading, login, logout }}>
      {!loading && children}
    </AuthContext.Provider>
  );
};
export default AuthProvider

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};
```

4.1.4. PrivateRoute.jsx (Phân quyền truy cập)

- Chức năng: Bảo vệ các tuyến đường (routes) yêu cầu đăng nhập.
- Logic: Kiểm tra Role của người dùng. Nếu khách hàng cố tình truy cập trang của nhân viên (Staff), hệ thống sẽ chuyển hướng về trang /unauthorized.



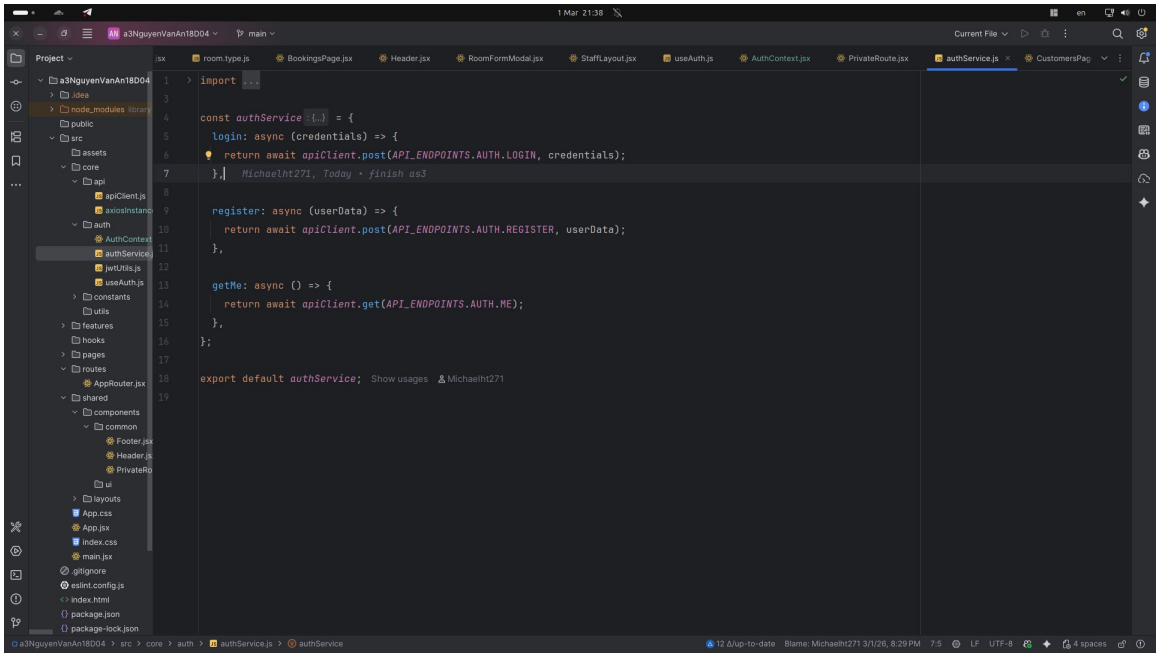
```
import { Navigate, useLocation } from 'react-router-dom';
import {useAuth} from "../../../core/auth/useAuth.js";
  Michaelht271, Today • finish as3

const PrivateRoute = ({ children, roles }) => {
  const { user, isAuthenticated, loading } = useAuth();
  const location = useLocation();

  if (loading) return null; // or a loading spinner

  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  if (roles && !roles.includes(user.role)) {
    // If user doesn't have required role, redirect to their default page
    const redirectPath = user.role === 'STAFF' ? '/staff/dashboard' : '/';
    return <Navigate to={redirectPath} replace />;
  }

  return children;
};

export default PrivateRoute;
```

4.2. Feature: Authentication (Đăng nhập & Đăng ký)

4.2.1. authService.js

- Chức năng: Chứa các hàm gọi API /api/auth/login và /api/auth/register.

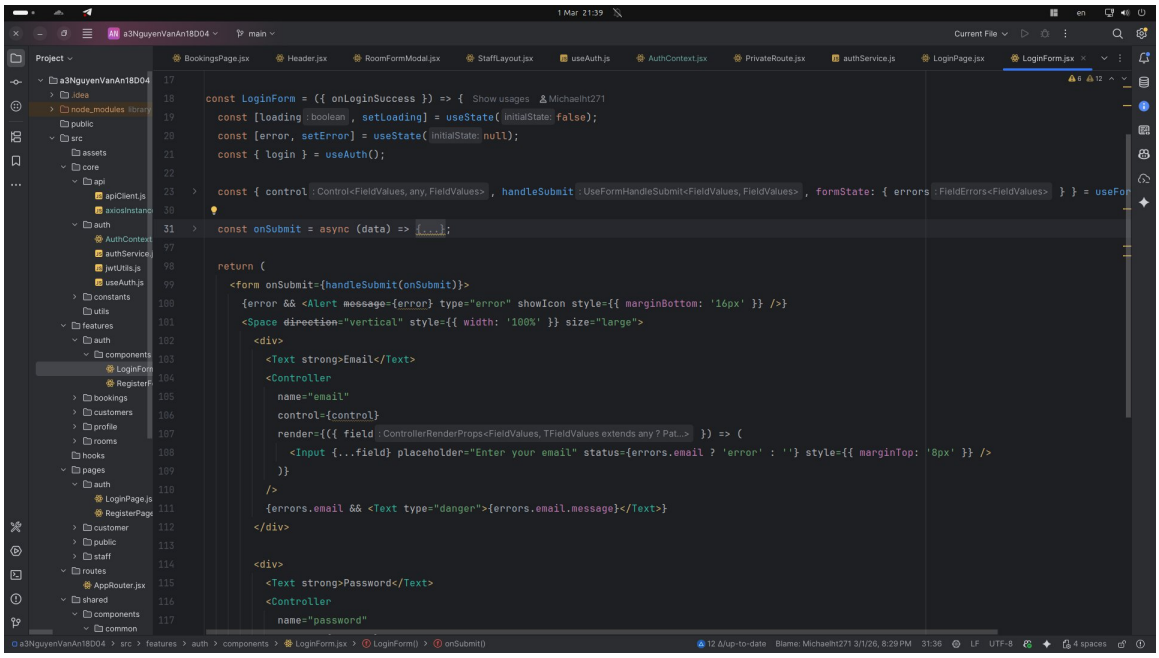
- Kết quả: Trả về dữ liệu người dùng và Token cho AuthContext xử lý.



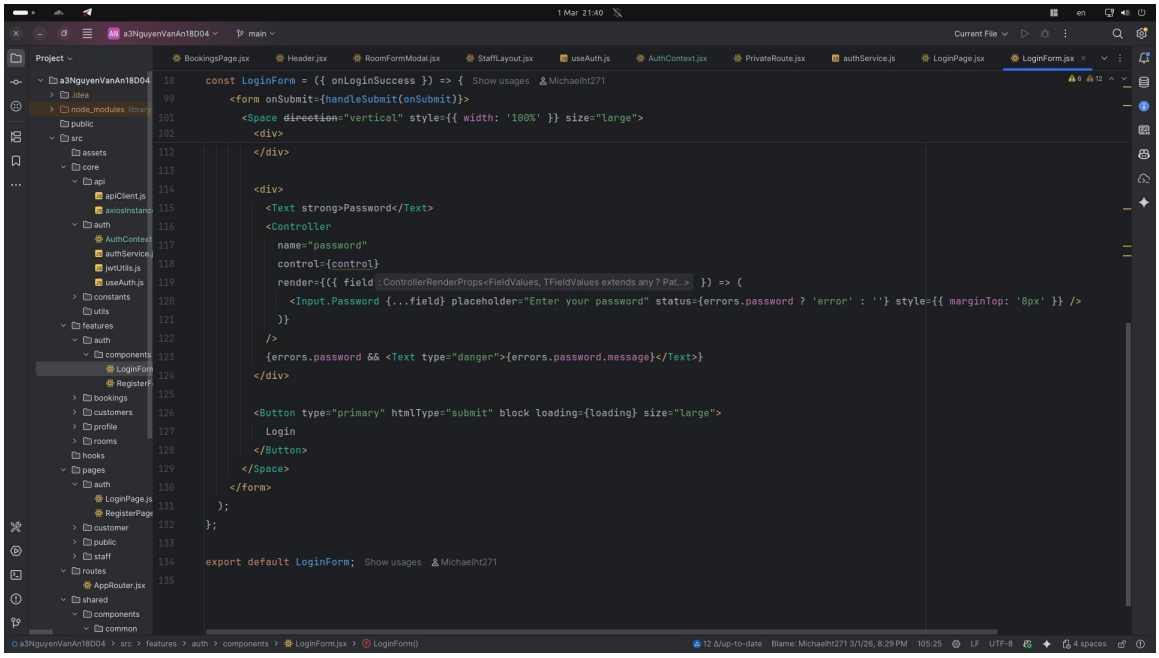
4.2.2. LoginForm.jsx & RegisterForm.jsx

- Chức năng: Xử lý nhập liệu từ người dùng.

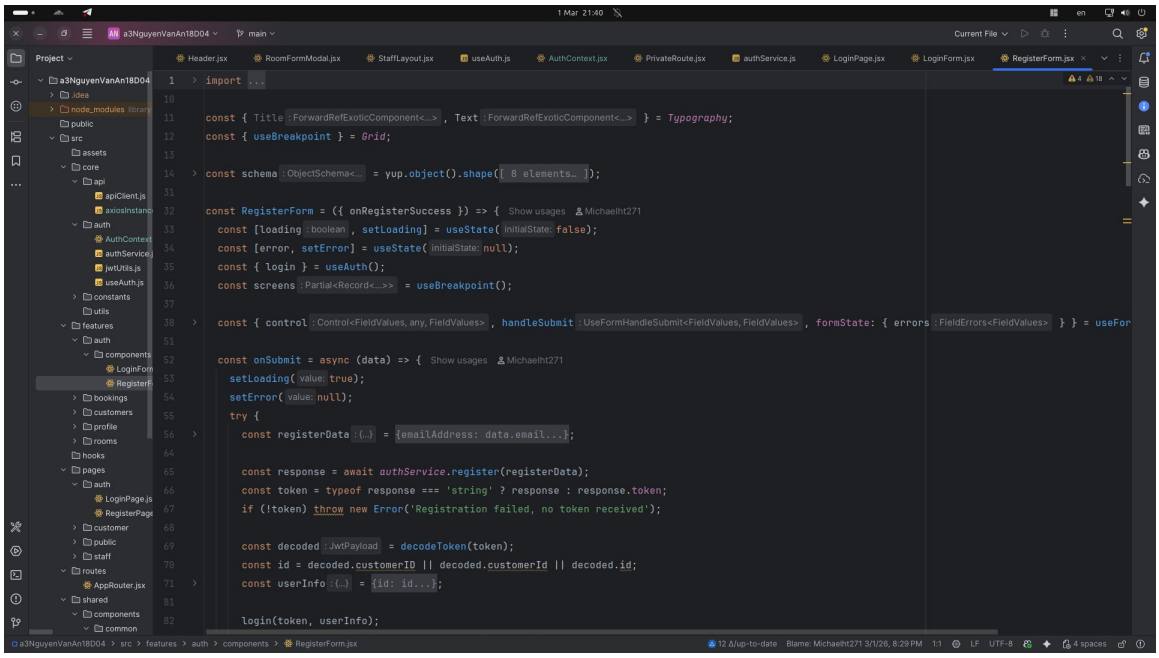
- Công nghệ: Sử dụng react-hook-form để quản lý state của Form và antd để tạo giao diện UI.



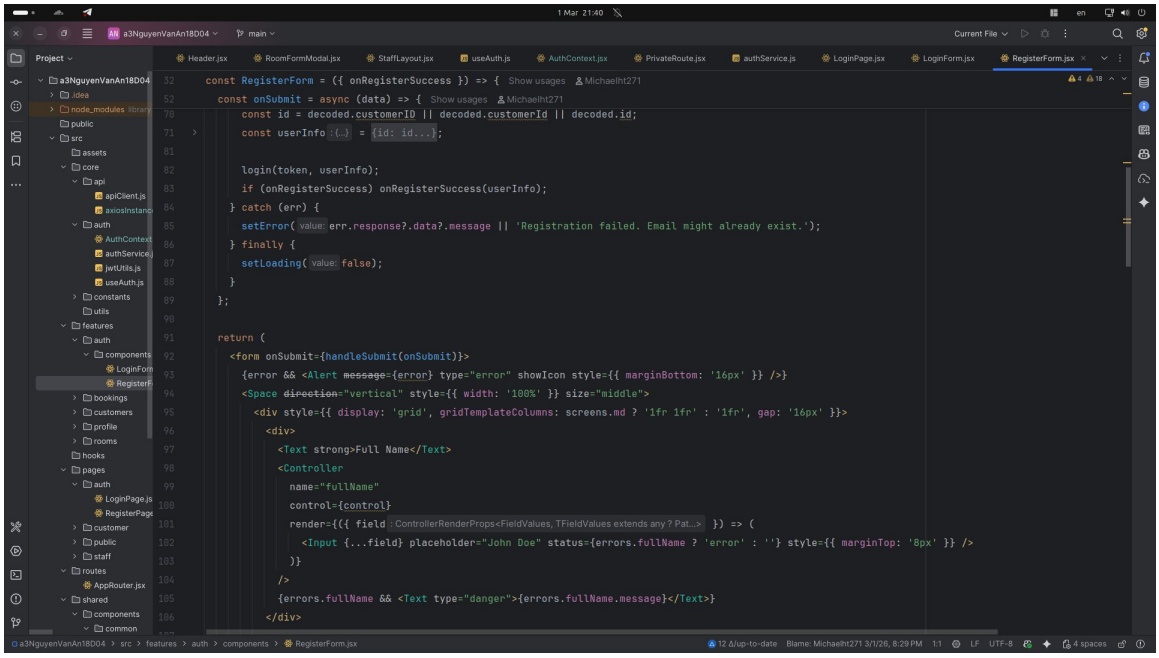
```
const LoginForm = ({ onLoginSuccess }) => {  Show usages  Michaelht271
  const [loading : boolean , setLoading] = useState( initialState: false);
  const [error, setError] = useState( initialState: null);
  const { login } = useAuth();

  const { control : Control<FieldValues, any, FieldValues> , handleSubmit : UseFormHandleSubmit<FieldValues, FieldValues> , formState: { errors : FieldErrors<FieldValues> } } = useFor

  const onSubmit = async (data) => {...};

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      {error && <Alert message={error} type="error" showIcon style={{ marginBottom: '16px' }} />}
      <Space direction="vertical" style={{ width: '100%' }} size="large">
        <div>
          <Text strong>Email</Text>
          <Controller
            name="email"
            control={control}
            render={({{ field : ControllerRenderProps<FieldValues, TFieldValues extends any ? Pat...> }) => (
              <Input {...field} placeholder="Enter your email" status={errors.email ? 'error' : ''} style={{ marginTop: '8px' }} />
            )}
          />
          {errors.email && <Text type="danger">{errors.email.message}</Text>}
        </div>

        <div>
          <Text strong>Password</Text>
          <Controller
            name="password"
```



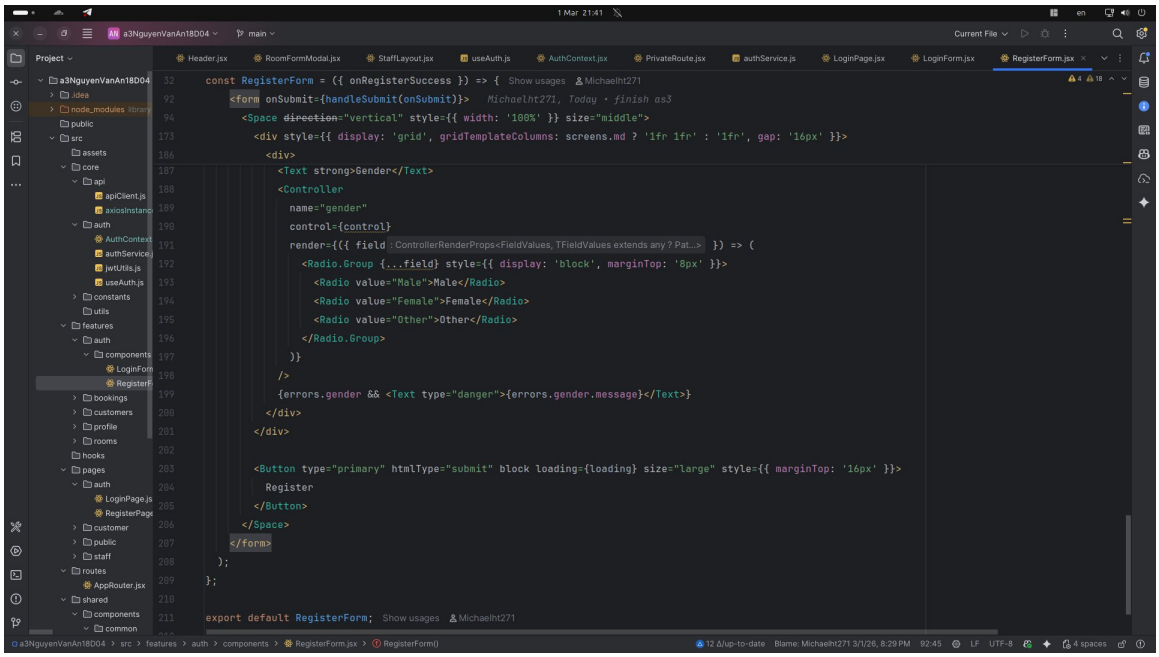
```
const LoginForm = ({ onLoginSuccess }) => {  Show usages  Michaelht271
    <form onSubmit={handleSubmit(onSubmit)}>
      <Space direction="vertical" style={{ width: '100%' }} size="large">
        <div>
        </div>

        <div>
          <Text strong>Password</Text>
          <Controller
            name="password"
            control={control}
            render={({{ field : ControllerRenderProps<FieldValues, TFieldValues extends any ? Pat...> }) => (
              <Input.Password {...field} placeholder="Enter your password" status={errors.password ? 'error' : ''} style={{ marginTop: '8px' }} />
            )}
          />
          {errors.password && <Text type="danger">{errors.password.message}</Text>}
        </div>

        <Button type="primary" htmlType="submit" block loading={loading} size="large">
          Login
        </Button>
      </Space>
    </form>
  );
};

export default LoginForm;  Show usages  Michaelht271
```



```
import ...

const { Title : ForwardRefExoticComponent<...> , Text : ForwardRefExoticComponent<...> } = Typography;
const { useBreakpoint } = Grid;

const schema : ObjectSchema<... = yup.object().shape([ 8 elements… ]);

const RegisterForm = ({ onRegisterSuccess }) => {  Show usages  Michaelht271
  const [loading : boolean , setLoading] = useState( initialState: false);
  const [error, setError] = useState( initialState: null);
  const { login } = useAuth();
  const screens : Partial<Record<...>> = useBreakpoint();

  const { control : Control<FieldValues, any, FieldValues> , handleSubmit : UseFormHandleSubmit<FieldValues, FieldValues> , formState: { errors : FieldErrors<FieldValues>  } } = useFor

  const onSubmit = async (data) => {  Show usages  Michaelht271
    setLoading( value: true);
    setError( value: null);
    try {
      const registerData : {…} = {emailAddress: data.email...};

      const response = await authService.register(registerData);
      const token = typeof response === 'string' ? response : response.token;
      if (!token) throw new Error('Registration failed, no token received');

      const decoded : JwtPayload = decodeToken(token);
      const id = decoded.customerID || decoded.customerId || decoded.id;
      const userInfo : {…} = {id: id...};

      login(token, userInfo);
```



```
const RegisterForm = ({ onRegisterSuccess }) => {  Show usages  Michaelht271
  const onSubmit = async (data) => {  Show usages  Michaelht271
      const id = decoded.customerID || decoded.customerId || decoded.id;
      const userInfo : {…} = {id: id...};

      login(token, userInfo);
      if (onRegisterSuccess) onRegisterSuccess(userInfo);
    } catch (err) {
      setError( value: err.response?.data?.message || 'Registration failed. Email might already exist.');
    } finally {
      setLoading( value: false);
    }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      {error && <Alert message={error} type="error" showIcon style={{ marginBottom: '16px' }} />}
      <Space direction="vertical" style={{ width: '100%' }} size="middle">
        <div style={{ display: 'grid', gridTemplateColumns: screens.md ? '1fr 1fr' : '1fr', gap: '16px' }}>
          <div>
            <Text strong>Full Name</Text>
            <Controller
              name="fullName"
              control={control}
              render={({ field : ControllerRenderProps<FieldValues, TFieldValues extends any ? Pat...> }) => (
                <Input {...field} placeholder="John Doe" status={errors.fullName ? 'error' : ''} style={{ marginTop: '8px' }} />
              )}
            />
            {errors.fullName && <Text type="danger">{errors.fullName.message}</Text>}
          </div>
```

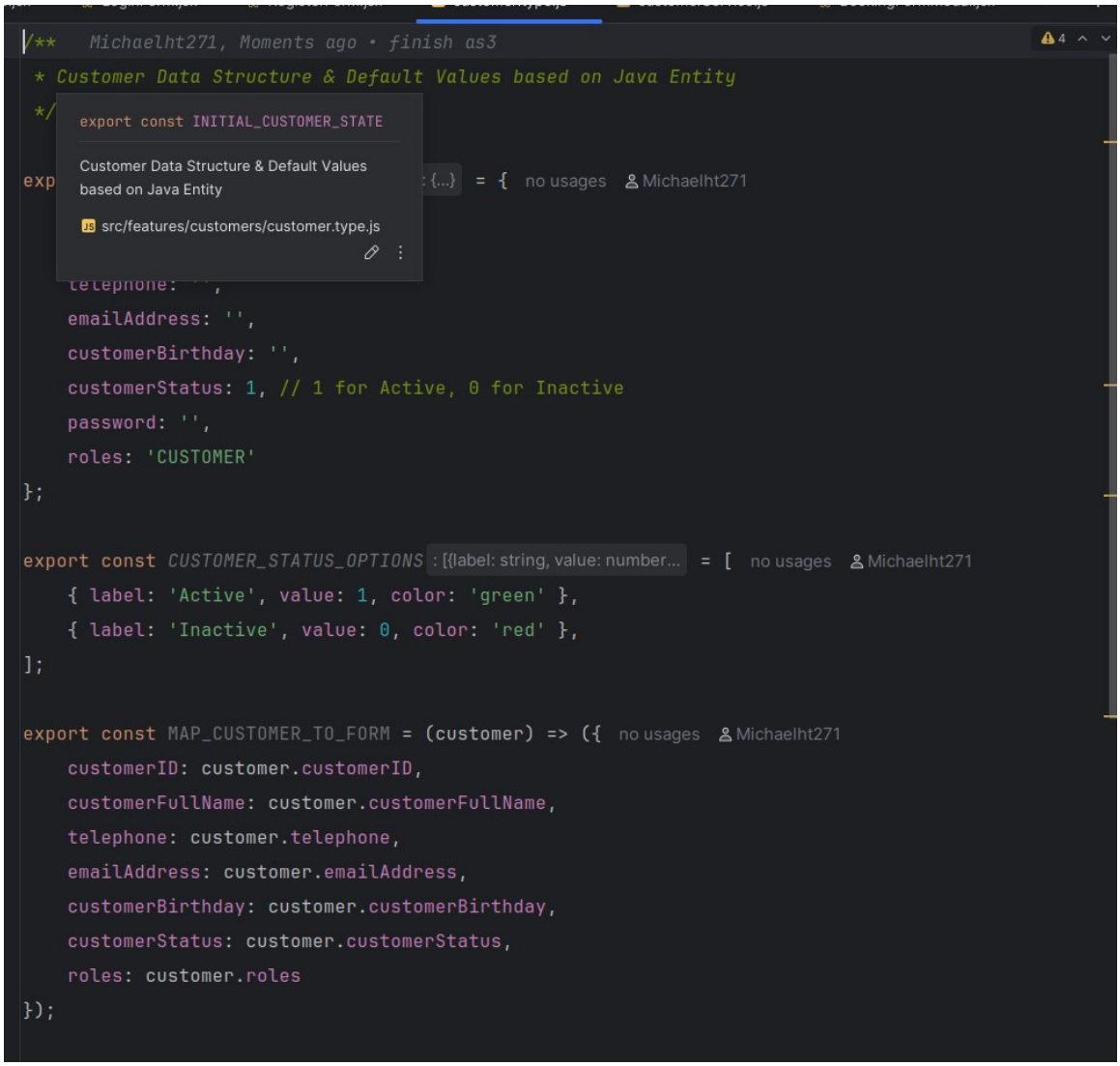

1 Mar 21:41
a3NguyenVanAn18D04
main
Current File
Header.jsx
RoomFormModal.jsx
StaffLayout.jsx
useAuth.js
AuthContext.jsx
PrivateRoute.jsx
authService.js
LoginPage.jsx
LoginForm.jsx
RegisterForm.jsx
Project
const RegisterForm = ({ onRegisterSuccess }) => { Show usages Michaelht271
<form onSubmit={handleSubmit(onSubmit)}> Michaelht271, Today • finish as3
<Space direction="vertical" style={{ width: '100%' }} size="middle">
<div style={{ display: 'grid', gridTemplateColumns: screens.md ? '1fr 1fr' : '1fr', gap: '16px' }}>

<Text strong>Gender</Text>
<Controller
name="gender"
control={control}
render={({ field ControllerRenderProps<FieldValues, TFieldValues extends any ? Pat...> }) => (
<Radio.Group {...field} style={{ display: 'block', marginTop: '8px' }}>
<Radio value="Male">Male</Radio>
<Radio value="Female">Female</Radio>
<Radio value="Other">Other</Radio>
</Radio.Group>
)}
/>
{errors.gender && <Text type="danger">{errors.gender.message}</Text>}

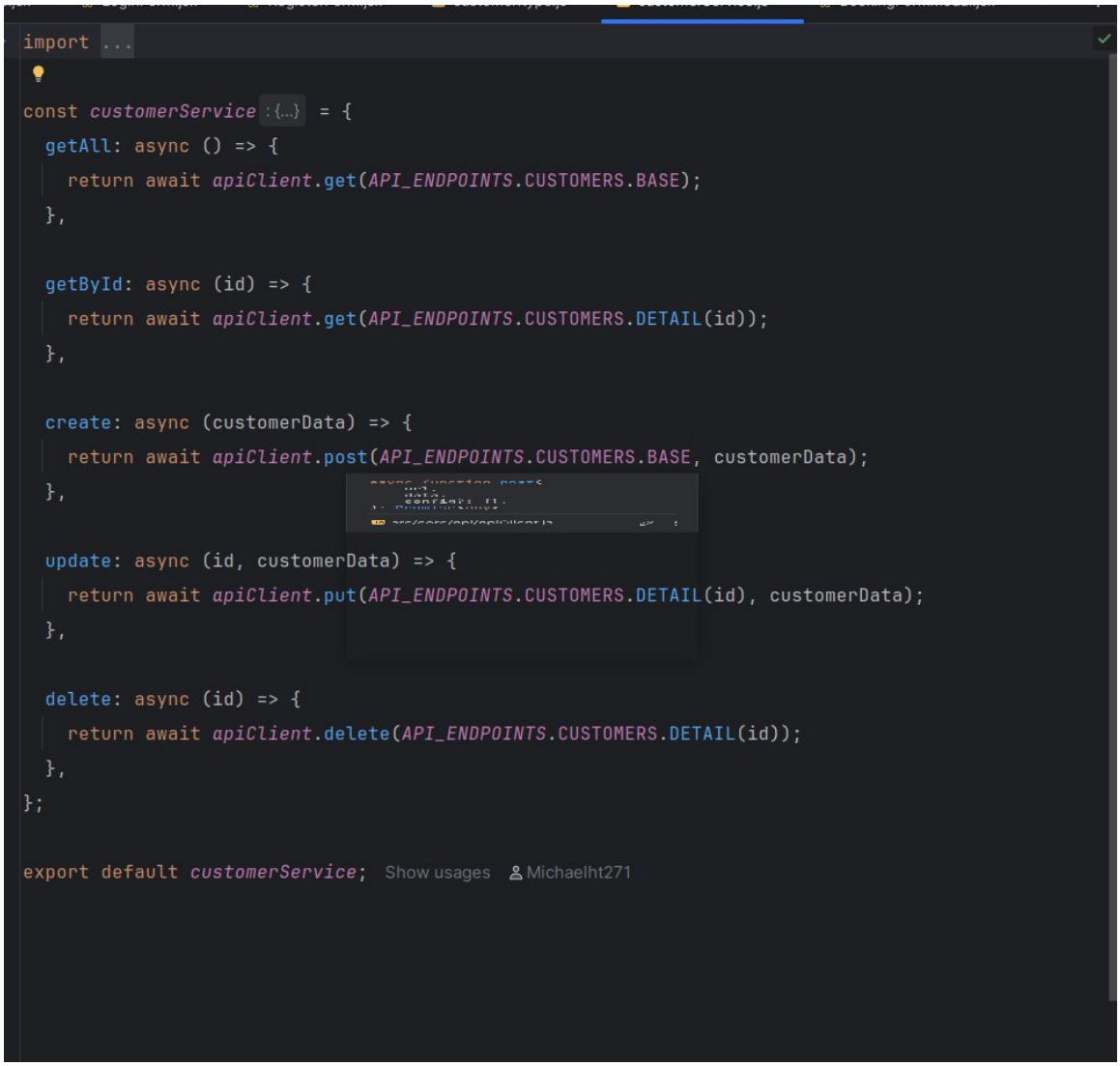
<Button type="primary" htmlType="submit" block loading={loading} size="large" style={{ marginTop: '16px' }}>
Register
</Button>
</Space>
</form>
);
};
export default RegisterForm; Show usages Michaelht271
a3NguyenVanAn18D04 > src > features > auth > components > RegisterForm.jsx > RegisterForm()
12 Δ/up-to-date
Blame: Michaelht271 3/1/26, 8:29 PM
92:45
LF
UTF-8
4 spaces

4.2. Customer Management

4.2.1. CustomerType

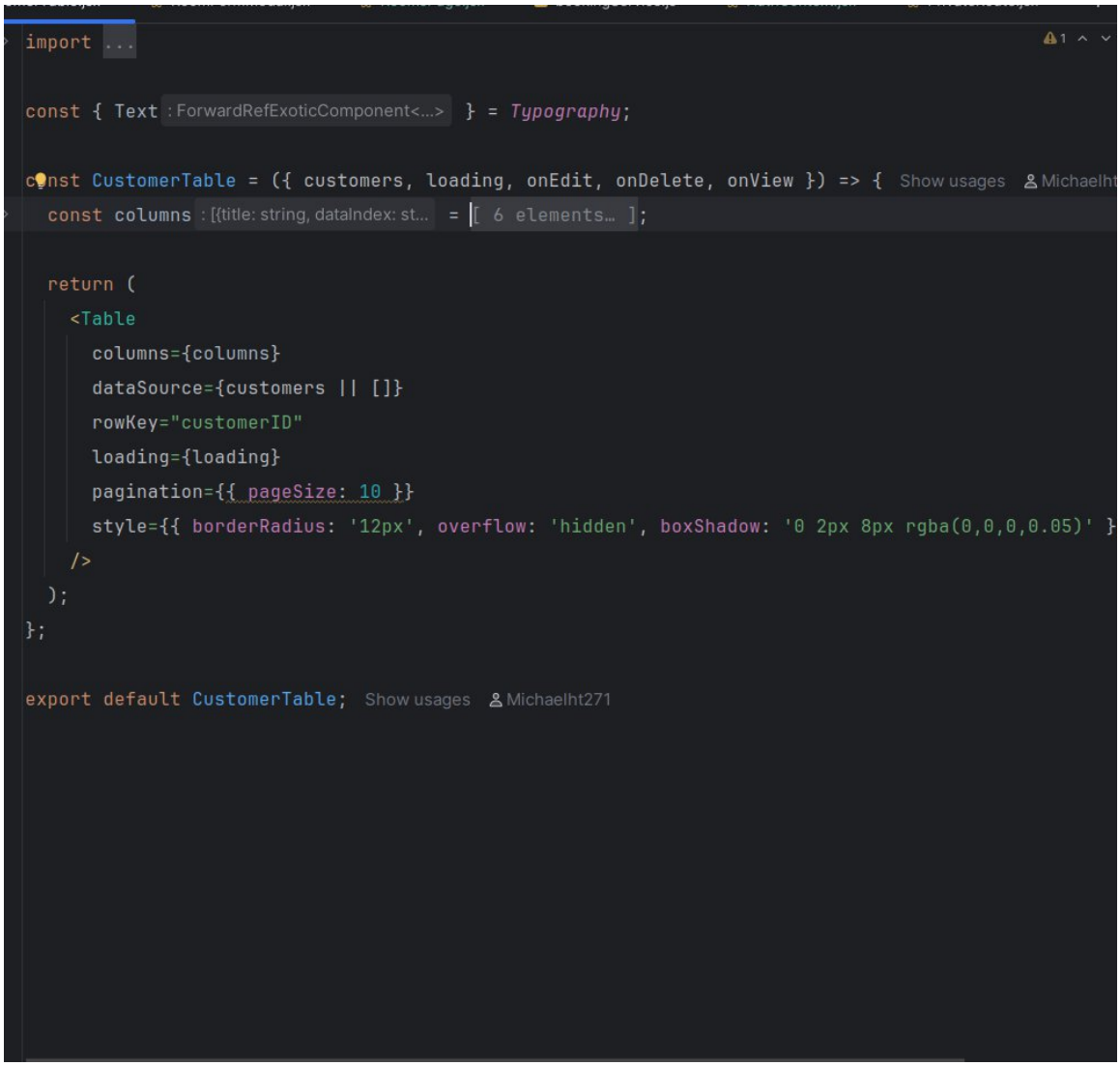


4.2.2. CustomerService

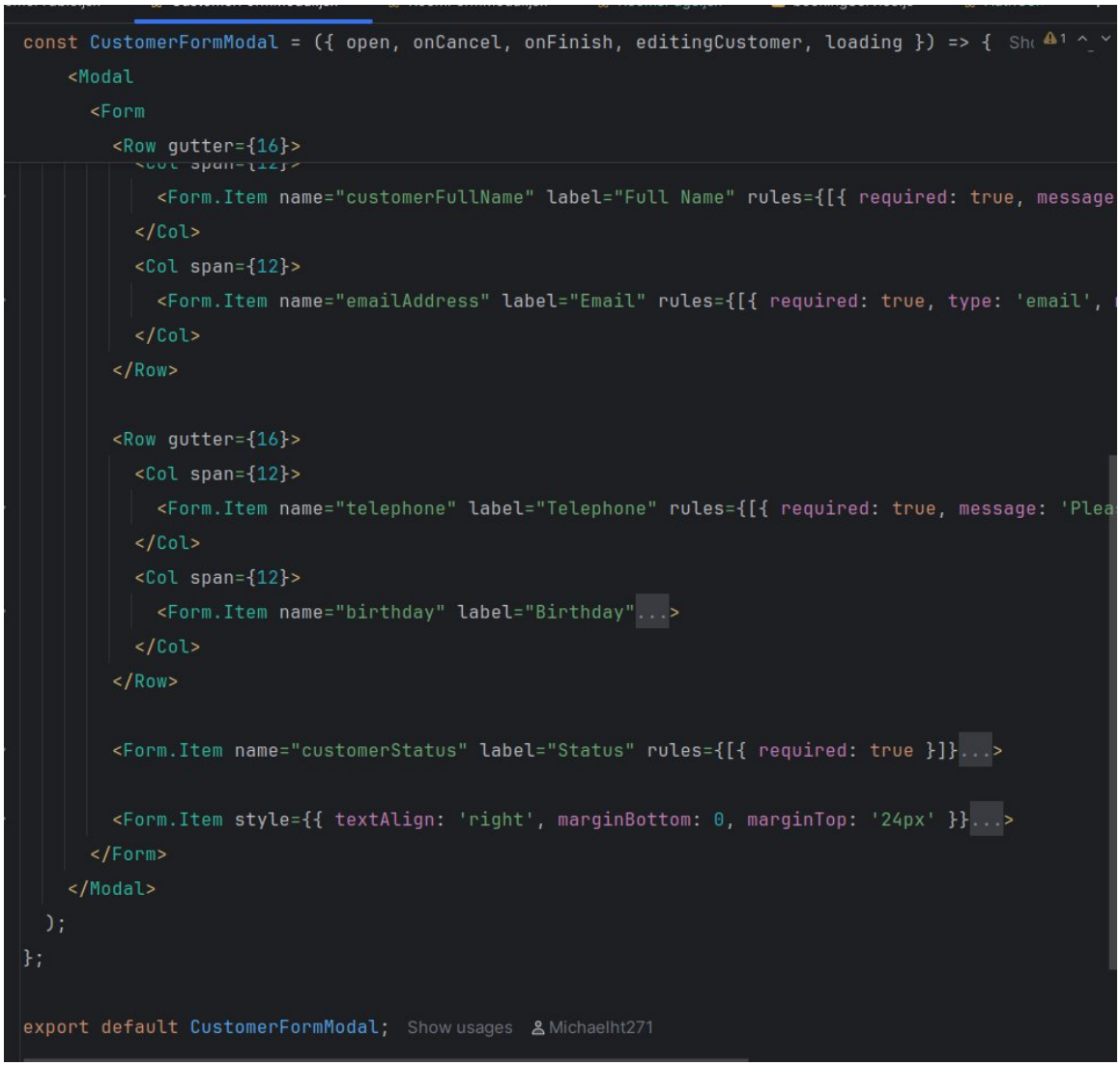


```
import ...

const customerService {…} = {
  getAll: async () => {
    return await apiClient.get(API_ENDPOINTS.CUSTOMERS.BASE);
  },

  getById: async (id) => {
    return await apiClient.get(API_ENDPOINTS.CUSTOMERS.DETAIL(id));
  },

  create: async (customerData) => {
    return await apiClient.post(API_ENDPOINTS.CUSTOMERS.BASE, customerData);
  },

  update: async (id, customerData) => {
    return await apiClient.put(API_ENDPOINTS.CUSTOMERS.DETAIL(id), customerData);
  },

  delete: async (id) => {
    return await apiClient.delete(API_ENDPOINTS.CUSTOMERS.DETAIL(id));
  },
};

export default customerService; Show usages  Michaelht271
```

4.2.3. CustomerTable

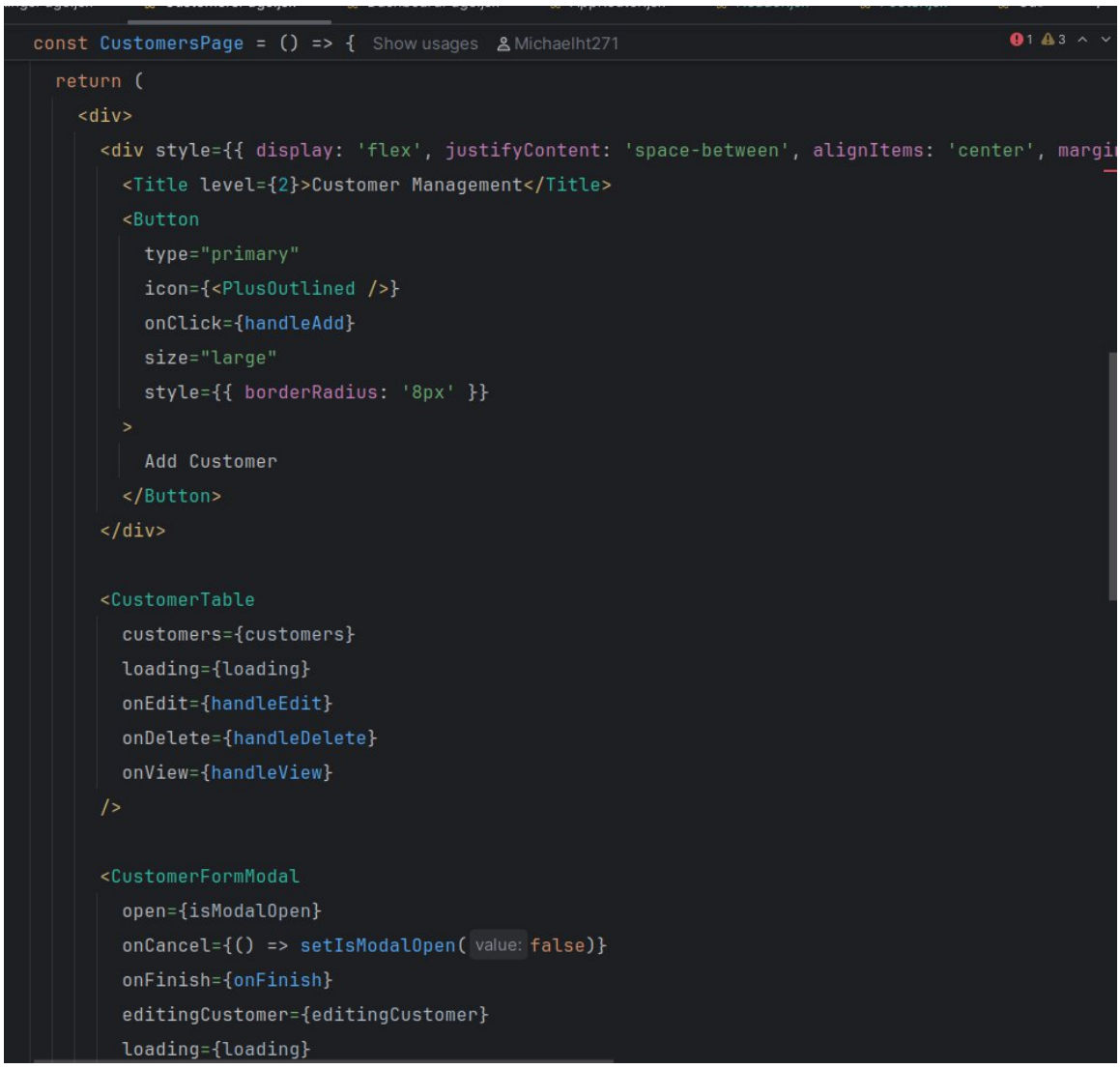


4.2.4. CustomerFormModal

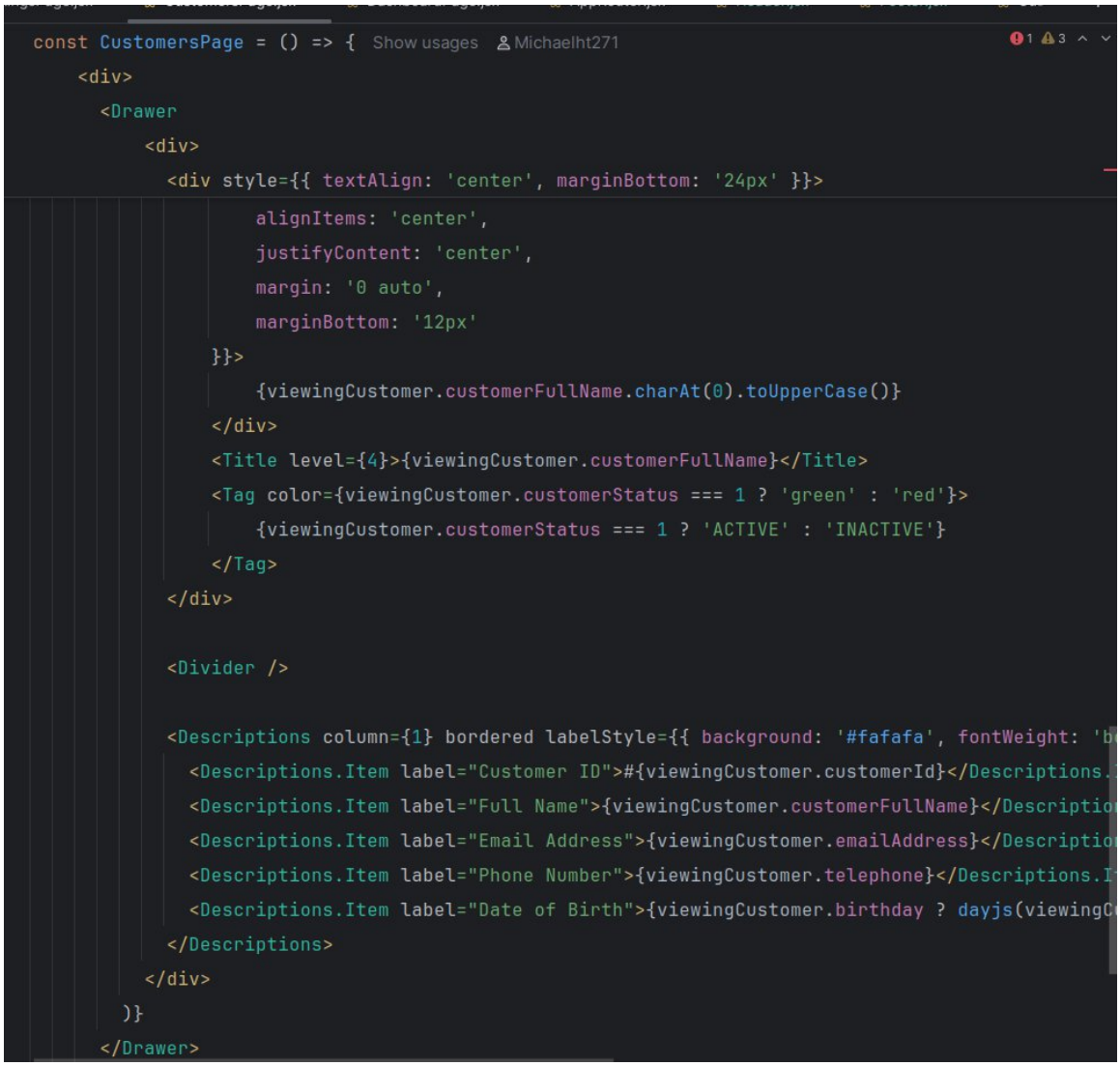


4.2.5. CustomerPages

```
import ...

const { Title : ForwardRefExoticComponent<...> } = Typography;

const CustomersPage = () => {  Show usages  Michaelht271
  const [customers : any[] , setCustomers] = useState( initialState: []);
  const [loading : boolean , setLoading] = useState( initialState: false);
  const [isModalOpen : boolean , setIsModalOpen] = useState( initialState: false);
  const [isDrawerOpen : boolean , setIsDrawerOpen] = useState( initialState: false);
  const [editingCustomer, setEditingCustomer] = useState( initialState: null);
  const [viewingCustomer, setViewingCustomer] = useState( initialState: null);

  const fetchCustomers = async () => {  Show usages  Michaelht271
    setLoading( value: true);
    try {...} catch (error) {
      message.error( content: 'Failed to fetch customers');
      setCustomers( value: []);
    } finally {
      setLoading( value: false);
    }
  };

  useEffect( effect: () => {...}, deps: []);

  const handleAdd = () => {...};

  const handleEdit = (record) => {...};

  const handleView = (record) => {  Show usages  Michaelht271
    setViewingCustomer(record);
```



```
const CustomersPage = () => {  Show usages  Michaelht271
  return (
    <div>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', margi
        <Title level={2}>Customer Management</Title>
        <Button
          type="primary"
          icon={<PlusOutlined />}
          onClick={handleAdd}
          size="large"
          style={{ borderRadius: '8px' }}
        >
          Add Customer
        </Button>
      </div>

      <CustomerTable
        customers={customers}
        loading={loading}
        onEdit={handleEdit}
        onDelete={handleDelete}
        onView={handleView}
      />

      <CustomerFormModal
        open={isModalOpen}
        onCancel={() => setIsModalOpen( value: false)}
        onFinish={onFinish}
        editingCustomer={editingCustomer}
        loading={loading}
```

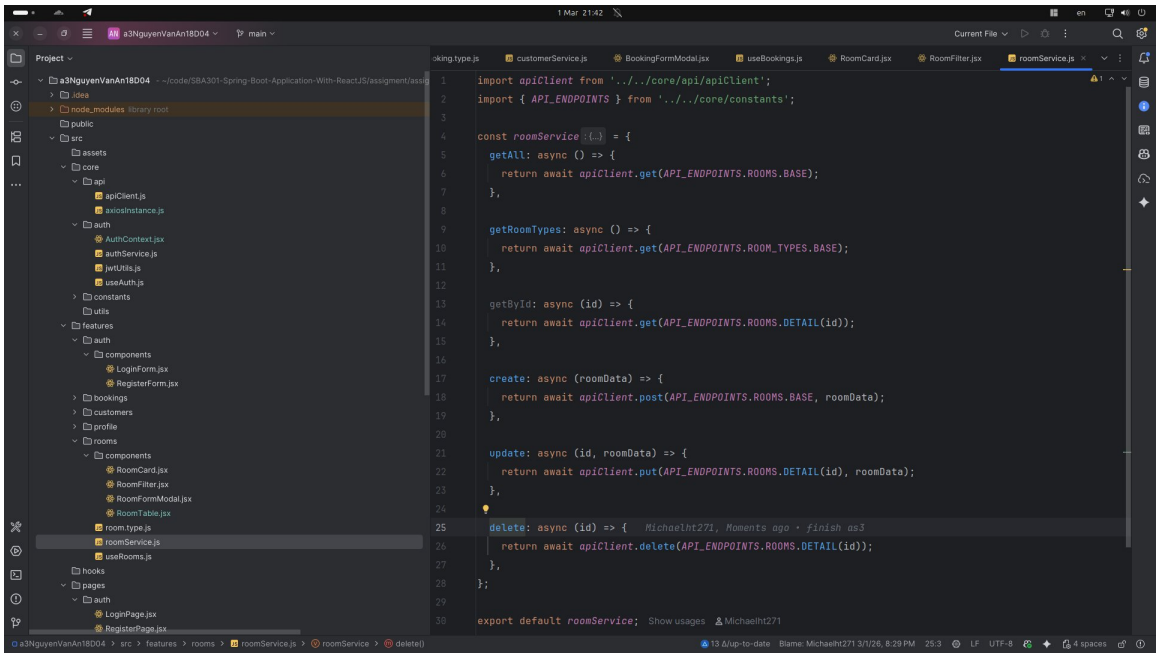


4.3. Feature: Quản lý Phòng (Room Management)

4.3.1. roomService.js

- Chức năng: Chứa các hàm getAllRooms, createRoom, updateRoom, deleteRoom.

- API: Gọi trực tiếp các Endpoints từ RoomInformationController của Backend.

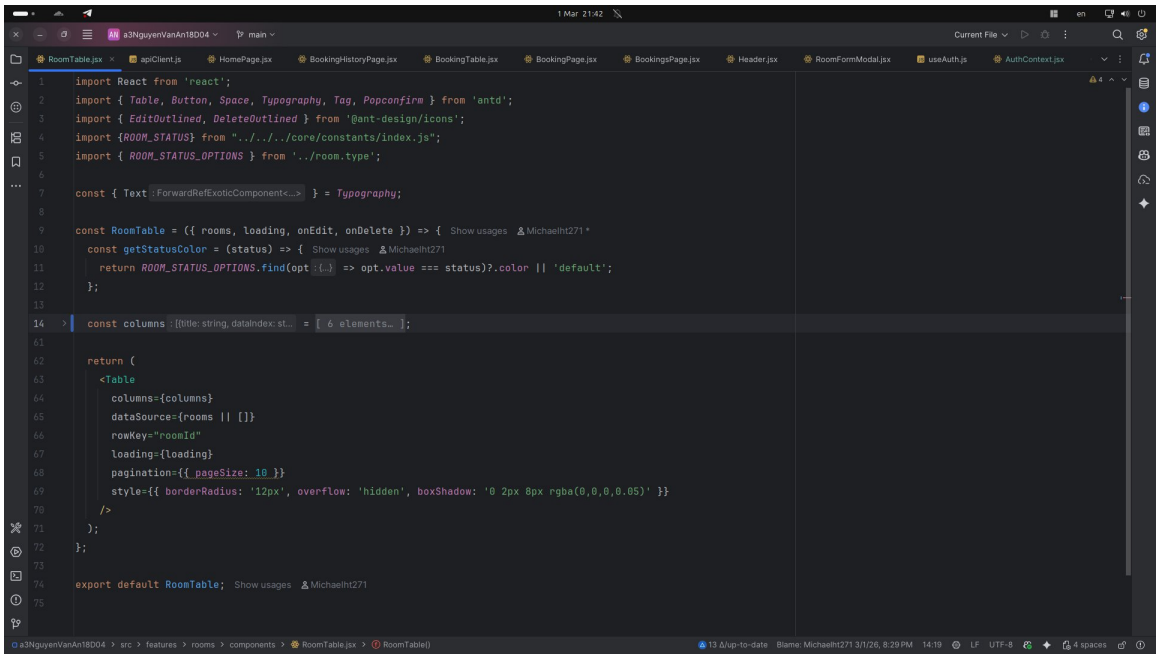


```
import apiClient from '../../core/api/apiClient';
import { API_ENDPOINTS } from '../../core/constants';

const roomService = {
  getAll: async () => {
    return await apiClient.get(API_ENDPOINTS.ROOMS.BASE);
  },

  getRoomTypes: async () => {
    return await apiClient.get(API_ENDPOINTS.ROOM_TYPES.BASE);
  },

  getById: async (id) => {
    return await apiClient.get(API_ENDPOINTS.ROOMS.DETAIL(id));
  },

  create: async (roomData) => {
    return await apiClient.post(API_ENDPOINTS.ROOMS.BASE, roomData);
  },

  update: async (id, roomData) => {
    return await apiClient.put(API_ENDPOINTS.ROOMS.DETAIL(id), roomData);
  },

  delete: async (id) => {   Michaelht271, Moments ago • finish as3
    return await apiClient.delete(API_ENDPOINTS.ROOMS.DETAIL(id));
  },
};

export default roomService;
```

4.3.2. RoomTable.jsx (Staff View)

- Chức năng: Hiển thị danh sách phòng dưới dạng bảng cho Nhân viên.

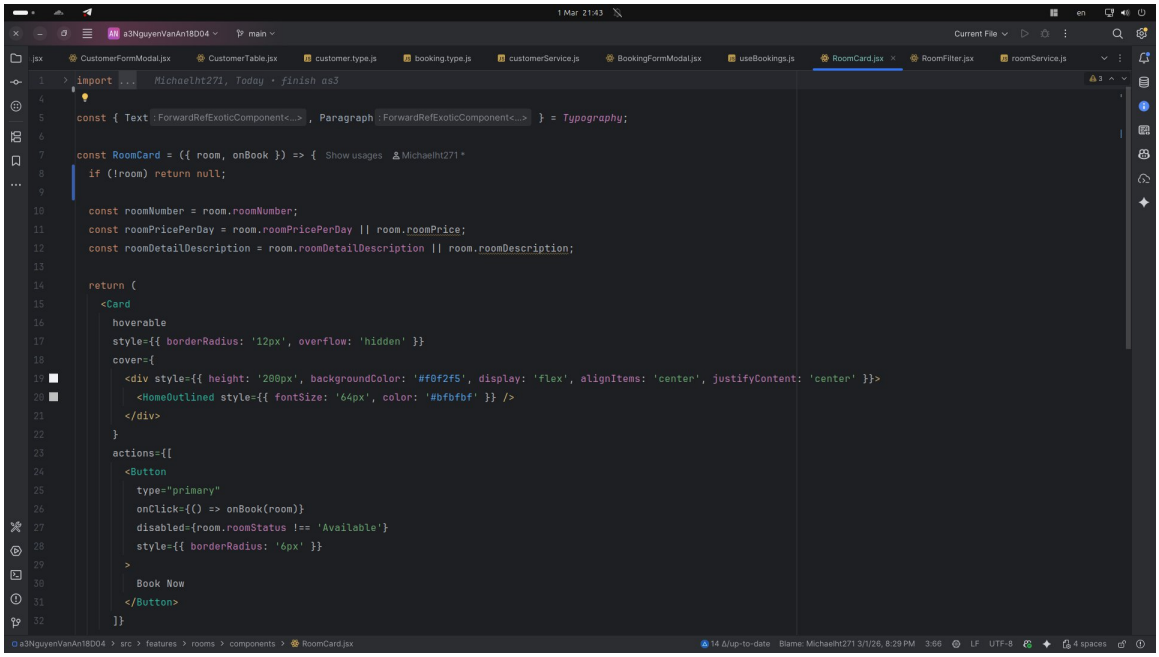
- Tính năng: Tích hợp nút Sửa (Edit) và Xóa (Delete).



```
import React from 'react';
import { Table, Button, Space, Typography, Tag, Popconfirm } from 'antd';
import { EditOutlined, DeleteOutlined } from '@ant-design/icons';
import {ROOM_STATUS} from "../../../core/constants/index.js";
import { ROOM_STATUS_OPTIONS } from '../room.type';

const { Text } = Typography;

const RoomTable = ({ rooms, loading, onEdit, onDelete }) => {
  const getStatusColor = (status) => {
    return ROOM_STATUS_OPTIONS.find(opt => opt.value === status)?.color || 'default';
  };

  const columns = [ 6 elements… ];

  return (
    <Table
      columns={columns}
      dataSource={rooms || []}
      rowKey="roomId"
      loading={loading}
      pagination={{ pageSize: 10 }}
      style={{ borderRadius: '12px', overflow: 'hidden', boxShadow: '0 2px 8px rgba(0,0,0,0.05)' }}
    />
  );
};

export default RoomTable;
```

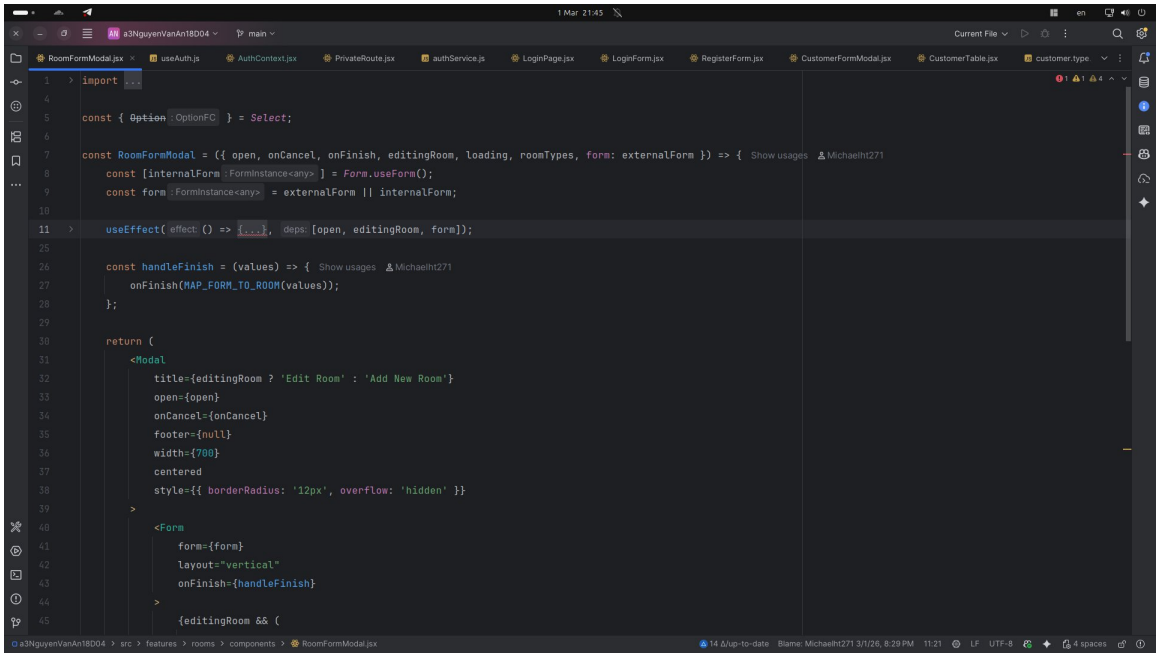
4.3.2. RoomCard.jsx & RoomFilter.jsx (Customer View)

- Chức năng: Hiển thị phòng dưới dạng thẻ trực quan và cung cấp bộ lọc tìm kiếm theo loại phòng hoặc giá tiền.

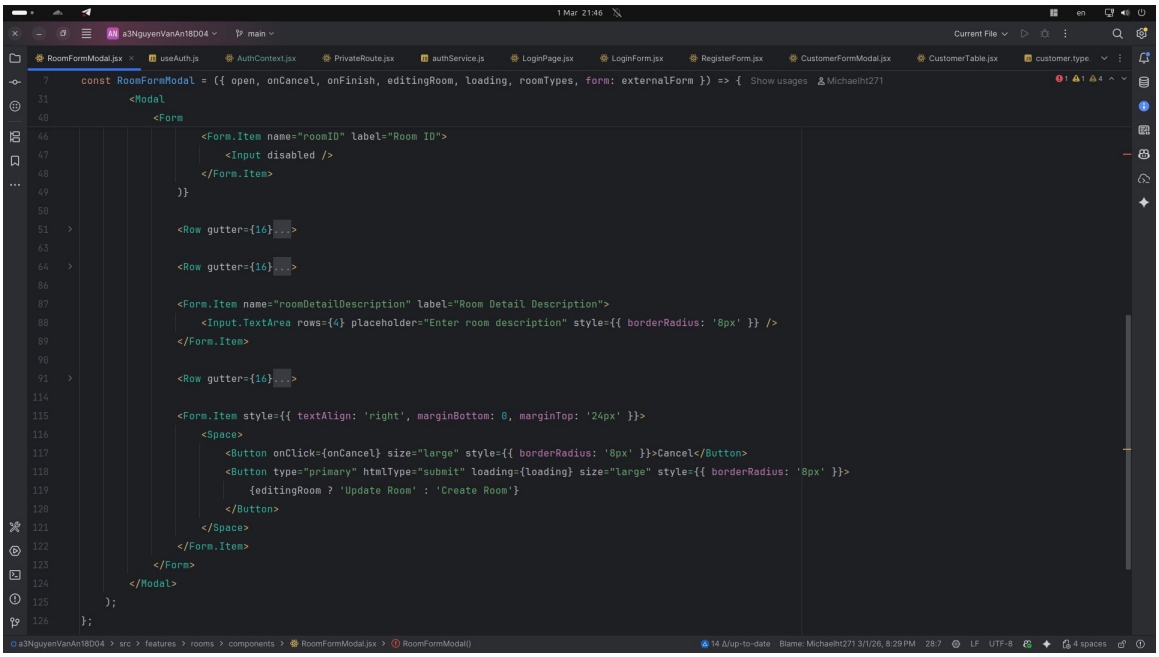


```jsx
import ...   Michaelht271, Today • finish as3

const { Text : ForwardRefExoticComponent<...> , Paragraph : ForwardRefExoticComponent<...> } = Typography;

const RoomCard = ({ room, onBook }) => {  Show usages  Michaelht271 *
  if (!room) return null;

  const roomNumber = room.roomNumber;
  const roomPricePerDay = room.roomPricePerDay || room.roomPrice;
  const roomDetailDescription = room.roomDetailDescription || room.roomDescription;

  return (
    <Card
      hoverable
      style={{ borderRadius: '12px', overflow: 'hidden' }}
      cover={
        <div style={{ height: '200px', backgroundColor: '#f0f2f5', display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
          <HomeOutlined style={{ fontSize: '64px', color: '#bfbfbf' }} />
        </div>
      }
      actions={[
        <Button
          type="primary"
          onClick={() => onBook(room)}
          disabled={room.roomStatus !== 'Available'}
          style={{ borderRadius: '6px' }}
        >
          Book Now
        </Button>
      ]}
```

4.3.3. RoomFormModal

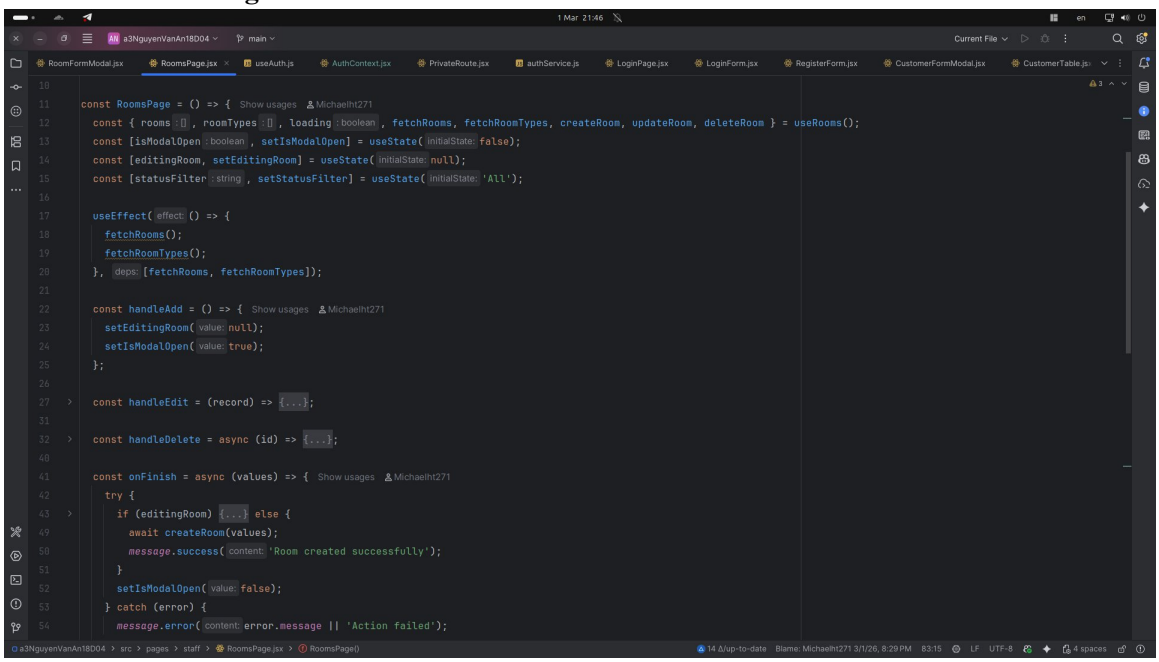


```jsx
import ...

const { Option : OptionFC } = Select;

const RoomFormModal = ({ open, onCancel, onFinish, editingRoom, loading, roomTypes, form: externalForm }) => {  Show usages  Michaelht271
    const [internalForm : FormInstance<any> ] = Form.useForm();
    const form : FormInstance<any>  = externalForm || internalForm;

    useEffect( effect: () => {...}, deps: [open, editingRoom, form]);

    const handleFinish = (values) => {  Show usages  Michaelht271
        onFinish(MAP_FORM_TO_ROOM(values));
    };

    return (
        <Modal
            title={editingRoom ? 'Edit Room' : 'Add New Room'}
            open={open}
            onCancel={onCancel}
            footer={null}
            width={700}
            centered
            style={{ borderRadius: '12px', overflow: 'hidden' }}
        >
            <Form
                form={form}
                layout="vertical"
                onFinish={handleFinish}
            >
                {editingRoom && (
```



```
const RoomFormModal = ({ open, onCancel, onFinish, editingRoom, loading, roomTypes, form: externalForm }) => {
        <Modal
            <Form
                    <Form.Item name="roomID" label="Room ID">
                        <Input disabled />
                    </Form.Item>
                )}

                <Row gutter={16}...>

                <Row gutter={16}...>

                <Form.Item name="roomDetailDescription" label="Room Detail Description">
                    <Input.TextArea rows={4} placeholder="Enter room description" style={{ borderRadius: '8px' }} />
                </Form.Item>

                <Row gutter={16}...>

                <Form.Item style={{ textAlign: 'right', marginBottom: 0, marginTop: '24px' }}>
                    <Space>
                        <Button onClick={onCancel} size="large" style={{ borderRadius: '8px' }}>Cancel</Button>
                        <Button type="primary" htmlType="submit" loading={loading} size="large" style={{ borderRadius: '8px' }}>
                            {editingRoom ? 'Update Room' : 'Create Room'}
                        </Button>
                    </Space>
                </Form.Item>
            </Form>
        </Modal>
    );
};
```

4.3.4. RoomPage

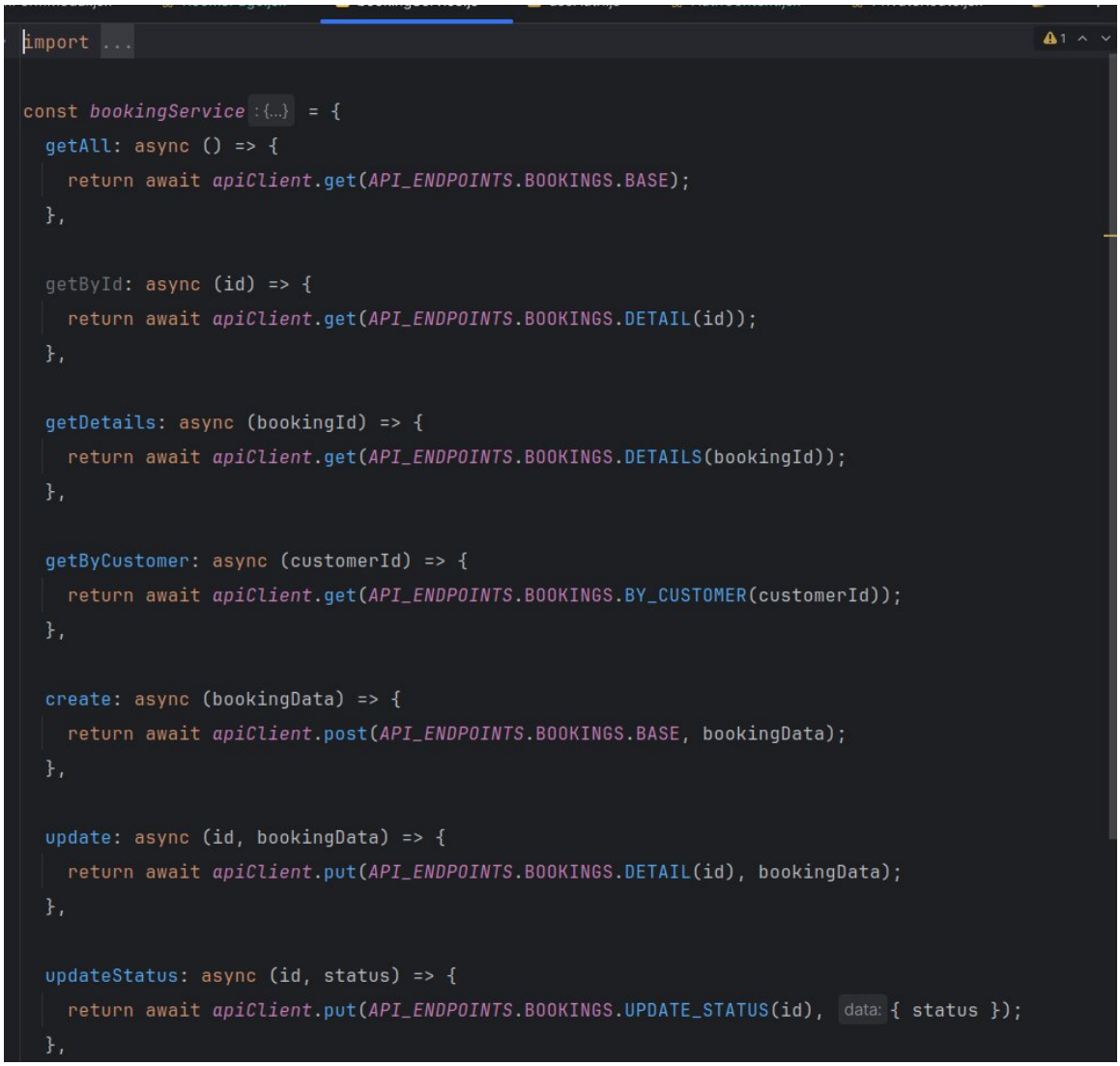


```
const RoomsPage = () => {
  const { rooms = [], roomTypes = [], loading = boolean, fetchRooms, fetchRoomTypes, createRoom, updateRoom, deleteRoom } = useRooms();
  const [isModalOpen = boolean, setIsModalOpen] = useState(initialState: false);
  const [editingRoom, setEditingRoom] = useState(initialState: null);
  const [statusFilter = string, setStatusFilter] = useState(initialState: 'All');

  useEffect(effect: () => {
    fetchRooms();
    fetchRoomTypes();
  }, deps: [fetchRooms, fetchRoomTypes]);

  const handleAdd = () => {
    setEditingRoom(value: null);
    setIsModalOpen(value: true);
  };

  const handleEdit = (record) => {...};

  const handleDelete = async (id) => {...};

  const onFinish = async (values) => {
    try {
      if (editingRoom) {...} else {
        await createRoom(values);
        message.success(content: 'Room created successfully');
      }
      setIsModalOpen(value: false);
    } catch (error) {
      message.error(content: error.message || 'Action failed');
```

4.4. Feature: Quản lý Đặt phòng (Booking Management)

4.4.1. bookingService.js

- Chức năng: Xử lý logic đặt phòng (createBooking) và lấy lịch sử (getBookingHistory).

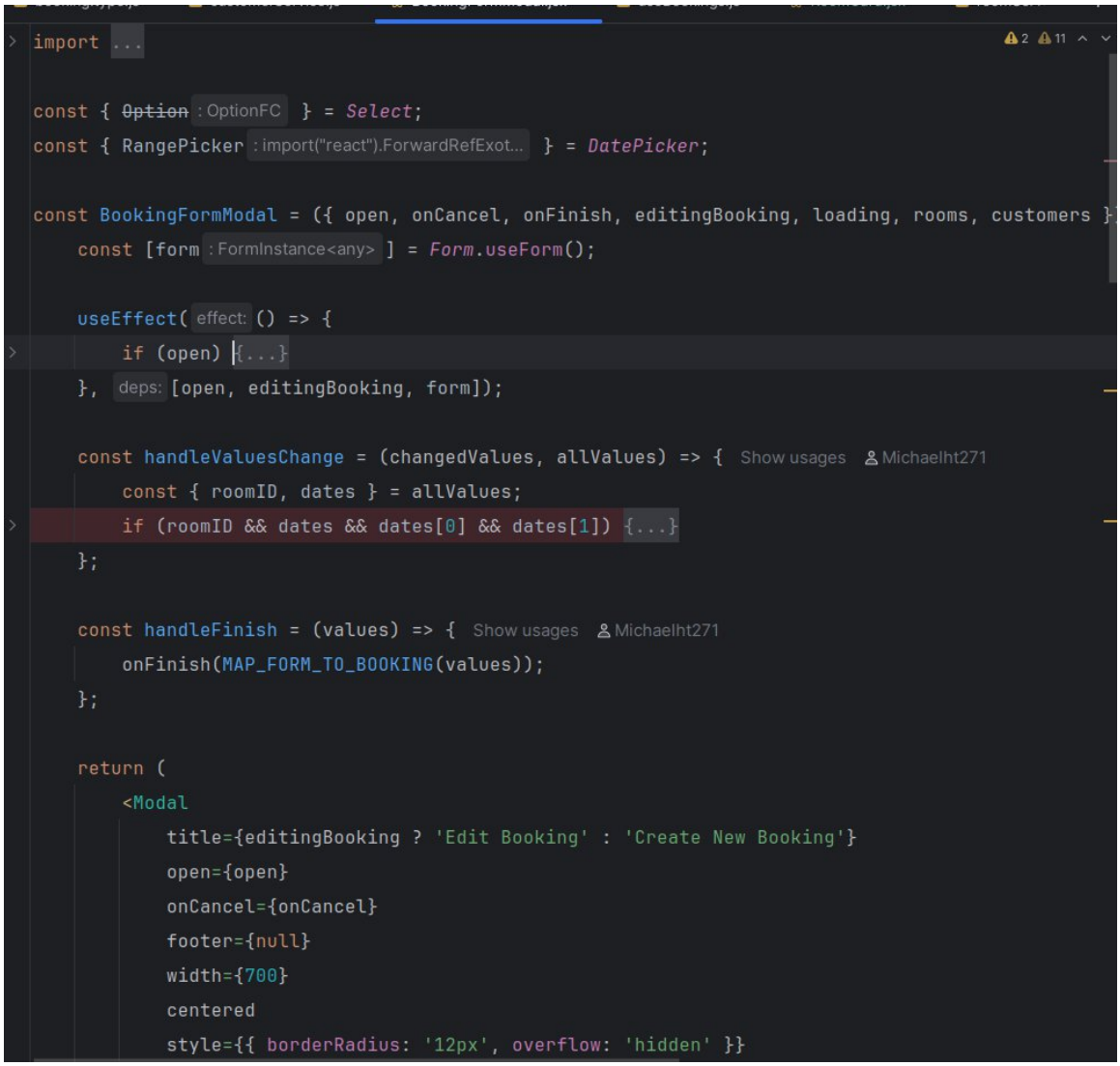


```
import ...

const bookingService :{...} = {
  getAll: async () => {
    return await apiClient.get(API_ENDPOINTS.BOOKINGS.BASE);
  },

  getById: async (id) => {
    return await apiClient.get(API_ENDPOINTS.BOOKINGS.DETAIL(id));
  },

  getDetails: async (bookingId) => {
    return await apiClient.get(API_ENDPOINTS.BOOKINGS.DETAILS(bookingId));
  },

  getByCustomer: async (customerId) => {
    return await apiClient.get(API_ENDPOINTS.BOOKINGS.BY_CUSTOMER(customerId));
  },

  create: async (bookingData) => {
    return await apiClient.post(API_ENDPOINTS.BOOKINGS.BASE, bookingData);
  },

  update: async (id, bookingData) => {
    return await apiClient.put(API_ENDPOINTS.BOOKINGS.DETAIL(id), bookingData);
  },

  updateStatus: async (id, status) => {
    return await apiClient.put(API_ENDPOINTS.BOOKINGS.UPDATE_STATUS(id), data: { status });
  },
```

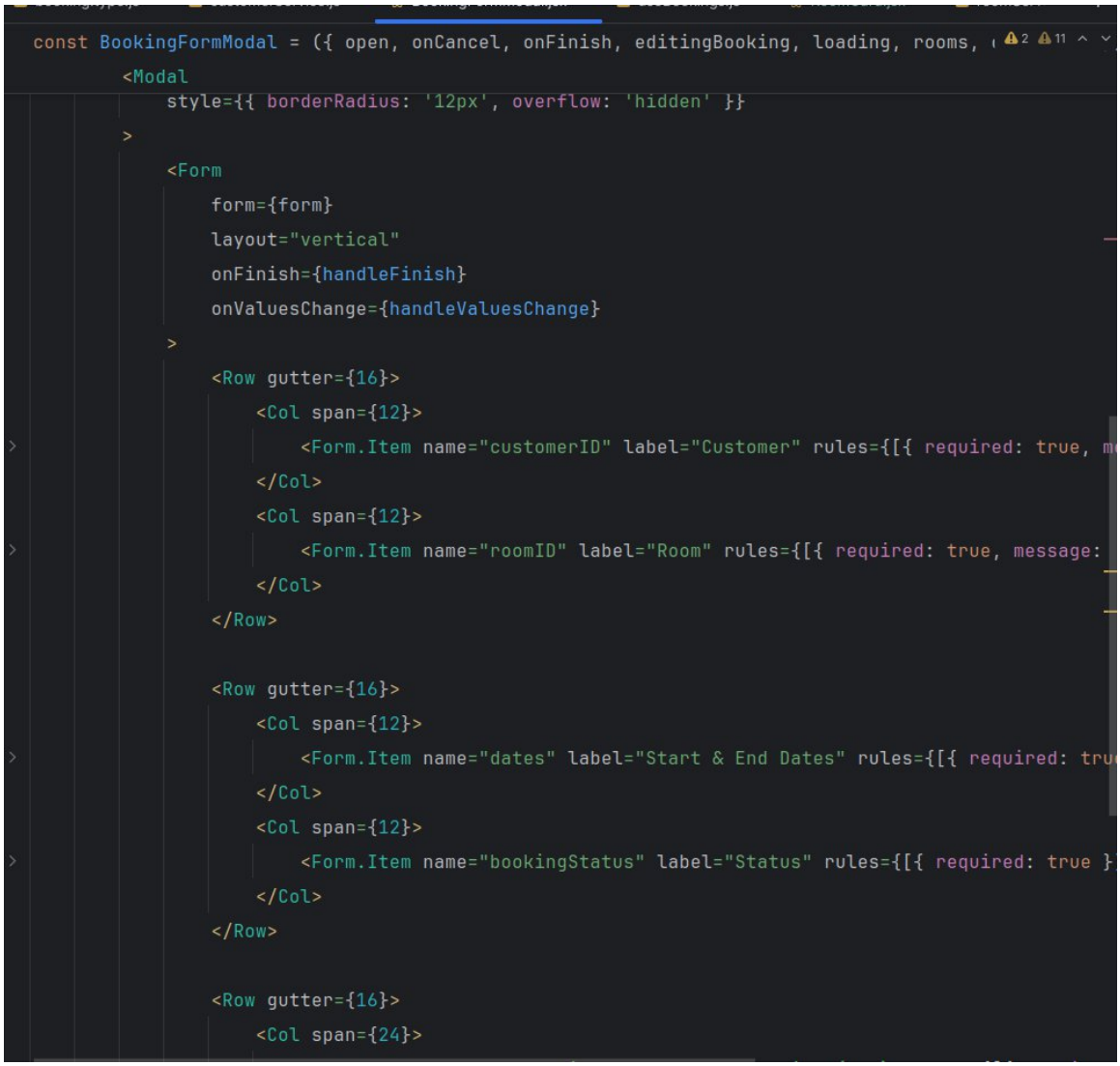
4.4.2. BookingFormModal.jsx

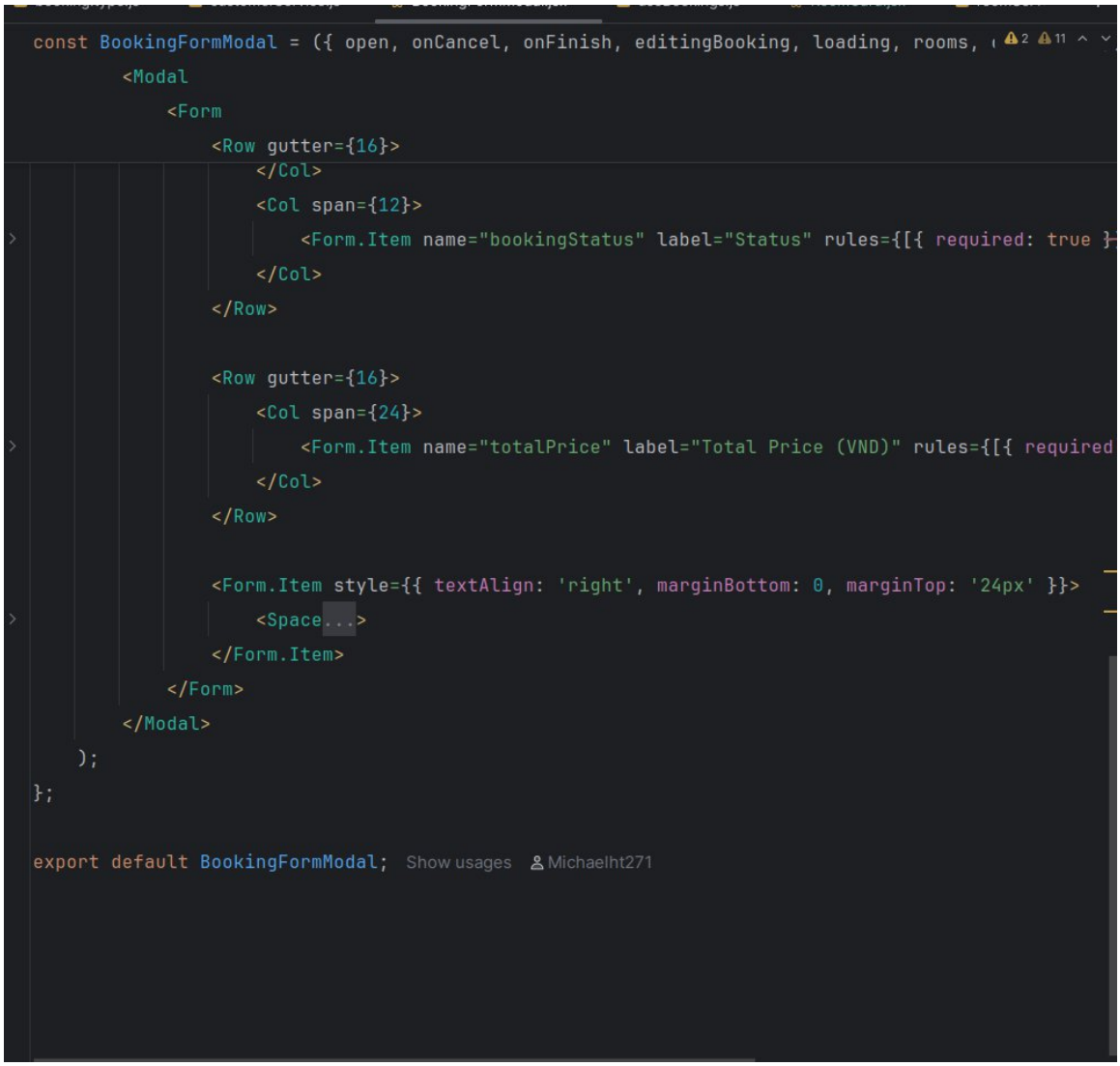
- Chức năng: Form cho phép khách hàng chọn ngày nhận phòng, ngày trả phòng và chọn nhiều phòng cùng lúc.

- Logic: Tự động tính toán tổng tiền dựa trên số ngày và đơn giá phòng.



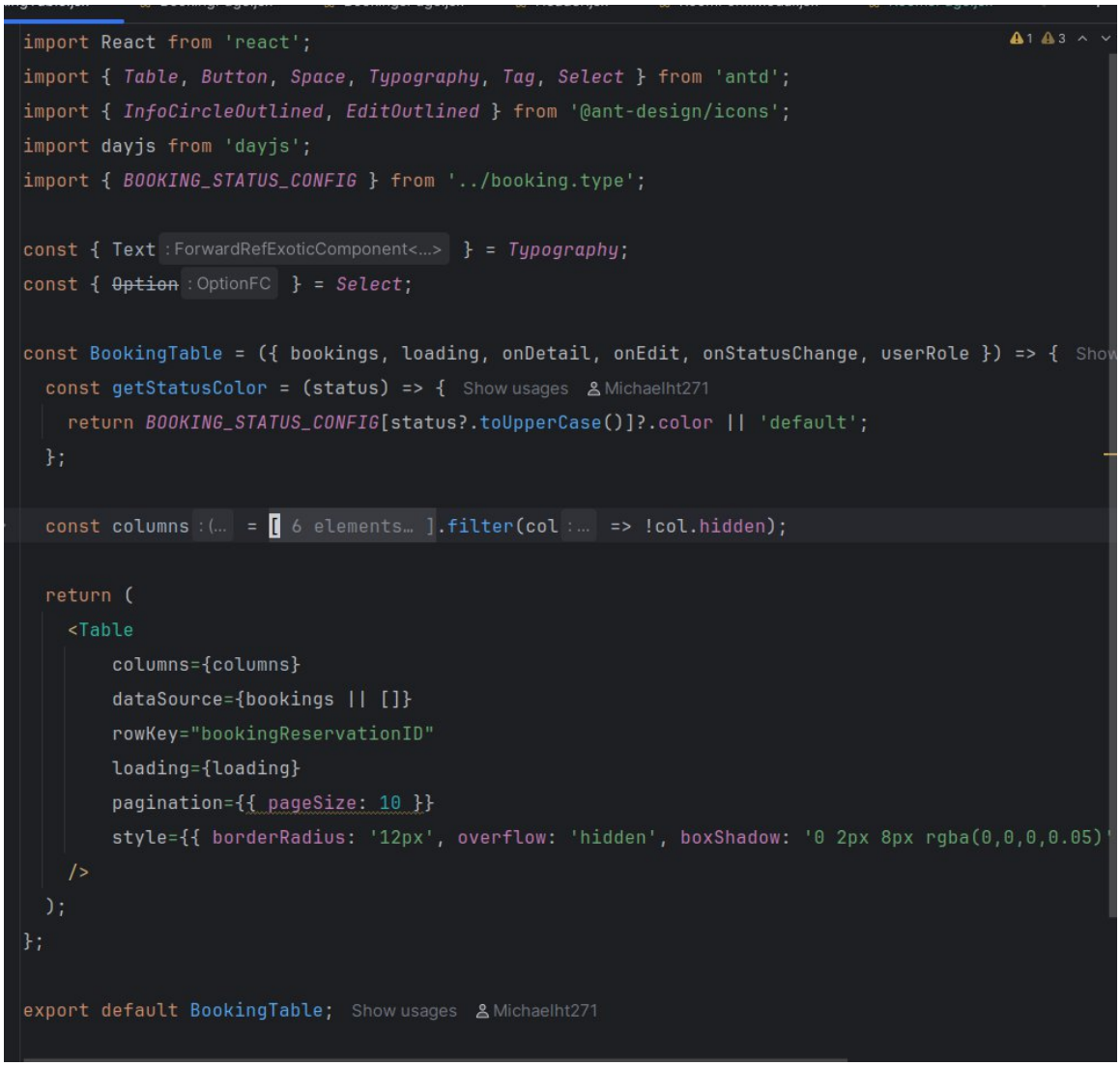
```
import ...

const { Option : OptionFC } = Select;
const { RangePicker : import("react").ForwardRefExot... } = DatePicker;

const BookingFormModal = ({ open, onCancel, onFinish, editingBooking, loading, rooms, customers }
    const [form : FormInstance<any> ] = Form.useForm();

    useEffect( effect: () => {
        if (open) {...}
    }, deps: [open, editingBooking, form]);

    const handleValuesChange = (changedValues, allValues) => {  Show usages  Michaelht271
        const { roomID, dates } = allValues;
        if (roomID && dates && dates[0] && dates[1]) {...}
    };

    const handleFinish = (values) => {  Show usages  Michaelht271
        onFinish(MAP_FORM_TO_BOOKING(values));
    };

    return (
        <Modal
            title={editingBooking ? 'Edit Booking' : 'Create New Booking'}
            open={open}
            onCancel={onCancel}
            footer={null}
            width={700}
            centered
            style={{ borderRadius: '12px', overflow: 'hidden' }}
```


const BookingFormModal = ({ open, onCancel, onFinish, editingBooking, loading, rooms,
<Modal
style={{ borderRadius: '12px', overflow: 'hidden' }}
>
<Form
form={form}
layout="vertical"
onFinish={handleFinish}
onValuesChange={handleValuesChange}
>
<Row gutter={16}>
<Col span={12}>
<Form.Item name="customerID" label="Customer" rules={[{ required: true,
</Col>
<Col span={12}>
<Form.Item name="roomID" label="Room" rules={[{ required: true, message:
</Col>
</Row>
<Row gutter={16}>
<Col span={12}>
<Form.Item name="dates" label="Start & End Dates" rules={[{ required:
</Col>
<Col span={12}>
<Form.Item name="bookingStatus" label="Status" rules={[{ required: true }
</Col>
</Row>
<Row gutter={16}>
<Col span={24}>



4.4.3. . BookingTable.jsx (Staff View)

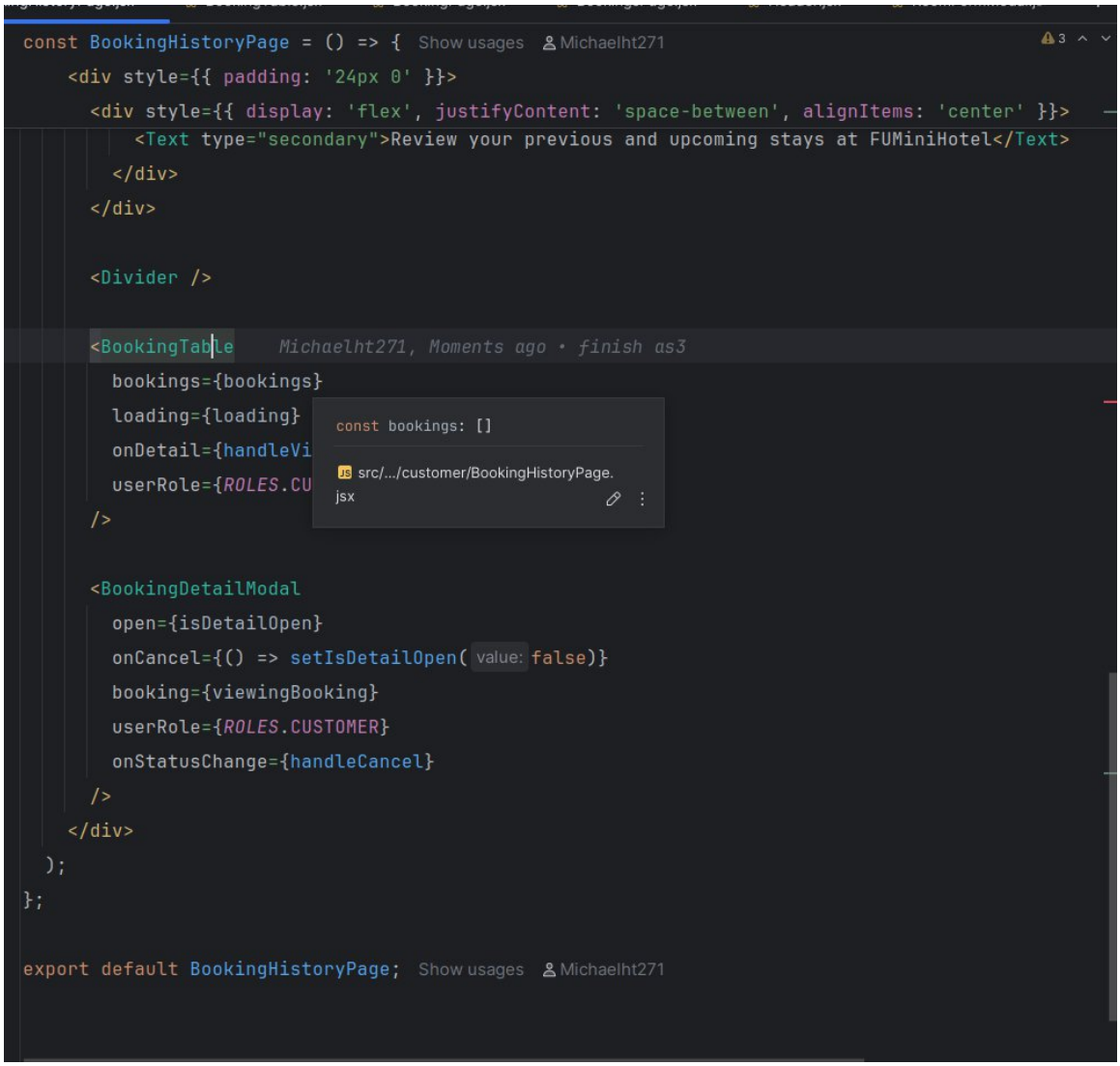
- Chức năng: Cho phép nhân viên quản lý trạng thái các đơn đặt phòng (Confirm, Check-in, Check-out).



```jsx
import React from 'react';
import { Table, Button, Space, Typography, Tag, Select } from 'antd';
import { InfoCircleOutlined, EditOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { BOOKING_STATUS_CONFIG } from '../booking.type';

const { Text : ForwardRefExoticComponent<...> } = Typography;
const { Option : OptionFC } = Select;

const BookingTable = ({ bookings, loading, onDetail, onEdit, onStatusChange, userRole }) => {
  const getStatusColor = (status) => {  Show usages  Michaelht271
    return BOOKING_STATUS_CONFIG[status?.toUpperCase()]?.color || 'default';
  };

  const columns : (... = [ 6 elements… ].filter(col : … => !col.hidden);

  return (
    <Table
        columns={columns}
        dataSource={bookings || []}
        rowKey="bookingReservationID"
        loading={loading}
        pagination={{ pageSize: 10 }}
        style={{ borderRadius: '12px', overflow: 'hidden', boxShadow: '0 2px 8px rgba(0,0,0,0.05)'
    />
  );
};

export default BookingTable;  Show usages  Michaelht271
```

4.4.4. BookingHistoryPage.jsx

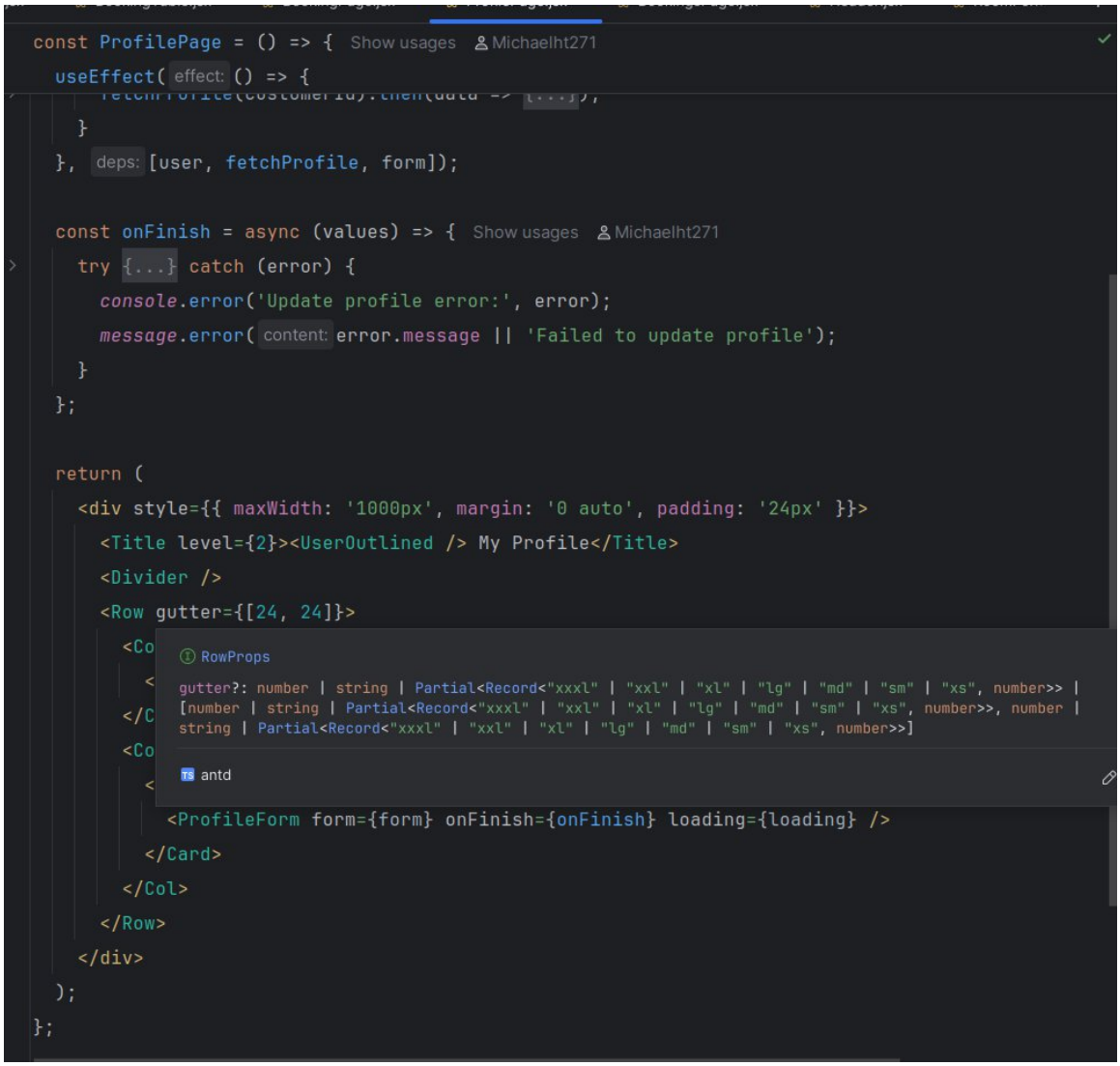
- Chức năng: Hiển thị danh sách các lần đặt phòng trước đây của chính khách hàng đó.


const BookingHistoryPage = () => { Show usages Michaelht271
<div style={{ padding: '24px 0' }}>
<div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
<Text type="secondary">Review your previous and upcoming stays at FUMiniHotel</Text>

<Divider />
<BookingTable Michaelht271, Moments ago • finish as3
bookings={bookings}
loading={loading}
onDetail={handleVi
userRole={ROLES.CU
/>
const bookings: []
src/.../customer/BookingHistoryPage.jsx
<BookingDetailModal
open={isDetailOpen}
onCancel={() => setIsDetailOpen(value: false)}
booking={viewingBooking}
userRole={ROLES.CUSTOMER}
onStatusChange={handleCancel}
/>

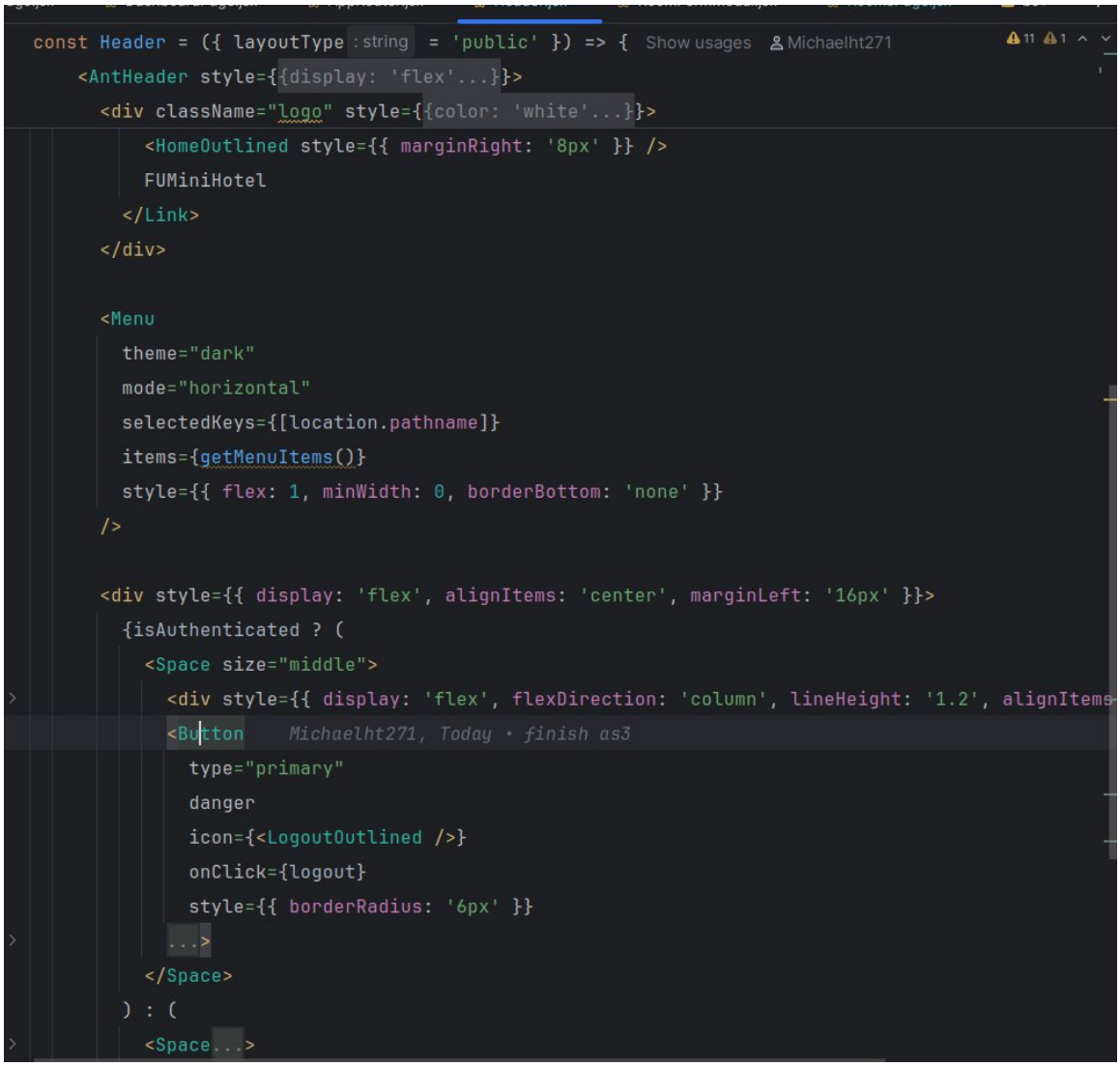
);
};
export default BookingHistoryPage; Show usages Michaelht271

4.5. Profile Page



4.6. Other

4.6.1. Header



4.6.2. Footer

```jsx
import ...

const { Footer: AntFooter : ForwardRefExoticComponent<...> } = Layout;
const { Text : ForwardRefExoticComponent<...> , Title : ForwardRefExoticComponent<...> , Link : ForwardRefExoticCompon

const Footer = () => {  Show usages  Michaelht271
  return (
    <AntFooter style={{
      backgroundColor: '#f0f2f5',
      padding: '48px 50px',    Michaelht271, Today • finish as3
      color: '#595959',
      boxShadow: '0 -2px 8px rgba(0,0,0,0.05)'
    }}>
      <Row gutter={[32, 32]}>
        <Col xs={24} md={8}>
          <Title level={4} style={{ marginBottom: '24px' }}>
            <HomeOutlined style={{ marginRight: '8px' }} />
            FUMiniHotel
          </Title>
          <Paragraph>
            The best hotel management system for FPT University students.
            Book your rooms quickly and securely with our mini hotel system.
          </Paragraph>
          <Space size="large" style={{ fontSize: '20px', marginTop: '16px' }}>
            <Link href="#"><FacebookOutlined /></Link>
            <Link href="#"><TwitterOutlined /></Link>
            <Link href="#"><InstagramOutlined /></Link>
          </Space>
        </Col>
```

```jsx
const Footer = () => {  Show usages  Michaelht271 *
    <AntFooter style={{
      <Row gutter={[32, 32]}>
        <Col xs={24} md={8}>
          <Space direction="vertical" style={{ width: '100%' }}>
          </Space>
        </Col>
      </Row>

      <Divider style={{ margin: '32px 0' }} />

      <div style={{ textAlign: 'center' }}>
        <Text type="secondary">
          FUMiniHotel ©{new Date().getFullYear()} Created by a3NguyenVanAn18D04
        </Text>
      </div>
    </AntFooter>
  );
};

const Paragraph = ({ children }) => (  Show usages  Michaelht271
  <div style={{ marginBottom: '16px', lineHeight: '1.6' }}>
    <Text type="secondary">{children}</Text>
  </div>
);

export default Footer;  Show usages  Michaelht271
```